TECHNISCHE UNIVERSITÄT
CHEMNITZ

Multi-Agent Template Filling using LLMs: Design and Comparative Evaluation with Speech Input Case Study

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc. Web Engineering

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Huzefa Ismail Jadliwala
Student ID: 806286
Date: 15.12.2025

Internal Supervisor: Verena Traubinger
External Supervisor: Dr. Christoph Carl Kling
Internal Examiner: Dr.-Ing. Sebastian Heil

Abstract

Filling templates or forms is a frequent and important task in many real-life situations. Examples include shift reports in manufacturing, patient intake forms in healthcare, and administrative documentation across various industries. However, converting unstructured spoken language into structured templates remains challenging, particularly when dealing with ambiguous or noisy inputs. Available form filling solutions struggle with ambiguous or noisy inputs, lack adaptability, and provide limited transparency. To address these limitations, the focus of this thesis is on developing a modular, multi-agent system for speech-driven template filling using large language models (LLMs). This thesis presents Invox, a system that decomposes the task into five subtasks—transcription interpretation, retrieval of closely related past templates, field mapping, value inference, and result verification—handled by specialized LLM agents. This thesis: (i) reviews existing solutions for automated form filling and identifies their key limitations; (ii) builds on existing techniques from LLM applications to create novel multi-agent approaches, implementing and comparing four architectural strategies: Single-LLM Full Input, Single-LLM One Field, Multi-LLM Full Input, and Multi-LLM One Field; (iii) evaluates these approaches using the MUC-4 dataset from literature, focusing on accuracy, consistency, latency, cost-efficiency, and modularity; and (iv) derives concrete instructions for the implementation of template filling applications, which will be of increasing importance as voice-driven interfaces become more prevalent. The Invox system integrates Whisper for transcription and LLMs including GPT-4 and Gemini for template filling. Evaluation results reveal distinct trade-offs among the architectural strategies across accuracy, consistency, latency, cost-efficiency, and modularity, offering practical guidance for selecting appropriate approaches based on application requirements.

Keywords: Large Language Models (LLMs), Multi-Agent Systems, Template Filling, Retrieval-Augmented Generation (RAG), Information Extraction, Prompt Engineering, Natural Language Understanding, Speech-to-Text Processing

Acknowledgment

I would like to express my deepest gratitude to all those who supported and guided me throughout the development of this Master's thesis.

First and foremost, I am sincerely thankful to my internal supervising tutor, Verena Traubinger, for her dedicated support, timely feedback, and constructive suggestions throughout the entire process. Her patience, encouragement, and willingness to engage in critical discussions greatly contributed to both the conceptual and practical aspects of my work.

I would also like to extend my heartfelt appreciation to my external supervising tutor, Dr. Christoph Carl Kling, for his expert guidance and valuable insights. His technical knowledge and practical experience provided a crucial perspective that helped refine and strengthen the foundation of this thesis. His continuous motivation and thoughtful remarks challenged me to think critically and aim for a high-quality outcome.

Furthermore, I would like to thank the faculty members and academic staff of the program for creating an intellectually stimulating environment that inspired me to pursue research in this domain.

Special thanks also go to my friends and colleagues for their encouragement, helpful discussions, and emotional support during challenging times. I am especially grateful to my family for their unwavering belief in me, their unconditional support, and their motivation throughout my academic journey.

This thesis would not have been possible without the contributions and support of all these individuals. To each of them, I extend my sincere thanks.

Task Description

In domains such as healthcare, manufacturing, and administration, converting unstructured spoken language into structured forms is a critical yet challenging task. Traditional approaches using large language models (LLMs) typically employ monolithic systems that attempt to complete the entire template filling process in a single step. While computationally efficient, these systems are often brittle: they struggle with incomplete or ambiguous inputs, lack modularity for domain adaptation, and provide limited transparency when dealing with domain-specific terminology, noisy transcriptions, or real-world operational conditions.

This thesis proposes a modular, multi-agent approach to template filling, implemented through the **Invox** system. Instead of treating the problem as a single LLM task, Invox decomposes it into subtasks—transcription interpretation, field mapping, value inference, and result verification—each handled by dedicated LLM agents. The system leverages state-of-the-art components such as Whisper for transcription, GPT-4 and Claude for reasoning, and DeepSeek for semantic validation. Five architectural strategies are explored: (1) Single-Pass Full Input, (2) Iterative Single-Field Processing, (3) Multi-LLM Consensus (Full), (4) Multi-LLM Consensus (Iterative), and (5) Hybrid Refinement. These differ in terms of prompt structure, processing granularity, and verification mechanisms. All approaches are evaluated using the same criteria: accuracy, consistency, latency, cost-efficiency, and modularity. Datasets include the benchmark MUC-4 corpus and a real-world industrial dataset from steel manufacturing shift reports. The goal is not only to assess individual method performance, but to better understand the trade-offs introduced by modular, agent-based LLM systems in real-world deployment contexts.

The objective of this thesis is the creation of a solution or the combination of existing approaches to solve the problem described above of filling out templates with the help of Large Language Models. This comprises the following parts. An analysis of the state of the art on LLMs and prompt engineering, multi-agent systems, tools for filling out templates, and other relevant work. The thesis includes the implementation and comparison of five different approaches for a solution. A suitable evaluation should be conducted, where the approaches are tested on datasets regarding a set of benchmarks and the elicited requirements based on the literature research.

Contents

Abstract ii

Acknowledgment iii

Task Description iv

Contents v

List of Figures x

List of Tables xiv

List of Abbreviations xvi

1. Introduction 1

1.1. Problem Description 2

1.2. Motivating Scenario 3

1.3. Scope of the Thesis 5

1.4. Objectives 5

1.5. Thesis Outline 6

2. Analysis 7

2.1. Requirements 7

2.1.1. R1: Consistency 9

2.1.2. R2: Information Extraction 11

2.1.3. R3: Transparency 14

2.1.4. R4: User Correction 15

2.1.5. R5: Learning and Adaptation 16
2.1.6. R6: Usability 18
2.2. Related Work 19
2.2.1. Template Filling and Structured Information Extraction . . . 20
2.2.2. Prompt Engineering for Information Extraction 24
2.2.3. Multi-Agent LLM Systems 27
2.2.4. Overall Evaluation and Gaps 30

3. Concept 31
3.1. Concept Derivation from Analysis Results 32
3.1.1. From Monolithic to Modular Processing 32
3.1.2. Addressing Consistency Through Dedicated Normalization . . 32
3.1.3. Retrieval-Augmented Generation for Domain Adaptation . . . 33
3.1.4. Verification for Completeness and Cross-Field Consistency . . 33
3.1.5. Modularity as a Foundation for Flexibility 34
3.2. Five-Agent Architecture 34
3.2.1. Speech-to-Text Agent (STT) 35
3.2.2. Retrieval-Augmented Generation Agent (RAG) 35
3.2.3. Information Extraction Agent (IE) 35
3.2.4. Consistency Formatting Agent (CF) 36
3.2.5. Verification Agent (VER) 37
3.2.6. Agent Interfaces and Data Flow 39
3.3. Architectural Strategies 39
3.3.1. Strategy S1: Single-Pass (Full-Input, Single-LLM) 40
3.3.2. Strategy S2: Iterative (Slot-wise, Single-LLM) 41
3.3.3. Strategy S3: Multi-LLM Consensus (Full-Input) 42
3.3.4. Strategy S4: Multi-LLM Consensus (Slot-wise) 43
3.3.5. Summary and Trade-Off Analysis 44
3.4. System Architecture and Design 45
3.4.1. Context View: System Boundaries and External Actors 45

3.4.2. Container View: Internal Component Structure 46
3.4.3. Process View: Workflow and Decision Logic 47

4. Implementation 50
4.1. Technology Stack and Rationale 50
4.2. System Workflow 52
4.3. Agent Implementation 54
4.3.1. Speech-to-Text (STT) Agent 54
4.3.2. Retrieval-Augmented Generation (RAG) Agent 55
4.3.3. Information Extraction (IE) Agent 56
4.3.4. Consistency Formatting (CF) Agent 59
4.3.5. Verification Agent (VER) 59
4.3.6. Agent Orchestration 60
4.4. Strategy Implementation 61
4.4.1. S1 Implementation: Single-Pass Full-Input 61
4.4.2. S2 Implementation: Iterative Single-Field 61
4.4.3. S3 Implementation: Multi-LLM Consensus 61
4.4.4. S4 Implementation: Multi-LLM Per-Field 62
4.4.5. Strategy Orchestration Infrastructure 62
4.5. User Interface 62
4.5.1. Authentication and Template Access 63
4.5.2. Template Filling Interface 63
4.5.3. Template Export 65

5. Evaluation 66
5.1. Dataset and Preprocessing 66
5.1.1. Preprocessing 67
5.1.2. Schema Caveats for Slot Filling 68
5.2. Evaluation Metrics 69
5.2.1. Embedding-Based Accuracy (R2) 69

5.2.2. Decision Quality Under Typed Constraints (R1) 70
5.2.3. Consistency (R1-core): Two-Score Summary 70
5.2.4. Latency and Cost at Inference (R6) 71
5.2.5. User Correction and Human-in-the-Loop Reliability (R4) . . . 71
5.2.6. Modularity and Extensibility (R5) 71
5.3. Experimental Setup 72
5.3.1. Compared Strategies 72
5.3.2. Embedding Models 73
5.3.3. Environment and Settings 73
5.4. Results – LangExtract Baseline (S5) 73
5.5. Results – Strategy S1: Single-Pass Full Input 74
5.6. Results – Strategy S2: Iterative Single-Field 78
5.7. Results – Strategy S3: Multi-LLM Consensus (Full) 83
5.8. Results – Strategy S4: Multi-LLM Per-Field 88
5.9. Comparative Analysis of Strategies 93

6. Conclusion 96
6.1. Summary of Contributions 96
6.2. Key Findings 97
6.3. Limitations 97
6.4. Implications for Practice 98
6.5. Conclusion 98

7. Future Work 99

A. Detailed Implementation Results 100
A.1. Consistency Formatting (CF) Agent 100
A.2. IE Agent Supporting Types 100
A.3. Verification Agent API 101
A.4. Orchestrator Implementation 101
A.5. Strategy Function Implementations 103

B. Detailed Experimental Results **110**

B.1. LangExtract Baseline (S5) 110

B.2. Strategy 1: Single-Pass Full Input 112

B.3. Strategy 2: Iterative Single-Field Processing 113

B.4. Strategy 3: Multi-LLM Consensus (Full Input) 113

B.5. Strategy 4: Multi-LLM Consensus (Iterative) 114

B.6. Strategy Comparison Summary 115

Bibliography **118**

List of Figures

1.1. Invox: From spoken input to structured report 4

2.1. Example interaction with the proposed AI voice form assistant. . . . 9

2.2. Example of error propagation in a classical NLP pipeline: an early mistake in named entity recognition prevents correct relation extraction, leading to incomplete template filling. 22

2.3. Illustration of zero-shot vs. few-shot prompting for template filling: zero-shot uses only task instructions, whereas few-shot adds example pairs that guide the model toward the correct structure, typically improving extraction accuracy. 26

2.4. Comparison of single-model and multi-agent approaches. Single-model systems are simple and efficient but offer little transparency, while multi-agent systems increase cost and complexity in exchange for greater modularity and interpretability. 29

3.1. Overview of Invox's five-agent pipeline. The multi-step flow separates transcription, retrieval, extraction, normalization, and user correction into distinct agents. This modular design addresses key limitations of monolithic LLMs by improving extraction robustness, enforcing consistent formatting, enabling transparency through intermediate outputs, and allowing final user edits before submission. 38

3.2. Strategy: Full-Input, Single-LLM 40

3.3. Strategy: Slot-wise, Single-LLM 41

3.4. Strategy: Full-Input, Multiple-LLM 42

3.5. Strategy: Slot-wise, Multiple-LLM 44

3.6. Context diagram illustrating the system boundary of Invox and its interactions with external actors such as users, authentication services, LLM providers, speech-to-text components, and organizational knowledge sources, highlighting all inputs the system depends on. . . 46

3.7. Container diagram illustrating how requests move through Invox's internal services—STT, RAG, IE, CF, and VER—and how these components interact with external dependencies such as LLM providers, storage layers, and the client-facing API Gateway. 48

3.8. BPMN diagram illustrating the end-to-end workflow of Invox, including input handling, retrieval, extraction, formatting, verification, and the decision logic that governs clarification loops and final template delivery. 49

4.1. INVOX authentication interface with Keycloak integration 63

4.2. Template-filling interface used in Invox. The left panel offers voice recording and chat interaction for describing an incident, while the right panel displays the structured form populated by the pipeline. The layout illustrates how transcription, automated extraction, and user review occur within a unified workflow. 64

5.1. Performance of S1 strategy variants on the MUC-4 subset (N=100). The chart compares zero-shot prompting (S1.0), few-shot prompting on clean text (S1.1), and few-shot prompting on speech-style inputs (S1.2). Few-shot prompting improves key metrics such as OBS, SF1, and FDA while lowering HR and MR, with S1.2 maintaining most benefits despite noisier inputs. 75

5.2. Per-field extraction performance for S1.1 on the MUC-4 subset (N=100). Structured fields such as *incidentDate* and *weapon* show high accuracy, whereas open-ended categories like *incidentType* and *incidentLocation* remain challenging. The pattern illustrates where few-shot prompting is effective and where additional support mechanisms may be required. 75

5.3. Latency statistics for S1 variants (seconds). 76

5.4. Consistency (S1 variants): schema formatting vs. input-aware style. . 78

5.5. Headline metrics for S2 variants on MUC-4 (N=100). Few-shot prompting (S2.1) improves most accuracy-oriented metrics (OBS, SF1, RFA) and lowers hallucination rates compared to the zero-shot baseline (S2.0). The speech-style variant (S2.2) performs slightly below S2.1 but remains more stable than S2.0, showing how iterative slot-wise extraction benefits from example-based grounding while still being sensitive to ASR-like noise. 79

5.6. Per-field extraction performance for S2.1 on the MUC-4 subset (N=100). The iterative strategy shows uneven improvements across fields: structured categories such as *weapon* and *incidentStage* benefit most, whereas open-class fields like *incidentType* and *victim* remain comparatively challenging. These results illustrate how slot-wise prompting stabilizes certain fields while exposing residual difficulty in categories requiring interpretation or broader context. 80

5.7. Latency statistics for S2 variants (seconds). The plot compares execution times for S2.0 (no few-shot), S2.1 (few-shot), and S2.2 (few-shot with speech-style input). S2.1 shows the highest latency because few-shot prompts increase token processing, while S2.0 remains the fastest. The speech-style variant (S2.2) reduces extreme latencies relative to S2.1, suggesting that prompt length impacts cost more than input complexity. Overall, the results illustrate the trade-off between improved accuracy from few-shot prompting and the computational overhead of iterative, slot-wise extraction. 81

5.8. Consistency (S2 variants): schema formatting vs. input-aware style. . 83

5.9. Headline metrics for S3 variants on MUC-4 (N=100). The plot compares the multi-model consensus strategy across S3.0 (no few-shot), S3.1 (few-shot), and S3.2 (few-shot with speech-style input). Few-shot prompting (S3.1) offers consistent gains in overall accuracy (OBS), schema adherence (FDA), and fully correct fields (RFA). The speech-style variant (S3.2) remains close to S3.1, showing that consensus aggregation reduces sensitivity to noise introduced by ASR-like phrasing. 84

5.10. Per-field extraction performance for S3.1 on the MUC-4 subset (N=100). The consensus strategy improves stability across fields, with strong performance on structured attributes such as *incidentDate*, *incidentStage*, and *weapon*. More open-ended fields like *incidentType* and *incidentLocation* remain comparatively difficult, reflecting the inherent ambiguity of these categories. Overall, S3.1 delivers a balanced improvement across most fields compared to non-consensus baselines. 85

5.11. Latency statistics for S3 variants (seconds). The multi-model consensus strategy exhibits substantially higher latency than single-model approaches, with S3.1 (few-shot) producing the largest delays due to multiple model calls combined with longer prompts. The speech-style configuration (S3.2) reduces extreme latencies compared to S3.1 but remains significantly slower than the no–few-shot baseline (S3.0). These results highlight the computational cost of consensus-based extraction, where improved robustness and accuracy come at the expense of markedly increased processing time. 86

5.12. Consistency (S3 variants): schema formatting vs. input-aware style. . 87

5.13. Headline metrics for S4 variants on MUC-4 (N=100). Few-shot prompting (S4.1) improves most accuracy-oriented metrics (OBS, SF1, FDA, RFA) but still produces negative EAI values, indicating disagreement across models at the slot level. The speech-style variant (S4.2) performs similarly to S4.1, suggesting that per-field consensus mitigates some ASR-related noise. Overall, S4 offers strong schema adherence but also highlights how per-field aggregation can amplify model inconsistencies. 89

5.14. Per-field extraction performance for S4.1 on the MUC-4 subset (N=100). The per-field multi-LLM strategy performs well on structured fields such as *incidentStage*, *weapon*, and *incidentLocation*, but struggles on more context-dependent fields like *incidentType* and *incidentDate*. This pattern shows how per-field consensus can stabilize well-defined slots while amplifying uncertainty in categories that require broader reasoning. 90

5.15. Latency statistics for S4 variants (seconds). The per-field multi-LLM strategy shows the highest computational cost, with median latencies near 2 minutes and p99 values above 3 minutes. Few-shot prompting (S4.1) slightly lowers extreme latencies relative to S4.0, while the speech-style variant (S4.2) adds more variance. Overall, S4 highlights the trade-off between the robustness of per-field consensus and the considerable runtime required for multiple LLM calls. 90

5.16. Consistency (S4 variants): schema formatting vs. input-aware style. . 92

5.17. Performance comparison across four strategies (few-shot, orig MUC-4). 93

5.18. Cost vs. audio duration: $\text{cost}(D) = \text{base} + 0.60D$ (¢/doc). 94

5.19. Latency distribution across strategies (seconds). 94

5.20. Format compliance and style consistency across strategies. 95

List of Tables

2.1. Evaluation scale for R1: Consistency (example thresholds). 11

2.2. Evaluation scale for R2: Information Extraction Abilities. 13

2.3. Evaluation Scale for R3: Transparency 15

2.4. Evaluation Scale for R4: User Correction 16

2.5. Evaluation Scale for R5: Learning and Adaptation 18

2.6. Evaluation Scale for R6: Usability 19

2.7. Evaluation of template filling and structured extraction solutions against requirements R1–R6. 24

2.8. Evaluation of prompt engineering approaches against requirements R1–R6. 27

2.9. Evaluation of multi-agent approaches against requirements R1–R6. . . 29

3.1. Invox's agent interfaces and data flow. The table summarizes how information moves through the five-agent pipeline by showing the expected input and output of each agent. Reading the table top to bottom reveals the transformation path from raw audio to a fully verified template, highlighting how each agent contributes a distinct and on-overlapping function within the overall system. 39

3.2. Comparison of architectural strategies, showing how different orchestration approaches trade off robustness, latency, and cost depending on the number of LLM calls and the level of error isolation. 44

B.1. Headline metrics for LangExtract on MUC-4 (N=100 documents). . . 110

B.2. Per-field average scores for LangExtract (baseline regime; N=100). . 111

B.3. Embedding metrics (overall) for LangExtract. 111

B.4. Per-field embedding metrics for LangExtract (baseline; text/list fields). 111

B.5. Latency statistics for LangExtract (seconds). 112

B.6. Consistency (two-score) — schema format vs. style uniformity for LangExtract. 112
B.7. Headline metrics for S1 variants on MUC-4 (N=100). 112
B.8. Latency statistics for S1 variants (seconds). 113
B.9. Headline metrics for S2 variants on MUC-4 (N=100). 113
B.10. Latency statistics for S2 variants (seconds). 113
B.11. Headline metrics for S3 variants on MUC-4 (N=100). 113
B.12. Latency statistics for S3 variants (seconds). 114
B.13. Headline metrics for S4 variants on MUC-4 (N=100). 114
B.14. Latency statistics for S4 variants (seconds). 114
B.15. Macro comparison (clean text; few-shot original). 115
B.16. Latency profile (seconds; clean text). 115
B.17. Per-document cost (¢) vs. audio duration D (minutes). Totals follow $\text{cost}(D) = \text{base} + 0.60D$. 115
B.18. Cost decomposition at D=1 minute (¢ per document). 116
B.19. Consistency metrics for S1 variants on MUC-4 (N=100). 116
B.20. Consistency metrics for S2 variants on MUC-4 (N=100). 116
B.21. Consistency metrics for S3 variants on MUC-4 (N=100). 116
B.22. Consistency metrics for S4 variants on MUC-4 (N=100). 117

List of Abbreviations

ACE Automatic Content Extraction

ASR Automatic Speech Recognition

CF Consistency Formatting

Claude Anthropic Claude Language Model

CRF Conditional Random Field

EAI Empty Advantage Index

ERP Enterprise Resource Planning

F1 F1 Score

FDA Fill-Decision Accuracy

Gemini Google Gemini Language Model

GDPR General Data Protection Regulation

GPU Graphics Processing Unit

HMM Hidden Markov Model

HR Hallucination Rate

ISO International Organization for Standardization

LLM Large Language Model

LKI Linearised Knowledge Injection

LoRA Low-Rank Adaptation

MUC-4 Message Understanding Conference 4

NER Named Entity Recognition

NES Non-Empty Score

NLP Natural Language Processing

OBS Overall Baseline Score

OCR Optical Character Recognition

OAuth2 Open Authorization 2.0

OIDC OpenID Connect

ORM Object-Relational Mapping

RAG Retrieval-Augmented Generation

RBAC Role-Based Access Control

SDK Software Development Kit

SF1 Soft F1 Score

SLU Spoken Language Understanding

SLURP Spoken Language Understanding Resource Package

STT Speech-to-Text

tRPC TypeScript Remote Procedure Call

T5 Text-to-Text Transfer Transformer

Whisper OpenAI Whisper Speech Recognition Model

1. Introduction

During a night shift at a manufacturing plant, an operator notices unusual vibrations in a critical machine and must document equipment performance and anomalies for the incoming team. In a busy emergency room, a doctor treating a patient with chest pain must record symptoms, vitals, medical history, and treatments in a standardized intake form. Although from different domains, both tasks rely on structured information capture to ensure continuity, safety, and compliance.

Across industries—including healthcare, manufacturing, logistics, and customer service—workers routinely populate templates such as forms, logs, and reports. This general task, known as *template filling*, requires mapping unstructured observations into predefined fields. It is essential for documentation quality but also time-consuming and error-prone, especially when workers are fatigued. Manual entries often vary in detail, omit required fields, or contain stylistic inconsistencies that impede downstream analysis.

Template filling appears in many settings, from clinical intake forms [16] and industrial maintenance logs [50] to web automation [7] and customer service platforms [46]. The main challenge is not collecting information but reliably converting unstructured text—or speech—into structured formats. This cognitive effort can be significant, especially after demanding shifts where important details may be forgotten or recorded inconsistently.

AI assistants and modern NLP technologies offer an alternative: users can describe events freely, and AI systems convert these descriptions into structured template fields. This reduces cognitive load, improves consistency, and allows the system to detect missing or contradictory information. In hands-busy environments—such as technicians wearing protective gear or clinicians examining patients—speech-based filling is especially promising [50].

This shift is driven by advances in *automatic speech recognition (ASR)* and *large language models (LLMs)*. ASR systems like Whisper now transcribe speech accurately across accents, noise levels, and informal phrasing [39, 17]. Meanwhile, LLMs such as GPT-4 and Claude can interpret context, extract structured information, and resolve ambiguities in user descriptions [16, 33].

Most current ASR+LLM systems rely on a *monolithic* design in which a single model performs understanding and template filling [46, 7]. Such systems offer simplicity but lack transparency and are difficult to adapt: errors cannot be traced to

specific steps, and new forms often require retraining the entire model [32]. Spoken input further complicates the pipeline—speech is frequently disfluent, unordered, or incomplete [17].

To overcome these limitations, recent work advocates *modular or multi-agent architectures*, where transcription, field identification, value extraction, and validation are handled by separate components [33, 43]. This improves interpretability, maintainability, and adaptability to new domains. Building on this line of research, this thesis presents **Invox**, a modular system for speech-based template filling tailored to realistic industrial conditions.

1.1. Problem Description

Large language models (LLMs) have made automatic form filling more accessible, but most existing systems use a *monolithic design* in which a single model reads the input and generates all fields at once. Examples include zero-shot template generation [16], single-pass webform completion [7], and unified slot-filling pipelines [46]. These approaches work in controlled settings but break down with real-world input, which is often unstructured, incomplete, ambiguous, or contains irrelevant details. Whether the input is text, dialogue, or speech, the core challenge remains: converting loosely structured human communication into rigid template fields.

A major limitation is *fragility*. Monolithic models struggle when users speak naturally, reorder information, use unusual phrasing, or omit details [17, 32]. They may leave fields empty, fill them incorrectly, or produce inconsistent outputs [50, 46].

Another issue is *lack of transparency*. When one model performs every step, it becomes impossible to tell whether an error stems from transcription, understanding, or incorrect field assignment [43]. This makes failures hard to diagnose.

Adaptation is also difficult. Different industries and forms require different field structures and vocabularies, and updating a monolithic system often requires retraining or redesigning the entire pipeline [16, 33].

These problems persist because current research typically evaluates isolated components rather than full systems. Although specific models excel at tasks like extraction or validation, there is no flexible, end-to-end architecture that integrates them reliably. As a result, one-shot systems fail when templates change, input quality varies, or new fields must be added.

To overcome these limitations, a modular and transparent design is needed—one that breaks the task into interpretable steps, improving error detection, debugging, and adaptation across domains [33, 43].

1.2. Motivating Scenario

To illustrate the challenges of template filling in real-world environments, consider a typical shift in the *steel manufacturing industry*.

Steel plant operators work under intense conditions—loud machinery, high heat, and strict production deadlines. At the end of each 8-hour shift, they are expected to submit a *shift report* documenting safety incidents, equipment issues, production figures, and handover notes. These reports are essential for maintaining operational continuity, safety compliance, and accountability. Yet in practice, workers often fill them out hastily—sometimes incompletely or not at all—especially during late-night shifts or when fatigued.

To reduce the burden of manual form filling, workers and organizations have begun exploring various assistive technologies and workarounds. While many organizations have transitioned to digitalized forms accessible via tablets or computers, the fundamental challenge of structuring information remains unchanged—workers still must manually map their observations into predefined fields. Some organizations have experimented with dictation tools or voice recording as intermediate steps, where workers provide free-form descriptions that supervisors later process into structured reports. However, such approaches merely shift the burden rather than solving it: someone must still perform the cognitive work of listening to unstructured accounts and extracting relevant information for each template field. This highlights a critical need for intelligent systems that can assist in the transformation from unstructured human communication to structured data, regardless of the input modality.

Now imagine an intelligent system that could take these raw audio notes, transcribe them, interpret their meaning, and fill in the shift report automatically. For instance, if a worker says:

> "We had a minor safety incident near the blast furnace around 2 p.m.—someone tripped, but no injuries. Conveyor belt on Line B stopped twice, restarted after 10 minutes each time. Production target met—482 tons."

Such a system would ideally extract structured information like:

- **Safety Incident:** Minor, near blast furnace, no injuries
- **Time:** Around 2 p.m.
- **Equipment Issue:** Line B conveyor belt stopped twice
- **Production:** 482 tons

Although this task may seem straightforward, it is far from trivial. Spoken input is often informal, unordered, and filled with vague expressions or filler words. Terms like "blast furnace" or "Line B" reflect domain-specific knowledge that the system must understand. Noise, fatigue, and rushed speech introduce further variability.

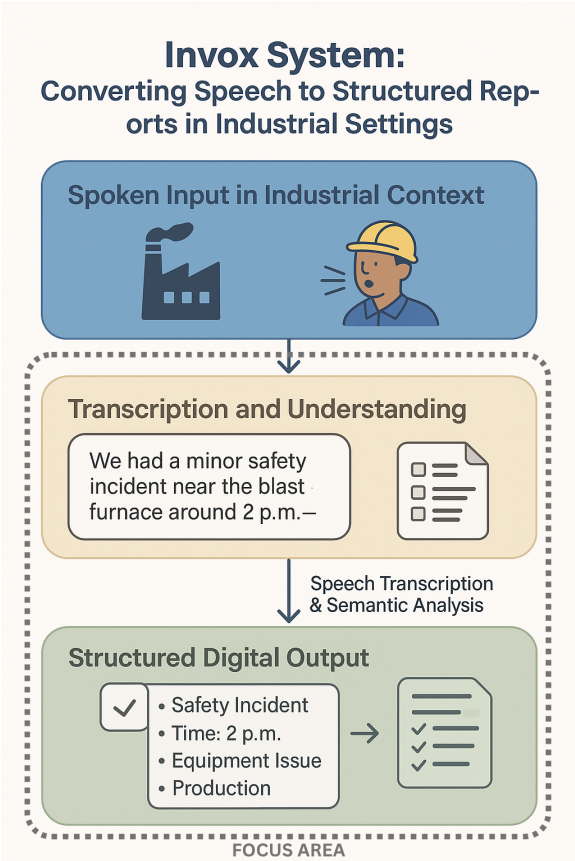

Figure 1.1.: Invox: From spoken input to structured report

This scenario reveals four core challenges in automating template filling:

- **Unstructured input:** Workers communicate naturally through free-form descriptions rather than following predefined template structures, requiring systems to interpret and map loosely organized information into rigid formats.
- **Inconsistency and incompleteness:** Human-generated descriptions vary widely in detail, writing style, and vocabulary, often omitting critical information or including irrelevant details that must be filtered out.
- **Domain-specific language:** Specialized terminology, abbreviations, and jargon must be correctly recognized and interpreted within their contextual meaning, which varies across industries and departments.
- **Lack of integrated systems:** Current solutions rely on isolated AI models rather than flexible, end-to-end systems that can adapt to different template formats, handle multiple input modalities, and evolve with changing organizational needs.

Together, these challenges underscore the need for a more intelligent and adaptable approach. A modular system like **Invox**, capable of processing speech, identifying relevant fields, and generating structured output, offers a scalable solution tailored to the complexities of real industrial environments.

1.3. Scope of the Thesis

This thesis focuses on practical form-filling methods that convert noisy, unstructured spoken input into structured digital records, introducing realistic benchmark datasets and, to the best of our knowledge, the first end-to-end system based on large-scale pretrained models that can be efficiently used in real-world applications.

The primary use case addressed is shift documentation in the steel manufacturing industry. Operators in this domain often record verbal summaries at the end of their shifts, describing production events, safety incidents, and equipment issues. These recordings are typically informal, multilingual, and captured in noisy environments. This makes the use case ideal for evaluating robustness in low-structure, real-world input scenarios [50, 17].

The technical scope of this work includes the use of state-of-the-art automatic speech recognition (ASR) models—specifically OpenAI's Whisper [39]—combined with large language models (LLMs) such as GPT-5, Gemini-2.5-pro, etc. These components are orchestrated in a modular agent-based architecture for handling tasks like transcription interpretation, field-value mapping, and validation. No fine-tuning or training of models is performed; the focus is on system-level orchestration and domain-adapted prompt engineering.

The evaluation is limited to the accuracy and reliability of the structured output produced from recorded input. This thesis does not cover areas such as UI/UX design, live streaming input, conversational turn-taking, or the deployment of on-device inference. Instead, it aims to explore how existing ASR and LLM technologies can be effectively combined into a practical and scalable speech-based form filling system suitable for high-noise industrial domains.

1.4. Objectives

The main objective of this thesis is to design and evaluate a modular system—called **Invox**—that can automatically fill out structured forms using unstructured speech or text input. The system aims to overcome the limitations of current single-model, end-to-end solutions by introducing a flexible, multi-agent approach that separates the task into smaller, specialized components.

To achieve this goal, the following two specific objectives are defined:

1. **Design a modular multi-agent architecture** for template filling that separates the process into subtasks such as transcription, field detection, answer generation, and verification—each handled by a dedicated agent. This design supports flexibility, transparency, and domain adaptability. As part of this design, five distinct prompting strategies are implemented using large language models (LLMs), ranging from single-pass full-input methods to iterative multi-model consensus approaches. These strategies are chosen to reflect different trade-offs in accuracy, latency, cost, and interpretability.
2. **Evaluate the system performance** using a hybrid strategy that combines standardized benchmarking and real-world testing. For benchmarking, the official MUC-4 evaluation protocol [10] is applied, using slot-level precision, recall, and F1-score to compare structured information extraction. Finally, results from all prompting strategies are compared to identify the most effective architectural approach under various real-world conditions.

These objectives guide the development and validation of a system that is not only technically sound but also practical and scalable in industrial settings.

1.5. Thesis Outline

Following this introduction, the thesis begins with a review of related work on template filling, speech-based systems, and large language models. The literature is analyzed to identify current limitations and to derive requirements for a more modular and reliable solution. This provides the conceptual foundation for understanding the need for a system like Invox.

The next chapter introduces the overall system concept. It presents the structure of Invox together with four design strategies that employ LLMs in different ways. After outlining the concept, the implementation chapter describes the software architecture, technology choices, and how core components—such as transcription, retrieval, extraction, and verification—were developed, with an emphasis on modularity and reuse.

The following chapters demonstrate and evaluate the system. A working prototype is shown through screenshots and examples illustrating how speech input is transformed into structured forms. The evaluation then examines system performance on a benchmark dataset across the four architectural strategies, using metrics such as accuracy, latency, consistency, and cost. The thesis concludes by summarizing key findings, discussing limitations, and outlining opportunities for future work.

2. Analysis

This chapter identifies the fundamental requirements for converting natural language into structured templates and critically examines existing approaches. The goal is to establish a systematic basis for evaluating current methods and to derive implications for the design of the proposed framework.

2.1. Requirements

This section outlines the core requirements that a system must fulfill to transform natural language input into structured templates with predefined fields. Such templates are widely used across domains, including healthcare (e.g., patient checklists), manufacturing (e.g., quality inspections), and logistics (e.g., delivery reports).

The primary goal of the system presented in this thesis, named Invox, is to support users in filling out these templates more easily, quickly, and accurately. Instead of using traditional tools like a mouse, keyboard, or touchscreen, users interact with the system using natural language. This input can be typed or spoken aloud—both are valid forms of unstructured data. However, it is important to note that voice-based input is just one way of using the system; the true innovation lies in how the system processes and understands natural human language and translates it into structured form data.

In the introduction, we defined six core requirements (R1–R6) for a practical, LLM-based form-filling system. These requirements capture the desired capabilities (e.g., handling unstructured input, correctly populating forms) as well as non-functional aspects such as robustness, efficiency, and applicability in realistic scenarios.

In this chapter, we analyse the proposed multi-agent architecture with respect to these requirements. For each requirement R1–R6, we outline how it is evaluated, and to what extent the system satisfies it in practice.

However, building such a system is not straightforward. Several challenges must be addressed to ensure the solution is usable, reliable, and trustworthy across real-world scenarios. To clearly define what such a system should achieve, we identify the following six requirements:

R1 – Consistency: The system must be able to transform heterogeneous and informal input—such as voice transcriptions, chat messages, or handwritten notes—into consistently populated template fields. It should normalize wording, tonality, length, and semantic specificity, and enforce formatting standards so that entries are comparable, unambiguous, and suitable for downstream analysis.

R2 – Information Extraction Abilities: The system should accurately identify and assign relevant details from noisy, fragmented, or multilingual input to the correct fields, while leaving irrelevant fields empty. It must recognize when required information is missing, handle contradictions by prompting for clarification, and integrate scattered fragments into coherent entries.

R3 – Transparency: To foster trust and accountability, the system must make its decision-making process visible. It should highlight the input fragments that support each extracted value and provide confidence scores to indicate the certainty of those values. This enables users to verify outputs efficiently, focus on uncertain cases, and ensure auditability in regulated domains.

R4 – User Correction: The system must enable users to manually edit automatically populated template fields. This ensures immediate error resolution without requiring retraining and empowers users with direct control over the accuracy of the final data submitted.

R5 – Learning and Adaptation: The system should improve over time by consolidating knowledge from past interactions and domain resources. It must learn from manually filled or corrected templates as well as from structured sources such as ontologies, glossaries, or rule sets, progressively reducing repetitive errors and improving alignment with domain-specific practices.

R6 – Usability: The system must deliver prompt, near–real-time feedback so interaction feels natural and uninterrupted. This requirement applies to both spoken and typed input (speech is only one use case). Usability is evaluated via end-to-end latency and must be interpreted relative to the declared hardware and deployment profile (e.g., GPU/TPU vs. CPU-only, concurrency, network).

These six requirements were identified based on the challenges discussed in Chapter 1 and are aligned with recent work on natural language processing, conversational interfaces, and intelligent form-filling systems. Each of the following sections will examine one requirement in detail. We explain what the requirement means, why it matters, and illustrate it with examples. For each requirement, we also define assessment standards that help evaluate how well the system satisfies the requirement in practice.

To make these requirements more concrete, Figure 2.1 shows a practical usage scenario of the Invox framework. On the left, the user provides a natural language description, which the assistant processes into a structured template on the right. Some fields are automatically filled with extracted values, while others remain highlighted as

incomplete and require user clarification. This illustration highlights the central challenges addressed by the requirements: ensuring consistent representation of entries (R1), accurately extracting relevant details (R2), providing transparency about what was filled and why (R3), enabling user correction of missing or incorrect fields (R4), creating a foundation for gradual learning and adaptation over time (R5), and maintaining responsiveness so updates appear promptly during interaction (R6).

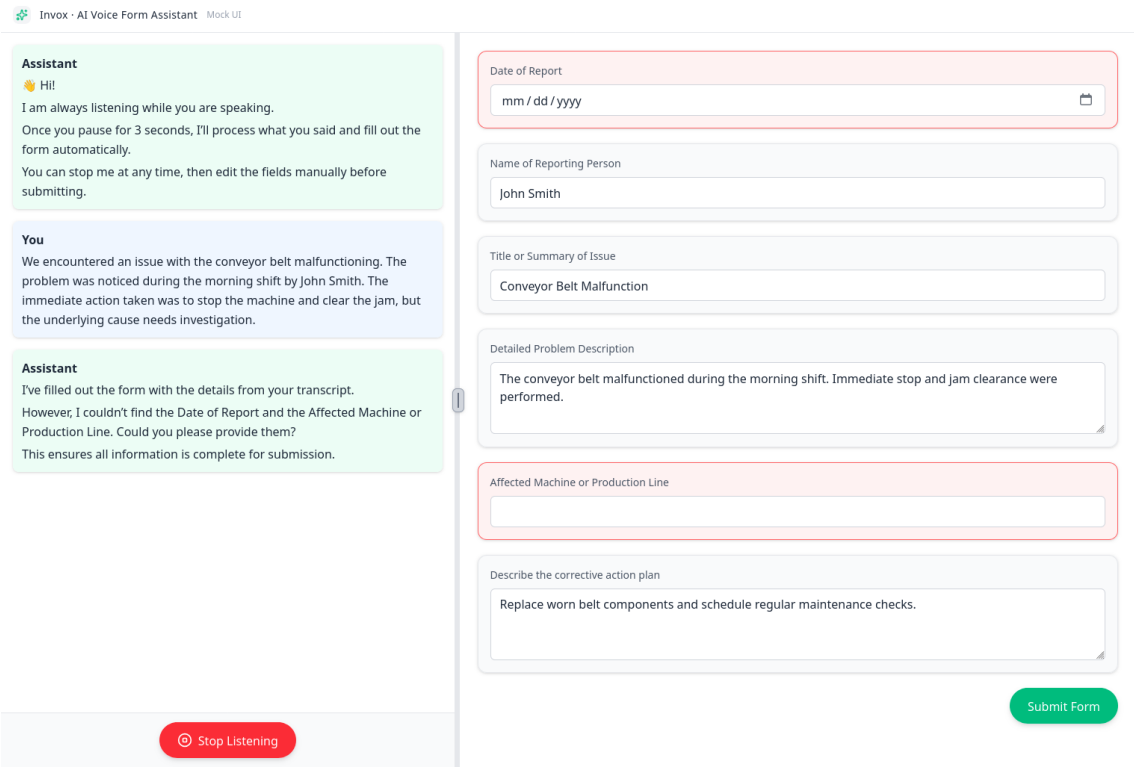

Figure 2.1.: Example interaction with the proposed AI voice form assistant.

On the left, the user provides an unstructured spoken description of a conveyor belt malfunction, which is transcribed in the chat interface. On the right, the system has automatically populated the corresponding incident report form with structured fields such as the reporting person, issue title, problem description, and corrective action. Fields that cannot be reliably inferred from the input (e.g., *Date of Report* and *Affected Machine or Production Line*) are left empty, highlighted, and explicitly requested from the user. This illustrates how the framework maps free-form input onto a structured template while interactively resolving missing information.

2.1.1. R1: Consistency

Consistency refers to the framework's ability to transform heterogeneous and unstructured inputs—such as transcribed voice recordings, instant messages between

employees, or informal written notes—into uniformly populated templates with standardized styling and formatting.

Unlike free-text logs, templates require values that maintain consistent wording patterns, register, length conventions, and structural formatting. This is a core aspect of what the research field calls "authorship attribution" in the context of LLMs, where the goal is to identify a consistent stylistic "fingerprint" from a given system [25]. Prior work in conversational systems has shown that a lack of normalization makes it difficult to align user inputs across contexts [32], while studies in accessibility highlight how inconsistent representations reduce clarity and comparability for end users [12].

As illustrated in Figure 2.1, an ideal solution should ensure that once information is extracted, the resulting field values are expressed in a stylistically consistent way. This means that answers follow standardized wording patterns, tone, formatting conventions, and structural organization. For example, dates should always appear in the same format, technical issues should be described with consistent terminology and register, and descriptive fields should maintain comparable stylistic patterns.

This requirement is especially critical because heterogeneous sources of input inevitably produce stylistically inconsistent outputs if left unchecked. In industrial contexts, for example, one technician might describe a machine malfunction as a "slight issue," another as a "minor fault," and a third as a "mechanical malfunction." Even if these phrases convey similar information, their stylistic inconsistency complicates automated reporting and trend analysis [35]. In healthcare, freeform reports such as "bad cough," "patient has a cold," or "respiratory illness" may differ in register and formality. Medical informatics research has emphasized that without stylistic normalization, such variations undermine clinical decision support [19, 26].

In practice, consistency operates across multiple dimensions of stylistic representation. At the level of wording, normalization ensures that common phrases follow standardized terminology patterns. At the level of tonality, consistency harmonizes register and formality, replacing subjective expressions such as "slight issue" with standardized formulations such as "minor mechanical malfunction." At the level of length, entries maintain comparable levels of detail. At the level of formatting, structural conventions ensure that dates, identifiers, and technical codes follow uniform patterns—for instance, "August 16th" is consistently represented as "08/16/2025."

The advantages of consistent template filling are multifold. Consistency improves readability, as users can process and compare entries more quickly when values follow uniform stylistic patterns. It reduces ambiguity by replacing stylistically divergent expressions with standardized equivalents. It enhances comparability: homogeneous field values are easier to query, index, and aggregate, facilitating automated reporting and large-scale data analysis.

For evaluation, we use exact match or F1 scores for structured fields (such as identifiers, categories, and dates) and n-gram-based stylometric analysis for free-text

fields to assess stylistic alignment. Other similar benchmarks measuring formatting consistency and stylistic similarity can also be used depending on the domain and available resources. To evaluate the stylistic consistency of the system's output, we employ Cosine Similarity of n-gram distributions between each system's output and the golden benchmark. N-grams, which are contiguous sequences of words, provide a quantitative measure of the system's syntactic and stylistic patterns. Unlike Jaccard similarity, which only considers the presence or absence of n-grams, Cosine Similarity accounts for their frequency, making it sensitive to differences in writing style, register, and structural patterns. A higher similarity score indicates that the system's output is stylistically aligned with the human-authored standard.

Visual Score	Interpretation (with example thresholds)
●	**High consistency.** Structured fields follow formatting standards in most cases (e.g., macro exact match/F1 $\geq$ 0.80), and free-text fields show strong stylistic alignment (e.g., Cosine Similarity $\geq$ 0.70).
◕	**Medium consistency.** Structured fields show moderate formatting accuracy (e.g., macro exact match/F1 in $[0.65, 0.80)$), and free-text fields achieve moderate stylistic similarity (e.g., Cosine Similarity in $[0.55, 0.70)$).
◔	**Low consistency.** Structured fields often deviate from formatting standards (e.g., macro exact match/F1 in $[0.50, 0.65)$), and free-text fields show weak stylistic alignment (e.g., Cosine Similarity in $[0.40, 0.55)$).
○	**No consistency.** Structured formatting is very inconsistent (e.g., macro exact match/F1 $<$ 0.50) and free-text stylistic similarity is weak (e.g., Cosine Similarity $<$ 0.40).

Table 2.1.: Evaluation scale for R1: Consistency (example thresholds).

To summarize, consistency ensures that heterogeneous, informal, or stylistically varied user inputs are transformed into standardized, uniformly formatted template entries. By enforcing normalization across wording patterns, tonality, length conventions, and formatting standards, and by measuring stylistic alignment through n-gram-based similarity metrics, the system improves readability and ensures that structured data maintains uniform presentation standards.

2.1.2. R2: Information Extraction

Information extraction refers to the framework's ability to accurately identify and populate relevant template fields from unstructured or semi-structured input, while

leaving irrelevant fields empty. This process goes beyond simple keyword matching and instead relies on linguistic context, entity relationships, and domain-specific semantics to ensure that extracted values are both accurate and meaningful [22, 42]. In practice, information extraction determines whether heterogeneous and noisy inputs—such as partially transcribed voice recordings, fragmented text segments, or incomplete chat messages—can be transformed into reliable structured data [27, 32].

As illustrated in Figure 2.1, an ideal solution must be able to extract relevant details even when input is noisy, fragmented, or incomplete, and populate the corresponding template fields while leaving others empty. The highlighted gaps in the example show that extraction is not only about finding correct values but also about reliably detecting when information is missing and avoiding unsupported guesses.

This requirement matters because real-world text is rarely clean or complete. Voice transcriptions may drop words or introduce errors, workplace chat logs often contain fragments, abbreviations, or extraneous commentary [3]. Without strong extraction capabilities, systems risk either leaving critical fields blank or, worse, filling them with incorrect assumptions. In regulated or safety-critical domains such as healthcare or industrial maintenance, such errors can result in compliance violations, misdiagnoses, or operational risks [39, 17].

In practice, effective extraction can be achieved through three complementary mechanisms. First, the system must be robust to incomplete input, recognizing when essential details are missing and preserving empty fields instead of filling them with fabricated values. For example, if a date of inspection is not provided, the system should explicitly leave the field empty rather than guessing. Second, conflict handling is essential. If a transcript contains contradictory information—such as "appointment on August 10" followed by "meeting scheduled for August 12"—the system should not arbitrarily choose one date but instead detect the inconsistency and prompt the user for clarification, ensuring reliability of the final record. Third, extraction requires contextual understanding. Information is often scattered across multiple parts of a conversation or document, and the system must combine relevant fragments into coherent field values while ignoring unrelated chatter. For instance, if a technician states, "We restarted the press at 23:14. Oh, and the noise got louder after ten minutes," the system must connect these details to the same inspection event rather than treating them as separate or unrelated entries.

The advantages of strong information extraction capabilities extend across multiple dimensions. They improve accuracy by minimizing both incorrect and missing field values, even when input is fragmented or linguistically complex. They reduce user workload by lowering the amount of manual correction required after automatic field population. They enhance robustness to noise by maintaining performance in the presence of typos, filler words, or irrelevant side remarks. Finally, they enable better multi-source integration by combining relevant information from different segments into a single coherent template entry, which is particularly valuable in collaborative or multi-turn interactions [45].

For evaluation, we measure semantic correctness through cosine similarity of embeddings computed at the field level. For each template field, both the complete system output and the complete gold standard answer are converted into dense vector representations using pre-trained sentence embedding models (e.g., Sentence-BERT, Universal Sentence Encoder). This approach appropriately credits semantically equivalent extractions such as "machine overheating" versus "equipment temperature exceeds threshold," as it is robust to paraphrasing and stylistic variation. Other similar benchmarks measuring information correctness and semantic alignment can also be used depending on the domain and available resources.

Visual Score	Interpretation (based on criteria)
●	**All three criteria satisfied.** The system reliably handles incomplete text, correctly detects and flags conflicts, and successfully integrates related information spread across different text parts. Semantic similarity scores are high (e.g., cosine similarity ≥ 0.75).
◕	**Two criteria satisfied.** For instance, the system handles incomplete input and detects conflicts but fails to integrate scattered contextual information. Semantic similarity scores are moderate (e.g., cosine similarity in $[0.60, 0.75)$).
◔	**One criterion satisfied.** For instance, the system handles incomplete text but fails to detect conflicts and integrate contextual information. Semantic similarity scores are low (e.g., cosine similarity in $[0.45, 0.60)$).
○	**No criteria satisfied.** The system fails to handle incomplete input, cannot detect conflicts, and does not integrate information spread across different parts of the input. Semantic similarity scores are very low (e.g., cosine similarity < 0.45).

Table 2.2.: Evaluation scale for R2: Information Extraction Abilities.

To summarize, information extraction abilities determine whether heterogeneous and noisy user inputs can be reliably transformed into structured templates. A robust system not only identifies and assigns relevant details but also recognizes gaps, resolves contradictions, and integrates scattered fragments into coherent entries. By measuring semantic correctness through embedding-based similarity metrics at the field level, we can evaluate whether extracted values capture the intended meaning regardless of stylistic variation. This reduces user workload, improves reliability, and ensures that the extracted data supports both operational decision-making and automated analysis.

2.1.3. R3: Transparency

Transparency refers to the framework's ability to make its decision-making process visible to users by exposing both the evidence behind each extracted value and the system's level of certainty in that value. Unlike black-box systems that present outputs without explanation, a transparent framework not only shows the populated fields but also reveals the reasoning path by which they were derived. This includes identifying the input fragments that served as evidence and providing explicit indicators of how confident the system is in its interpretations [14, 41].

As illustrated in Figure 2.1, transparency in an ideal framework means that populated values are never shown in isolation but are accompanied by their supporting evidence and a confidence indicator. The figure highlights how a field such as a date is linked back to the exact phrase in the transcript and annotated with a certainty level, making the reasoning process visible rather than opaque.

This requirement matters because users often interact with template filling systems in high-stakes environments where correctness cannot be taken for granted. In domains such as healthcare, industrial inspection, or compliance reporting, an incorrectly extracted date, measurement, or symptom could lead to operational errors or legal consequences. Without transparency, users are left to either blindly trust the system or manually re-check all outputs. Both outcomes undermine efficiency and adoption. By contrast, when the system highlights the origin of each extracted field and communicates its confidence level, users can selectively verify uncertain or ambiguous entries, thereby improving both trust and productivity [52, 40].

In practice, transparency is achieved through two complementary mechanisms. First, source highlighting makes explicit which text fragments in the input led to the population of a given field. For instance, if the transcript contains the phrase "scheduled for 08/16/2025," the system should highlight this fragment as the evidence for the "Date" field. This allows users to directly trace values back to their textual origins and verify their correctness. Second, confidence scores quantify the system's certainty about each extracted value. These can be expressed as probabilities (e.g., 0.82 confidence in the extracted date) or as categorical levels (e.g., high, medium, low). Confidence scores help users prioritize where to invest their attention, focusing on entries that are most likely to contain errors while trusting those that are more secure. Together, these mechanisms turn opaque predictions into traceable, verifiable decisions.

The advantages of transparency are multifold. It builds user trust by establishing a clear link between input data and structured outputs, reducing the perception of the system as a "black box." It enables efficient error detection, since users can quickly locate uncertain or weakly supported values without re-checking the entire template. Transparency also facilitates training and feedback, as developers and domain experts can identify systematic error patterns by examining low-confidence predictions and their sources [47]. Finally, transparency supports compliance in

regulated industries by providing auditable evidence trails that show not only what the system decided but also why it decided it.

Visual Score	Interpretation
●	**Both criteria satisfied.** The system highlights the exact text fragments that led to each populated field and provides confidence scores for all extracted values.
◑	One criterion satisfied (e.g., confidence scores are displayed but no source highlighting is provided, or vice versa).
○	No criteria satisfied. The system provides populated fields without indicating their source or the certainty of extraction, leaving the process opaque.

Table 2.3.: Evaluation Scale for R3: Transparency

To summarize, transparency ensures that extracted template values are not only presented as results but also explained in terms of their origins and associated confidence levels. By exposing the reasoning process, the system strengthens user trust, enables efficient verification, and provides the auditability necessary for both iterative improvement and compliance in regulated domains.

2.1.4. R4: User Correction

User correction refers to the framework's ability to allow manual modifications of populated template fields after automatic extraction has taken place. Unlike automated adaptation mechanisms, which rely on system learning from prior interactions, user correction focuses on immediate, explicit interventions that ensure the accuracy of individual outputs. It grants end-users direct control over the final contents of a template, ensuring that errors in recognition, interpretation, or formatting can be addressed before the data is finalized [5, 45].

As illustrated in Figure 2.1, an ideal framework must let users directly modify any field highlighted as incorrect or uncertain. The figure shows how entries flagged by the system can be overwritten or adjusted before finalization, ensuring that human judgment always has the final word in the template.

This requirement matters because no automated extraction system can achieve perfect accuracy across all input conditions. Transcriptions may introduce errors, ambiguous phrases may be misinterpreted, and domain-specific terminology may be inconsistently mapped. In practical contexts such as medical documentation or maintenance inspections, even small inaccuracies—such as recording the wrong date of an incident or misreporting a measurement—can have disproportionate consequences.

By enabling users to directly edit system-generated values, frameworks ensure that such errors can be corrected in real time without the need for retraining, secondary workflows, or downstream data cleaning [35].

In practice, correction functionality should be applied at the field level. Users must be able to select any automatically populated entry and replace it with the intended value, for example correcting "08/16/2025" to "08/15/2025" in a date field or updating "mechanical failure" to "electrical failure" in an inspection log. Such capabilities not only resolve errors but also give users confidence in the system by demonstrating that the outputs are not immutable black-box predictions. This aligns with broader human–computer interaction principles emphasizing user empowerment, flexibility, and control over system outputs [45, 24].

The advantages of user correction are twofold. First, it enables immediate error resolution: users do not need to wait for iterative model updates or retraining cycles to address mistakes, but can directly ensure accuracy within the current interaction. Second, it promotes user empowerment by giving individuals ownership of the final data submitted, strengthening their trust in the system and reducing frustration when mistakes inevitably occur. These benefits are particularly important in time-sensitive workflows, where operators must finalize records quickly while retaining the ability to guarantee correctness [1].

Visual Score	Interpretation
●	**Full correction support.** Users can edit any populated field directly, ensuring complete control over the final template.
○	**No correction support.** Automatically populated values cannot be modified by users, leaving inaccuracies unresolved.

Table 2.4.: Evaluation Scale for R4: User Correction

To summarize, user correction ensures that populated templates remain accurate by enabling individuals to directly edit extracted values. This requirement emphasizes immediate error resolution and user empowerment, giving people control over final outputs while reinforcing trust in the system's reliability.

2.1.5. R5: Learning and Adaptation

Learning and adaptation refer to the framework's ability to improve its performance over time by incorporating knowledge from user interactions, corrections, and structured domain information. Unlike static systems, which produce the same outputs regardless of experience, an adaptive framework evolves continuously. It refines

its internal models and decision-making strategies based on provided examples, explicit corrections, and formal domain rules, thereby becoming more accurate and contextually aligned with the environment in which it is deployed [14, 46].

As illustrated in Figure 2.1, finalized corrections and domain glossaries feed a feedback loop that informs future fills. This matters because using both sources reduces recurring errors and steadily improves accuracy.

This requirement matters because template filling systems are often deployed in highly specialized settings where general-purpose models cannot fully capture local terminology, workflows, or conventions. For example, a healthcare provider may require specific mappings between colloquial expressions such as "chest cold" and formal diagnostic categories, while a manufacturing plant may use internal identifiers or shorthand that differ from industry-wide terminology. Without learning and adaptation, the system risks repeating the same mistakes indefinitely, requiring users to make the same corrections again and again. By contrast, a system that consolidates knowledge from past interactions and domain inputs progressively reduces error rates and builds long-term efficiency [32, 33].

In practice, adaptation can occur through two complementary mechanisms. First, learning from provided filled-in templates allows the system to build expectations about the type and structure of values that belong in each field. For example, if historical forms consistently show that an employee ID consists of exactly five characters, the system can detect when a user provides only three and prompt them to complete the entry correctly. Over time, such feedback reduces recurring formatting or completeness errors by aligning new inputs with established patterns. Second, learning from domain knowledge integrates structured resources such as ontologies, glossaries, or formal rule sets. For instance, a medical ontology may specify that "flu" and "influenza" are synonymous terms, while an engineering glossary may define acceptable formats for torque values. By encoding such knowledge, the system improves both extraction and formatting, aligning outputs with established standards rather than ad-hoc interpretations [12, 8].

The advantages of learning and adaptation are cumulative. Accuracy improves progressively as the system incorporates examples and corrections, ensuring that the framework becomes more reliable the longer it is used in a given domain. Knowledge consolidation ensures that corrections made once benefit future cases, reducing the burden of repetitive manual adjustments. Continuous refinement also lowers the risk of systematic errors, as patterns of mistakes are gradually eliminated. These capabilities transform template filling from a static tool into an evolving system that adapts to organizational needs and grows more effective with sustained use [45, 1].

To summarize, learning and adaptation ensure that the framework evolves with use, incorporating both user-provided corrections and structured domain knowledge. This capability reduces repetitive errors, consolidates expertise, and progressively

Visual Score	Interpretation
●	**Both mechanisms satisfied.** The system learns from corrected or manually filled templates and incorporates structured domain knowledge such as ontologies and glossaries.
◑	One mechanism satisfied (e.g., the system learns from corrections but cannot integrate domain knowledge, or vice versa).
○	No mechanisms satisfied. The system is static, with no ability to learn from past usage or structured knowledge sources.

Table 2.5.: Evaluation Scale for R5: Learning and Adaptation

improves accuracy, enabling the system to remain aligned with the specialized needs of its deployment context.

2.1.6. R6: Usability

Usability in the context of this framework refers primarily to **latency**, i.e., the time delay between user input and the system's output (e.g., updated template fields becoming visible). While this thesis evaluates a speech-driven scenario, *speech is only one use case*; the core task is the transformation of *unstructured input (spoken or typed text)* into structured templates. Accordingly, the latency requirement applies to any unstructured-to-template pipeline (typed submissions, transcripts, or mixed modalities). In such settings, responsiveness directly determines whether interaction feels natural and uninterrupted. Human–computer interaction research consistently shows that feedback within two to three seconds is perceived as natural turn-taking, while longer delays disrupt concentration and diminish trust in the system [45]. For voice-first applications, user expectations are shaped by everyday experiences with assistants, where near-instantaneous responses are the norm [51].

As illustrated in Figure 2.1, the template should update promptly after a spoken or typed submission. Timely visual feedback preserves flow and enables quick verification.

Unlike accuracy or error tolerance, which are covered in other requirements (R1–R5), usability here is scoped strictly to latency. This requirement matters because template filling often occurs in time-pressured contexts (e.g., hospitals or industrial facilities), where long waiting times interrupt workflows. For example, a technician dictating a machine fault—or a clerk pasting a long free-text note—must continue seamlessly; if

the system takes ten or more seconds to update, the reporting process is disrupted and efficiency drops. Thus, latency is the decisive factor for adoption in real-world deployments.

However, latency is not an absolute property of the software alone. It is strongly influenced by the **hardware and deployment environment** on which inference is performed. High-performance accelerators such as GPUs and TPUs can significantly reduce end-to-end response time, whereas CPU-only or resource-constrained devices typically increase delays [45, 28]. Therefore, when evaluating usability, latency benchmarks must always be reported together with the hardware profile (e.g., GPU/TPU type, number of concurrent users, and network setting). This ensures that usability scores are interpreted relative to realistic deployment contexts rather than as abstract performance claims.

Visual Score	Interpretation
●	**High usability.** p95 end-to-end (E2E) latency $\leq 20\,\mathrm{s}$ on the declared hardware profile. Interaction feels natural; conversational pacing is preserved.
◑	**Medium usability.** p95 E2E latency in $(20, 40)\,\mathrm{s}$. Delay is noticeable but tolerable in most workflows.
○	**No usability.** p95 E2E latency $> 40\,\mathrm{s}$ or frequent timeouts. Workflow is impractical under this configuration.

Table 2.6.: Evaluation Scale for R6: Usability

To summarize, usability in this framework is defined as responsiveness measured through latency for unstructured-to-template conversion across modalities (speech and text). Because latency is hardware-dependent, evaluation must always contextualize results by reporting the target infrastructure. This ensures comparability across systems while keeping interpretations grounded in practical deployment realities.

2.2. Related Work

The transformation of unstructured language into structured templates is a core challenge in natural language processing, spanning information extraction, semantic parsing, and template filling. Classical rule-based and statistical methods often fail to handle the ambiguity and variability of free-form text [13]. With the rise of large language models (LLMs), this task has gained renewed attention, as modern models offer strong contextual understanding and improved structured output generation [4]. Recent work further shows that LLMs can generalize across domains and input

modalities, supporting more robust end-to-end systems for transforming unstructured text or speech into structured representations.

However, despite these advances, current approaches often struggle with the specific requirements identified in Section 2.1, particularly when dealing with noisy, domain-specific, or incomplete inputs common in real-world industrial and professional settings. This section examines the current state of research relevant to multi-agent template filling systems, with particular focus on how existing approaches address the core challenges of consistency, information extraction accuracy, transparency, user correction capabilities, learning mechanisms, and usability. The analysis reveals significant gaps in current methodologies, particularly in their monolithic design assumptions and limited adaptability to specialized domains.

The review is organized into three key areas that directly inform the design of the proposed Invox system.

i) Template filling and structured information extraction: we examine traditional and modern approaches, tracing the evolution from rule-based pipelines to contemporary LLM-driven methods.

ii) Prompt engineering techniques: we analyze strategies that guide LLMs toward reliable structured output generation and improved controllability.

iii) Multi-agent LLM systems: we explore architectures that decompose complex tasks into specialized components, addressing modularity, transparency, and the limitations of monolithic approaches.

2.2.1. Template Filling and Structured Information Extraction

This subsection reviews existing solutions that address the challenge of extracting structured information from unstructured text and populating predefined templates. These solutions have evolved from rule-based approaches to modern neural and generative methods, each offering different trade-offs in accuracy, robustness, and adaptability.

Extracting structured information from unstructured text has been a central challenge in natural language processing (NLP) for decades. Early systems were rule-based, using handcrafted grammars, dictionaries, and regular expressions to detect domain-specific patterns [22]. These methods worked well in narrow settings but were brittle, required heavy manual effort, and could not easily adapt to new domains.

Statistical models offered more flexibility. Hidden Markov Models (HMMs) were used to model sequential dependencies, while Conditional Random Fields (CRFs) became particularly influential by enabling richer feature-based sequence labeling [42]. CRFs were widely adopted for tasks such as Named Entity Recognition (NER) and slot filling, achieving strong results on shared tasks like CoNLL-2003 [48]. Despite

these advances, statistical models still struggled with domain transfer, long-range dependencies, and their reliance on handcrafted features.

Another important limitation of early work was the reliance on pipeline-based architectures. Classical information extraction was usually divided into subtasks such as entity recognition, relation classification, and template alignment. Each stage depended on the output of the previous one, which often led to error propagation [27]. For example, if the entity recognition module mislabeled an organization, that error would carry over to relation classification and template population. While pipelines offered modularity, they lacked robustness and transparency, making them hard to apply in noisy or domain-specific environments.

A major breakthrough came with the transformer architecture [49]. Transformers introduced self-attention and large-scale pretraining, which made it possible to capture long-range context and resolve ambiguity in ways earlier models could not. Building on this foundation, large language models (LLMs) can now generate structured outputs directly, for example in JSON, XML, or domain-specific templates. This has enabled information extraction in zero-shot or few-shot settings, significantly reducing the need for task-specific labeled data [6, 54]. This marks a shift from extractive, pipeline-based designs to generative, end-to-end systems.

2.2.1.1. End-to-End Generative Template Filling

One influential contribution in this direction is the work by Du et al. on *Template Filling with Generative Transformers* [16]. Their framework replaces the traditional two-step pipeline of NER followed by event classification with a single generative system. The model conditions on document tokens and event type markers to generate slot fillers directly, capturing dependencies across multiple events within a document and avoiding the error propagation typical of pipelines. On the MUC-4 dataset, their approach outperformed both pipeline-based and extractive neural baselines, particularly for documents that describe multiple events.

2.2.1.2. Speech-Based Template Filling

Work on speech-driven template filling has also emerged. **Sun et al. (2023)** study *Speech-based Slot Filling using Large Language Models* [46], focusing on extracting slot values from automatic speech recognition (ASR) transcripts under noisy conditions. Their evaluation of GPT-3.5, GPT-4, LLaMA, and Vicuna on the SLURP dataset introduces structured prompt design, noise-robust LoRA fine-tuning, and linearised knowledge injection (LKI). Their findings show that while LLMs generalize well in zero-shot and few-shot settings, robustness in real-world speech requires careful prompt design and adaptation strategies.

Figure 2.2.: Example of error propagation in a classical NLP pipeline: an early mistake in named entity recognition prevents correct relation extraction, leading to incomplete template filling.

2.2.1.3. Web-Based Structured Generation

LLMs have also been applied to structured generation beyond classical IE. **Chen et al. (2025)** propose *Automated Web Application Testing with LLMs and Screen Transition Graphs* [29]. Their method combines structural graphs of websites with LLM-based generation of Selenium test scripts. Screen transition graphs model navigation flows, while state graphs represent conditional forms. The LLM generates executable test cases consistent with these constraints. Although applied in a different domain, this work illustrates the broader applicability of schema-constrained structured generation.

2.2.1.4. Benchmarks and Evaluation

Benchmarks such as **MUC-4** [11], **ACE** [13], and **CoNLL-2003** [48] have shaped the development of information extraction systems (see Section 5.1 for a detailed description of MUC-4 in the evaluation setup). More recently, speech-centric datasets such as **SLURP** [4] support evaluating spoken language understanding. These benchmarks show where statistical approaches fail and where transformer-based models excel, but they also highlight that surface-level metrics such as F1 do not capture robustness or consistency.

2.2.1.5. Domain-Specific Challenges

Domain-specific environments impose additional challenges. Healthcare requires handling specialized terminology and implicit knowledge [19]. Industrial settings include technical identifiers, inconsistent documentation, and informal communication styles such as shift logs [51]. These contexts demand systems that are both robust and adaptable.

2.2.1.6. Practical Tools

Google's LangExtract [21] provides schema-based extraction using LLMs. It maps unstructured text into user-defined JSON schemas and is deployable in production. However, it relies on a single model, does not support modular correction or consensus across multiple LLMs, and reveals no intermediate reasoning, limiting transparency.

2.2.1.7. Schema Enforcement

Schema enforcement methods such as constrained decoding [2], schema-guided prompting [57], and external validation help ensure that LLM outputs adhere to predefined structures.

2.2.1.8. Evaluation Against Requirements

Table 2.7 evaluates the discussed solutions against requirements R1–R6. Each requirement uses its specific grading scale defined in Section 2.1.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
Du et al. [16]	◕	●	○	○	○	—
Sun et al. [46]	◕	◕	○	○	○	—
Chen et al. [29]	◕	◕	○	○	○	—
Google LangExtract [21]	●	●	○	○	○	—

Table 2.7.: Evaluation of template filling and structured extraction solutions against requirements R1–R6.

The evaluation highlights clear performance patterns across the reviewed systems. All generative approaches demonstrate strong capabilities in **information extraction** (R2), with Du et al. and LangExtract achieving the highest level of extraction robustness. Consistency (R1) is reached at a medium level by most research systems, while LangExtract attains high consistency due to strict schema enforcement and validation.

However, the analysis also reveals structural limitations shared across all reviewed solutions. None provide **transparency** (R3) through span-level evidence or confidence scores, and none support **user correction** (R4) within the extraction loop. Likewise, none implement mechanisms for continuous **learning and adaptation** (R5) based on user feedback or domain changes. Finally, none of the works report **usability metrics** (R6), such as latency or hardware constraints, leaving their real-world responsiveness unclear.

Overall, while current systems excel in extraction accuracy, they remain monolithic black-box architectures lacking the interactive, modular, and adaptive capabilities required for real-world deployment. These gaps motivate the next subsections on prompt engineering and multi-agent system design, which directly address transparency, correction, adaptability, and robustness.

2.2.2. Prompt Engineering for Information Extraction

Large language models can generate structured outputs, but their reliability strongly depends on how the task is formulated. Prompt engineering has therefore become

central for guiding LLMs toward schema-compliant and domain-aware outputs. By carefully designing instructions, examples, and schema descriptions, users can steer models to align their generations with the requirements of template filling.

The ability of LLMs to adapt to new tasks without retraining was first demonstrated through *in-context learning*. Brown et al. showed that GPT-3 could perform unseen tasks when given only natural-language instructions and a few examples [6]. Later, Wei et al. documented "emergent abilities," where large models exhibit strong zero-shot and few-shot performance once surpassing certain scale thresholds [54]. For information extraction, this means that models can often populate structured fields when the prompt clearly specifies the task and provides minimal demonstrations.

Designing effective prompts, however, is non-trivial. Sun et al. [46] propose structured prompts for speech-based slot filling that incorporate task definitions, slot descriptions, and linearized knowledge, improving robustness to ASR noise. Similarly, Lin et al. [31] show that including slot descriptions directly in prompts improves zero-shot dialogue state tracking by grounding the model in the semantics of each field. These studies highlight how prompts can embed domain knowledge to enhance extraction quality.

The choice between zero-shot and few-shot prompting also influences performance. Zero-shot prompts rely solely on schema descriptions and are flexible but less reliable. Few-shot prompting, which augments the prompt with exemplar input–output pairs, improves consistency but increases prompt length and inference latency. Pan et al. [36] evaluate ChatGPT for dialogue understanding and find that few-shot prompting consistently improves accuracy, albeit with slower responses.

Schema-constrained prompting has also received attention as a way to enforce structured output. Zhang et al. [57] introduced SGP-TOD, where prompts embed schema definitions for task-oriented dialogue systems. By instructing the model to generate values that must fit the schema, they reduce invalid or inconsistent outputs. Practical systems follow the same idea: Google's LangExtract [21] uses user-defined JSON schemas to constrain extraction results. Although this improves format consistency, prompt-based schema enforcement remains monolithic, lacking modular correction and multi-model verification mechanisms.

Beyond manual prompt design, recent studies explore *automated prompt optimization*. Peng et al. [38] present a framework where candidate prompts are iteratively refined using feedback signals such as factual errors and schema mismatches. This "prompt tuning without training" approach suggests that prompts themselves can be optimized algorithmically, reducing reliance on human intuition.

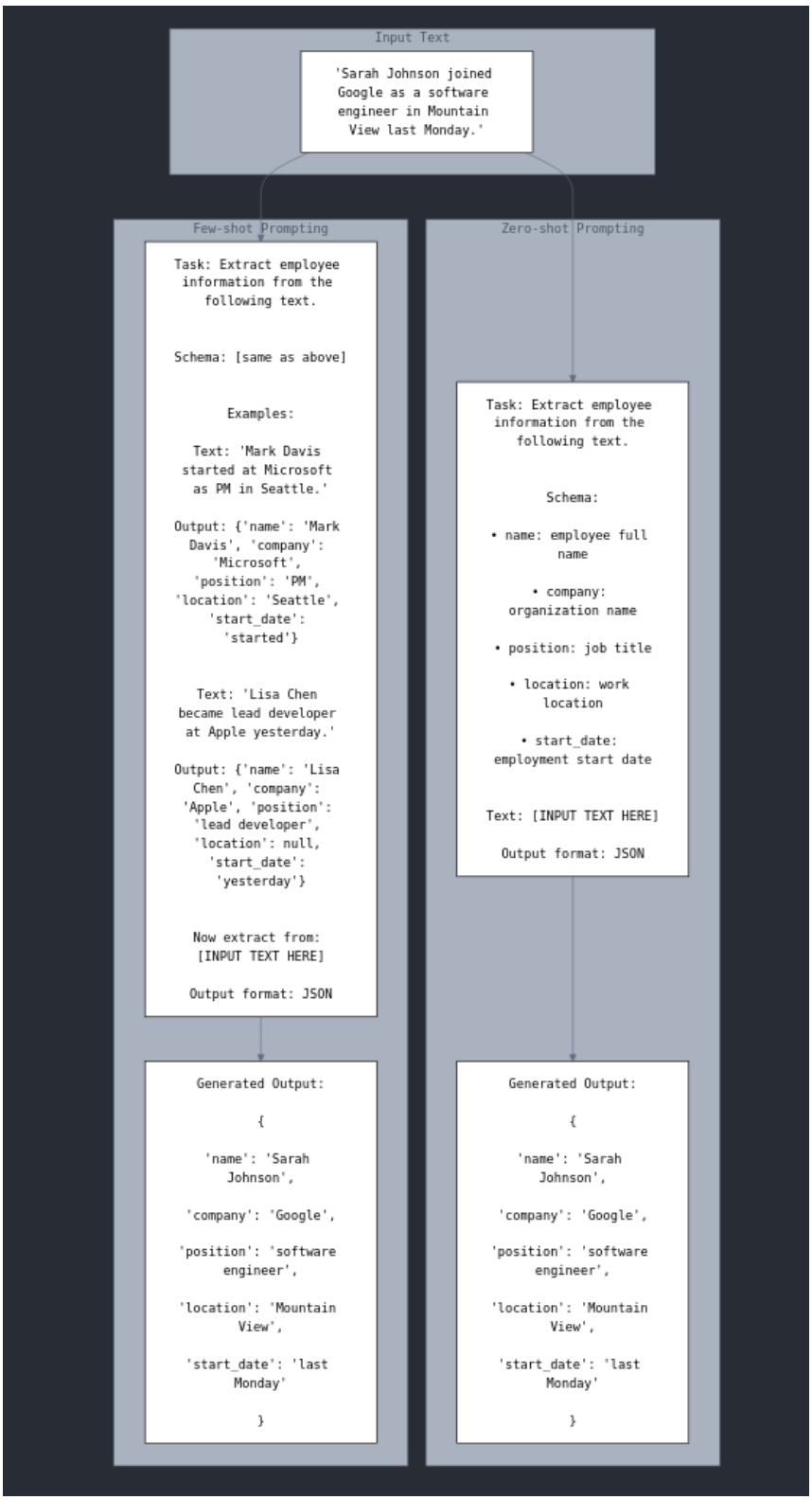

Figure 2.3.: Illustration of zero-shot vs. few-shot prompting for template filling: zero-shot uses only task instructions, whereas few-shot adds example pairs that guide the model toward the correct structure, typically improving extraction accuracy.

2.2.2.1. Evaluation Against Requirements

The evaluation shows that prompt engineering provides a flexible and powerful way to adapt LLMs to structured extraction tasks. Zero-shot and few-shot prompting enable template filling without task-specific training, while structured and schema-guided prompts improve consistency and robustness. Automated prompt optimization further shows that prompt quality can be systematically refined.

Table 2.8 evaluates the prompt engineering approaches discussed above against requirements R1–R6. Each requirement uses the grading scale defined in Section 2.1.

Solution	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
In-context learning [6]	◕	●	○	○	○	—
Structured prompting [46]	◕	◕	○	○	○	—
Schema-guided prompting [57]	●	●	○	○	○	—
Automated prompt optimization [38]	●	●	○	○	○	—

Table 2.8.: Evaluation of prompt engineering approaches against requirements R1–R6.

Despite these strengths, prompt engineering alone does not satisfy several core requirements identified in Section 2.1. None of the reviewed approaches support **transparency** (R3) through evidence tracing or confidence scores, nor do they provide **user correction** (R4) within the extraction loop. They also lack mechanisms for continuous **learning and adaptation** (R5), and none report **usability** measures (R6) such as latency or computational cost. Prompts additionally remain brittle: small phrasing changes can produce different outputs, and schema adherence often requires external validation.

Overall, while prompt engineering is an essential foundation, it remains insufficient for building reliable, transparent, and adaptive template-filling systems. These limitations motivate the next subsection on multi-agent LLM architectures, which introduce modular structure, delegation, and verification layers to overcome the constraints of single-prompt systems.

2.2.3. Multi-Agent LLM Systems

While single-model prompting can achieve strong results, it often struggles with reliability, transparency, and adaptability in complex tasks. Recent work has there-

fore explored *multi-agent systems*, where large language models are organized as a cooperating group of agents with different roles. Instead of relying on a single LLM to handle extraction, validation, reasoning, and correction, tasks can be decomposed into smaller subtasks performed by specialized agents.

One of the most prominent examples is **HuggingGPT**, introduced by Shen et al. [44]. In this system, an LLM serves as a controller that interprets user requests, decomposes them into subtasks, and delegates them to external models on the Hugging Face platform. The controller then integrates the results into a final output. Liang et al. [30] proposed a similar architecture with **TaskMatrix.AI**, which connects LLMs to thousands of APIs, enabling each subtask to be handled by the tool best suited for it. These studies conceptualize LLMs not as monolithic solvers but as managers capable of coordinating heterogeneous toolchains.

Multi-agent ideas have also been applied directly to structured reasoning and information extraction. Qian et al. [**?**] demonstrate that assigning agents complementary roles—such as extractor, verifier, and corrector—improves factual consistency and reduces hallucinations through inter-agent critique. This structured division of labour aligns well with template-filling workflows, where one agent extracts fields, another checks completeness, and a third corrects schema violations. Ye et al. [56] further show that guided and knowledge-grounded multi-agent debate improves factual verification by coordinating specialised agents with different reasoning strategies, illustrating how debate-driven refinement can enhance reliability in complex extraction pipelines.

Consensus-based multi-agent systems add another reliability mechanism. Wu et al. [55] introduce **AutoAgents**, an open-source framework where multiple agents generate alternative solutions that are then aggregated or voted on. This ensemble-style approach reduces variance and improves robustness, functioning similarly to redundancy and majority-voting techniques in classical ensemble learning but enhanced by the contextual reasoning abilities of LLMs. Consensus methods are particularly useful for noisy or ambiguous inputs, where a single-model output might fluctuate.

Despite these advantages, multi-agent systems also introduce challenges. Coordination overhead increases computational cost and latency, and poorly designed agent interfaces can create cascading errors. Evaluation of cooperation quality remains limited—most benchmarks measure final outputs but not the reliability of inter-agent communication. Multi-agent systems therefore offer greater transparency and robustness, but at the cost of complexity and resource usage.

Approach	Transparency	Cost	Latency	Complexity
Single-Model	Low	Low	Low	Low
Multi-Agent	High	High	High	High

Figure 2.4.: Comparison of single-model and multi-agent approaches. Single-model systems are simple and efficient but offer little transparency, while multi-agent systems increase cost and complexity in exchange for greater modularity and interpretability.

2.2.3.1. Evaluation Against Requirements

Table 2.9 evaluates the discussed multi-agent approaches against requirements R1–R6 using the scoring scales defined in Section 2.1.

Approach / System	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**
HuggingGPT [44]	◕	●	◑	○	◑	—
TaskMatrix.AI [30]	◕	●	◑	○	◑	—
Cooperative extraction [53]	◕	◕	◑	○	◑	—
Consensus systems [9]	●	●	◑	○	●	—

Table 2.9.: Evaluation of multi-agent approaches against requirements R1–R6.

The evaluation reveals that multi-agent approaches offer clear strengths in **information extraction** (R2), **transparency** (R3), and **adaptability** (R5). By distributing responsibilities across specialised agents, intermediate decisions become inspectable, and the system can combine diverse capabilities such as retrieval, validation, and consensus building. Consensus-based systems in particular achieve strong **consistency** (R1) through aggregation across multiple agent outputs.

At the same time, all reviewed systems lack integrated **user correction** mechanisms (R4) and do not report **usability** or latency characteristics (R6), making their real-world responsiveness unclear. Multi-agent designs also introduce coordination overhead, increasing computational cost and deployment complexity.

Overall, the table highlights a key trade-off: multi-agent architectures significantly improve reliability, interpretability, and modularity but at the expense of higher cost and complexity. This trade-off directly motivates the architectural decisions in this thesis, where multi-agent strategies are evaluated alongside monolithic and hybrid approaches.

2.2.4. Overall Evaluation and Gaps

Across the reviewed literature, each line of work contributes an essential piece to the broader challenge of structured information extraction. End-to-end generative models improve accuracy and reduce pipeline fragility; speech-focused methods add robustness under noisy ASR conditions; schema-guided prompting stabilizes formats; automated prompt optimization increases reliability; and multi-agent systems introduce modularity, traceability, and division of labour. Together, these developments trace a trajectory from brittle rule-based systems to flexible but opaque single-model LLMs, and finally toward coordinated, role-specialized multi-agent architectures.

Despite this progress, the consolidated evaluation shows that no existing approach satisfies all six requirements (R1–R6). Transparency, user-centered correction, adaptive learning, and measurable usability remain underdeveloped, and most methods excel only in narrow subsets of requirements. This reveals a clear gap: the field lacks an integrated solution that handles noisy, speech-driven inputs while remaining interpretable, correctable, and operationally robust.

These limitations directly motivate the system design developed in this thesis. Chapter 3 builds on these insights to propose a modular architecture that combines the strengths of prior work—generative extraction, schema enforcement, iterative refinement, and multi-agent coordination—while explicitly addressing the unmet requirements identified here. The following chapter translates this analysis into concrete design principles, architectural decisions, and processing pipelines behind the proposed Invox system.

3. Concept

This chapter presents the conceptual foundation of the proposed solution, grounded in the requirements and limitations identified in Chapter 2. The core objective is to transform unstructured narrative input—whether spoken or typed—into structured, schema-conformant templates through a modular, agent-based architecture. Unlike monolithic approaches that attempt end-to-end extraction in a single inference step, the proposed system decomposes the task into specialized subtasks, each handled by a dedicated agent with clearly defined responsibilities and interfaces.

The design is motivated by three critical shortcomings observed in existing work. First, single-model systems lack transparency: when extraction fails, it is difficult to trace whether the error originated in entity recognition, field mapping, or format normalization [16, 46]. Second, monolithic architectures are brittle under noisy or domain-specific input, as they cannot isolate and recover from localized failures [50]. Third, current solutions provide limited support for iterative refinement, user correction, or learning from domain knowledge, restricting their adaptability in real-world deployments [33, 37].

The proposed Invox system addresses these limitations through a five-stage pipeline (see Figure 3.1) that enforces separation of concerns while preserving end-to-end coherence. Each agent operates on well-defined inputs and produces structured outputs that serve as inputs to downstream components. This modular design directly supports the six requirements established in Section 2.1: consistency is enforced through dedicated normalization (R1), extraction accuracy benefits from retrieval-augmented context (R2), intermediate artifacts enable traceability (R3), outputs remain editable before final submission (R4), the system learns from historical templates and domain resources (R5), and architectural strategies can be selected to meet latency constraints (R6).

The remainder of this chapter is organized as follows. Section 3.1 derives the conceptual design from the analysis results, explaining how each requirement motivates specific architectural decisions. Section 3.2 describes the five modular agents that comprise the pipeline, detailing their individual responsibilities, inputs, outputs, and internal mechanisms. Section 3.3 presents four architectural strategies—Single-Pass Full Input, Iterative Single-Field Processing, Multi-LLM Consensus (Full), and Multi-LLM Consensus (Iterative)—that instantiate the same pipeline with different trade-offs in accuracy, latency, and cost. Section 3.4 provides the high-level system architecture and design, including context diagrams, container views, and

process flows that illustrate how components interact in deployment scenarios. Detailed implementation and experimental validation are deferred to Chapters 4 and 5, respectively.

3.1. Concept Derivation from Analysis Results

The conceptual design of the Invox system is derived systematically from the six requirements identified in Section 2.1 and the gaps observed in existing approaches reviewed in Section 2.2. This section traces how each architectural decision directly addresses specific limitations in current template-filling systems, establishing the rationale for a modular, multi-agent design.

3.1.1. From Monolithic to Modular Processing

The analysis in Section 2.2 revealed that monolithic approaches—where a single LLM performs extraction, normalization, and validation in one inference step—suffer from three fundamental weaknesses. First, they propagate errors across stages without providing mechanisms for localized recovery [46]. When entity recognition fails, subsequent field mapping inherits these errors, compounding inaccuracies. Second, they lack transparency: users cannot trace which part of the input led to which output, making error diagnosis difficult and undermining trust [40]. Third, they provide no intermediate points for user intervention, forcing corrections to be applied post-hoc rather than during processing [1].

These observations directly motivate the decomposition of the template-filling task into discrete, sequential stages. By separating transcription, retrieval, extraction, normalization, and verification into dedicated agents, the system gains three critical capabilities. Errors can be isolated to specific stages, allowing targeted debugging and recovery without reprocessing the entire pipeline. Intermediate outputs become visible and editable, supporting both transparency (R3) and user correction (R4). Finally, individual agents can be improved or replaced independently, enabling iterative refinement without destabilizing the overall architecture.

3.1.2. Addressing Consistency Through Dedicated Normalization

Requirement R1 demands that heterogeneous inputs—whether informal speech transcripts, chat messages, or multilingual notes—be transformed into uniform, comparable template entries. Existing generative systems often produce stylistically inconsistent outputs because the same model is responsible for both content extraction and format enforcement [25]. This dual responsibility introduces variance: one

inference may produce "08/16/2025" while another outputs "August 16, 2025" for the same date, even when guided by identical prompts.

One solution employed in this thesis is to delegate normalization to a specialized Consistency Formatting (CF) agent that operates independently of extraction. Once the Information Extraction (IE) agent proposes candidate values, CF applies deterministic transformations—date standardization, entity canonicalization, and vocabulary alignment—ensuring that all outputs conform to predefined schemas. This separation keeps extraction logic focused on identifying correct content, while formatting logic enforces structural and stylistic uniformity. The result is higher consistency across diverse inputs, directly satisfying R1.

3.1.3. Retrieval-Augmented Generation for Domain Adaptation

Robust information extraction under noisy, incomplete, and domain-specific conditions **(R2)** remains a challenge for single-shot LLMs. As discussed in Section 2.2, zero-shot models struggle with specialized terminology, abbreviations, and implicit references common in industrial and healthcare environments [50]. Few-shot prompting mitigates some of these issues, but manually curating examples for every deployment scenario is impractical and does not scale across domains [54].

In Invox, this issue is addressed using a Retrieval-Augmented Generation (RAG) agent positioned between transcription and extraction. The RAG agent queries an indexed corpus of historical templates and domain glossaries, retrieving the k most semantically similar examples for the current input. These examples are then passed to the IE agent as dynamic few-shot context, grounding its reasoning in domain-specific patterns without requiring manual prompt engineering. This design directly supports R2 by improving extraction accuracy on specialized vocabulary, and it also contributes to R5 (learning and adaptation) by leveraging organizational knowledge accumulated over time.

3.1.4. Verification for Completeness and Cross-Field Consistency

Existing template-filling systems rarely validate their own outputs. Once fields are populated, the result is presented to the user without checking for missing required fields, contradictory values, or schema violations [46]. This places the entire verification burden on the user, increasing workload and reducing trust.

The Verification (VER) agent addresses this gap by performing three types of checks after normalization. Completeness checks ensure that all required fields have been populated or explicitly marked as unavailable. Cross-field consistency checks detect contradictions, such as an incident date that falls outside the reported shift time. Confidence scoring highlights uncertain extractions, directing user attention

to fields most likely to require correction. When issues are detected, VER can trigger a clarification loop, prompting the user for additional input before re-applying CF and VER on the updated values. This design supports R3 (transparency) by making verification criteria explicit, and R4 (user correction) by enabling targeted intervention.

3.1.5. Modularity as a Foundation for Flexibility

The analysis in Section 2.2 showed that no existing system satisfies all six requirements simultaneously. Approaches optimized for accuracy often sacrifice latency [37], while methods prioritizing speed tend to compromise on transparency or adaptability [21]. These trade-offs are inherent to monolithic designs, where architectural decisions are fixed within a single model and cannot be adjusted post-deployment.

The modular pipeline enables flexible deployment strategies without changing the underlying components. The same five agents can be orchestrated in multiple ways: as a sequential single-pass system for low-latency scenarios, as an iterative slot-parallel architecture for higher accuracy, or as a multi-model consensus mechanism for applications where reliability outweighs cost. The remainder of this section will show how each architectural strategy can be instantiated using the same agent set, and how their respective trade-offs align with different operational requirements. This flexibility directly addresses R6 (usability) by allowing latency and resource constraints to guide strategy selection while preserving modularity for future improvements.

This section summarizes how each architectural decision maps to the requirements established in Chapter 2. The modular design is not an arbitrary choice, but a direct consequence of the limitations observed in existing work and the operational constraints of real-world deployment contexts. The next section details the five agents that instantiate this design, specifying their inputs, outputs, and internal processing logic.

3.2. Five-Agent Architecture

The modular architecture of *Invox* is organized into five specialized agents. Each agent exposes a clear input/output contract, enabling component-level analysis, flexible substitution, and isolated improvement. This section characterizes the roles of the agents and their interconnections, independent of any particular deployment strategy.

3.2.1. Speech-to-Text Agent (STT)

Invox employs an STT agent that converts spoken language into text using a Whisper-based transcription model [39]. This STT agent accepts audio recordings as input and outputs transcribed text together with metadata such as word-level confidence scores, timestamps, and language detection results. As the focus of this thesis is the mapping from unstructured text to template fields, the STT component functions as an adapter rather than a core architectural element. Textual inputs bypass this stage and enter directly into the RAG agent.

Design considerations. The STT agent is deliberately shallow to maintain generality. Whisper was selected for its robustness to noise and multilingual support, which are critical in industrial environments where background machinery, accents, and code-switching are common [17]. Alternative ASR systems could be substituted without affecting downstream components, provided they produce UTF-8 text output.

3.2.2. Retrieval-Augmented Generation Agent (RAG)

The RAG agent of Invox enhances extraction accuracy by retrieving relevant examples from historical data before template filling begins. It accepts the transcribed or typed text as input and queries an indexed corpus—comprising previously completed templates—using semantic similarity search over dense vector embeddings. The agent outputs the top-k most relevant examples, which are then passed alongside the user input to the IE agent as few-shot context.

Retrieval mechanism. The RAG agent encodes input text using a pretrained sentence transformer and performs cosine similarity search against stored template embeddings. The system returns k ranked examples (typically $k = 3$–5), each containing the original text and its corresponding template completion.

Design rationale. This agent directly addresses R2 (information extraction) and R5 (learning and adaptation). By dynamically selecting examples tailored to the current input, the system avoids the brittleness of static few-shot prompts while benefiting from organizational memory [33]. The semantic search ensures that examples are retrieved based on conceptual similarity, making the system robust to paraphrasing and vocabulary variations. Over time, as more templates are completed and added to the index, extraction accuracy improves without requiring model retraining.

3.2.3. Information Extraction Agent (IE)

After employing a RAG agent, Invox proceeds with an Information Extraction (IE) stage that transforms unstructured text into structured template fields. Its input

consists of three components: (1) the free-form text (transcribed or provided directly), (2) the retrieved few-shot examples from the RAG agent, and (3) the target template schema specifying which fields must be filled. Based on these inputs, the IE agent produces candidate slot–value pairs for all required fields.

Extraction is carried out by prompting a large language model with task-specific instructions and examples. For each field in the schema, the model attempts to locate relevant evidence in the input text; if no evidence is found, the field is explicitly assigned a null value rather than being omitted. This maintains completeness and preserves a consistent output structure.

Processing approach. The IE stage follows a schema-driven strategy that iterates over all fields specified in the template. Values are assigned only when directly supported by the input text, reducing the risk of hallucinated content. Explicit null handling ensures that missing information is represented transparently.

Prompt structure. The prompt supplied to the model integrates four components: a task definition describing the schema, retrieved few-shot examples illustrating valid extraction behavior, the user's input text, and output constraints requiring JSON-formatted responses that include all fields. This structure enhances consistency while leveraging contextual signals from the RAG stage [57].

3.2.4. Consistency Formatting Agent (CF)

Once candidate values have been extracted, Invox applies a Consistency Formatting (CF) stage that transforms these raw outputs into standardized, schema-compliant formats. This stage takes the slot–value pairs produced by the IE component and enforces uniformity through deterministic normalization and schema validation.

Schema validation and normalization. The system first converts extracted values to the appropriate data types (e.g., dates to ISO format, numbers to numeric types) and validates them against the template's schema definitions. It then applies canonicalization rules to remove surface-form variation, such as mapping "Sept. 11" to "2001-09-11" or "USA" to "United States." Controlled vocabulary mapping ensures domain-level consistency, aligning extracted terms with predefined categories for incident types, equipment codes, and other structured fields. A final schema-compliance check verifies that all values satisfy structural and semantic constraints, including type, format, and admissible ranges.

Normalization approach. This stage combines deterministic transformations with LLM-assisted resolution for ambiguous cases. Structured types such as dates, numbers, and identifiers are normalized using parsing libraries and regular expressions. When ambiguity arises—for instance, informal location names or context-dependent equipment codes—the system can query an LLM using the original text and domain glossaries to propose a canonical interpretation. This hybrid approach maintains

determinism for well-defined formats while leveraging contextual reasoning where needed.

Design rationale. Separating normalization from extraction ensures that output formatting remains consistent regardless of input variation, directly addressing R1 (consistency). It also allows the IE stage to focus on content identification rather than format enforcement [20], and reduces evaluation noise by ensuring that comparisons across extraction methods are not confounded by formatting discrepancies.

3.2.5. Verification Agent (VER)

After normalization, Invox performs a verification stage (VER) that conducts comprehensive quality assessment on the structured template produced by the CF component. This stage accepts the normalized slot–value pairs and produces a fully verified template enriched with confidence scoring, field-level annotations, and mechanisms for interactive clarification.

Quality assessment components. Multiple checks contribute to the reliability of the final template. Each field receives a confidence score derived from extraction quality, strength of supporting evidence, and internal consistency metrics. Field-level annotations document evidence sources, applied processing steps, and relevant notes, increasing transparency. Conflict detection identifies contradictions across related fields—for example, incompatible dates or mismatched equipment types—while completeness validation flags required fields that are empty or insufficiently supported, enabling targeted review.

Interactive clarification system. When low-confidence fields, conflicts, or missing information are detected, the system generates focused clarification prompts for the user. These prompts reference the problematic fields directly and incorporate conversation history to support iterative refinement through user feedback.

Final output generation. The verification stage outputs a completed template that includes the structured data along with quality annotations, confidence scores, evidence references, and indicators for fields requiring user attention. This makes reliability explicit and supports downstream processing or human review.

Design rationale. Centralizing quality assessment and user interaction in a dedicated verification stage directly supports R3 (transparency) and R4 (user correction). It ensures consistent verification standards while maintaining clean separation from extraction and normalization processes [37].

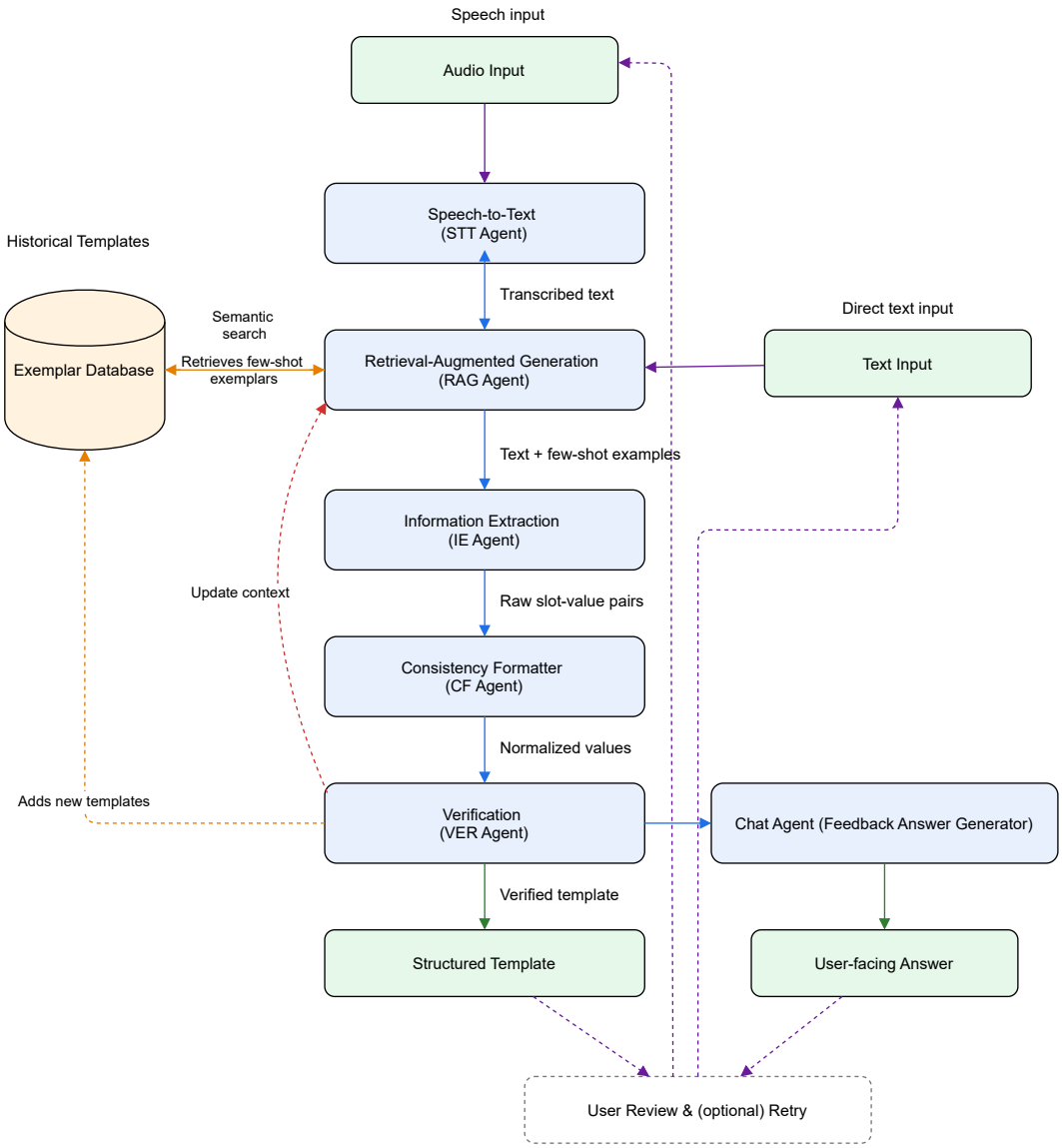

Figure 3.1.: Overview of Invox's five-agent pipeline. The multi-step flow separates transcription, retrieval, extraction, normalization, and user correction into distinct agents. This modular design addresses key limitations of monolithic LLMs by improving extraction robustness, enforcing consistent formatting, enabling transparency through intermediate outputs, and allowing final user edits before submission.

3.2.6. Agent Interfaces and Data Flow

Table 3.1 summarizes the interface of each agent. By separating their responsibilities and outputs, the architecture supports substitution of different models, logging of intermediate states, and detailed error analysis. Figure 3.1 illustrates the sequential data flow through the five agents for both audio and text inputs. The next section formalizes four deployment strategies that instantiate this architecture with different orchestration patterns, trading off accuracy, latency, and computational cost.

Agent	**Input**	**Output**
STT	Audio signal	Transcribed text with metadata (confidence scores, timestamps)
RAG	Text (transcribed or direct input)	Top-k retrieved examples with relevance scores and template mappings
IE	Text + retrieved examples + template schema	Candidate slot–value pairs (null for unavailable fields)
CF	Raw slot–value pairs	Normalized values with schema validation and format standardization
VER	Normalized slot–value pairs	Verified template with confidence scores, field annotations, and clarification prompts

Table 3.1.: Invox's agent interfaces and data flow. The table summarizes how information moves through the five-agent pipeline by showing the expected input and output of each agent. Reading the table top to bottom reveals the transformation path from raw audio to a fully verified template, highlighting how each agent contributes a distinct and on-overlapping function within the overall system.

3.3. Architectural Strategies

While the modular five-agent pipeline defines the general structure of the *Invox* system, its effectiveness depends on how large language models (LLMs) are deployed within the *Information Extraction* and *Verification* stages. Different architectural strategies offer distinct trade-offs in terms of accuracy, robustness, latency, cost, and transparency. This section outlines four strategies and illustrates them using a common example to enable direct comparison.

For all examples, we use the following input derived from the MUC-4 benchmark: "*On March 3, 1992, in Bogotá, a powerful car bomb exploded outside the Ministry of*

Defense, damaging nearby buildings and injuring 25 people, though no fatalities were reported. Authorities suspect a left-wing guerrilla group, but responsibility remains unconfirmed."

This example introduces multiple attributes (date, location, event type, casualties, suspected perpetrators, and epistemic uncertainty), allowing us to highlight the strengths and weaknesses of each strategy under realistic conditions.

3.3.1. Strategy S1: Single-Pass (Full-Input, Single-LLM)

In the simplest approach, a single LLM receives the complete transcript—along with few-shot examples retrieved by the RAG agent—and directly generates all template fields in one inference pass. The extracted values then proceed through Consistency Formatting (CF) and Verification (VER) before finalization.

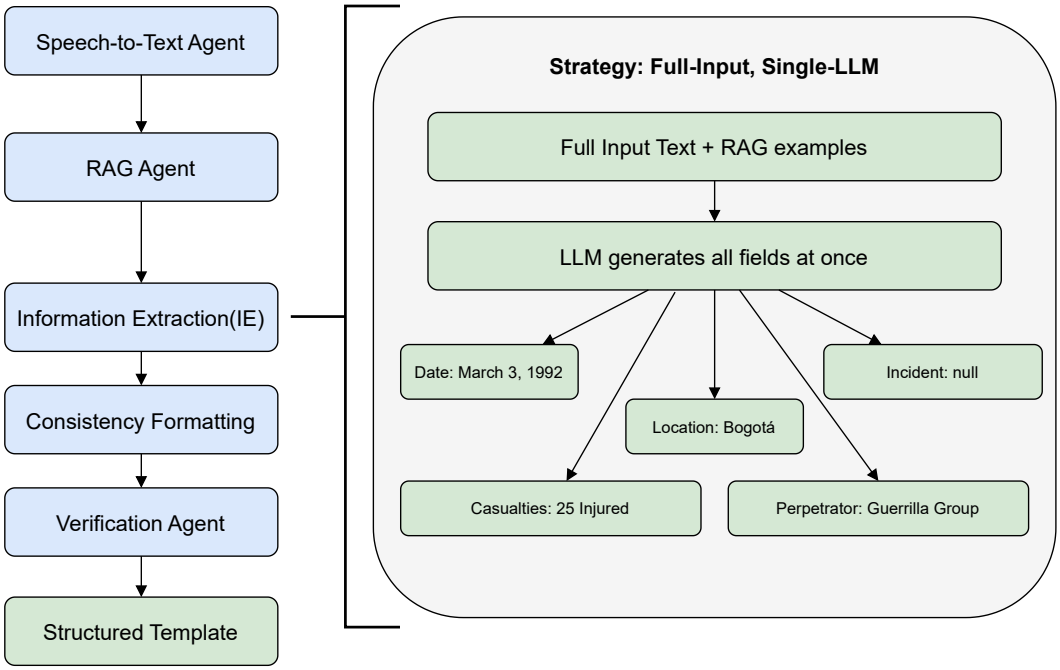

Figure 3.2.: Strategy: Full-Input, Single-LLM

Advantages. This strategy is computationally efficient, requiring only one IE model call. It minimizes latency and API costs, making it suitable for high-throughput scenarios. The inclusion of RAG-retrieved examples improves domain adaptation compared to zero-shot prompting, while CF and VER stages provide downstream quality assurance.

Limitations. If the LLM misinterprets a critical ambiguity—such as treating "suspected guerrilla group" as a confirmed perpetrator—the error propagates through CF (which normalizes the incorrect value) and may only be flagged by VER if confidence thresholds are exceeded. The single inference point offers no redundancy, making the system vulnerable to model-specific biases or hallucinations [15].

3.3.2. Strategy S2: Iterative (Slot-wise, Single-LLM)

Each template slot is extracted by an independent LLM call with a slot-specific prompt. Slots can be processed sequentially or in parallel. The RAG agent retrieves relevant examples once at the beginning, and these are included in every slot-level prompt. Results are then passed to CF and VER for normalization and validation.

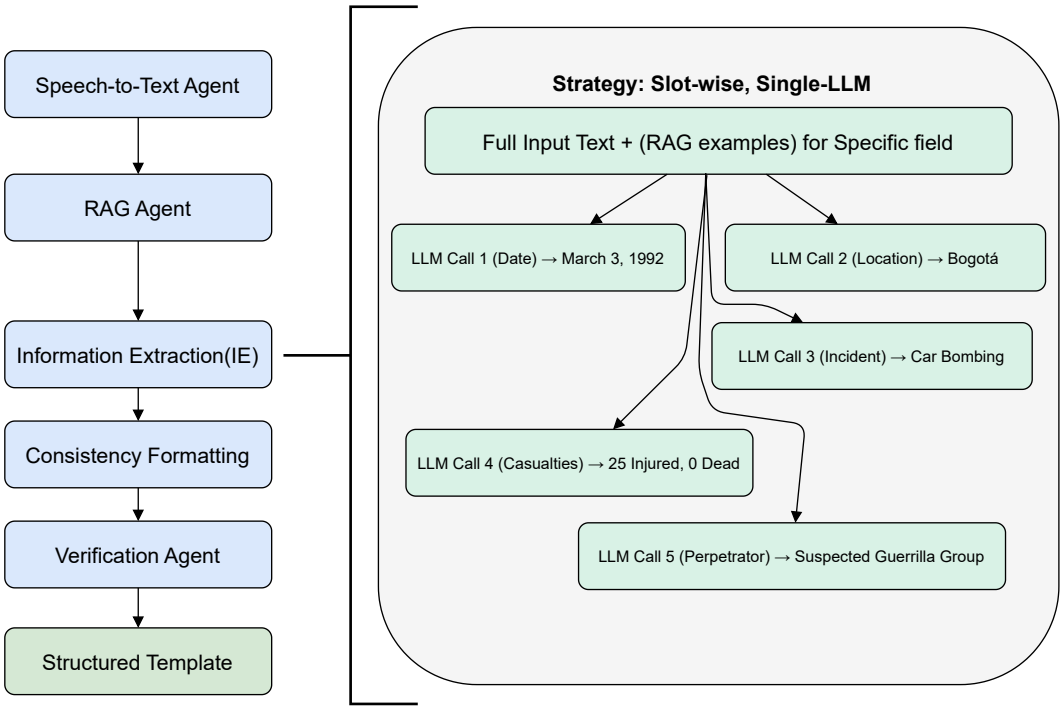

Figure 3.3.: Strategy: Slot-wise, Single-LLM

Advantages. Slot independence isolates errors: if the perpetrator field is uncertain, only that slot requires re-extraction. This improves transparency (R3) by making it clear which fields are problematic. Parallelization across slots can reduce wall-clock latency on multi-core systems, though total computational cost increases. The strategy also supports slot-specific prompt engineering, allowing prompts to be tuned for date extraction, entity recognition, or casualty parsing independently.

Limitations. The approach increases inference cost linearly with the number of slots. For templates with 15–20 fields, this becomes expensive. Additionally, slot-level prompts lack cross-field context: the model extracting casualties does not see the perpetrator information, which may reduce coherence in cases where fields are interdependent [46].

3.3.3. Strategy S3: Multi-LLM Consensus (Full-Input)

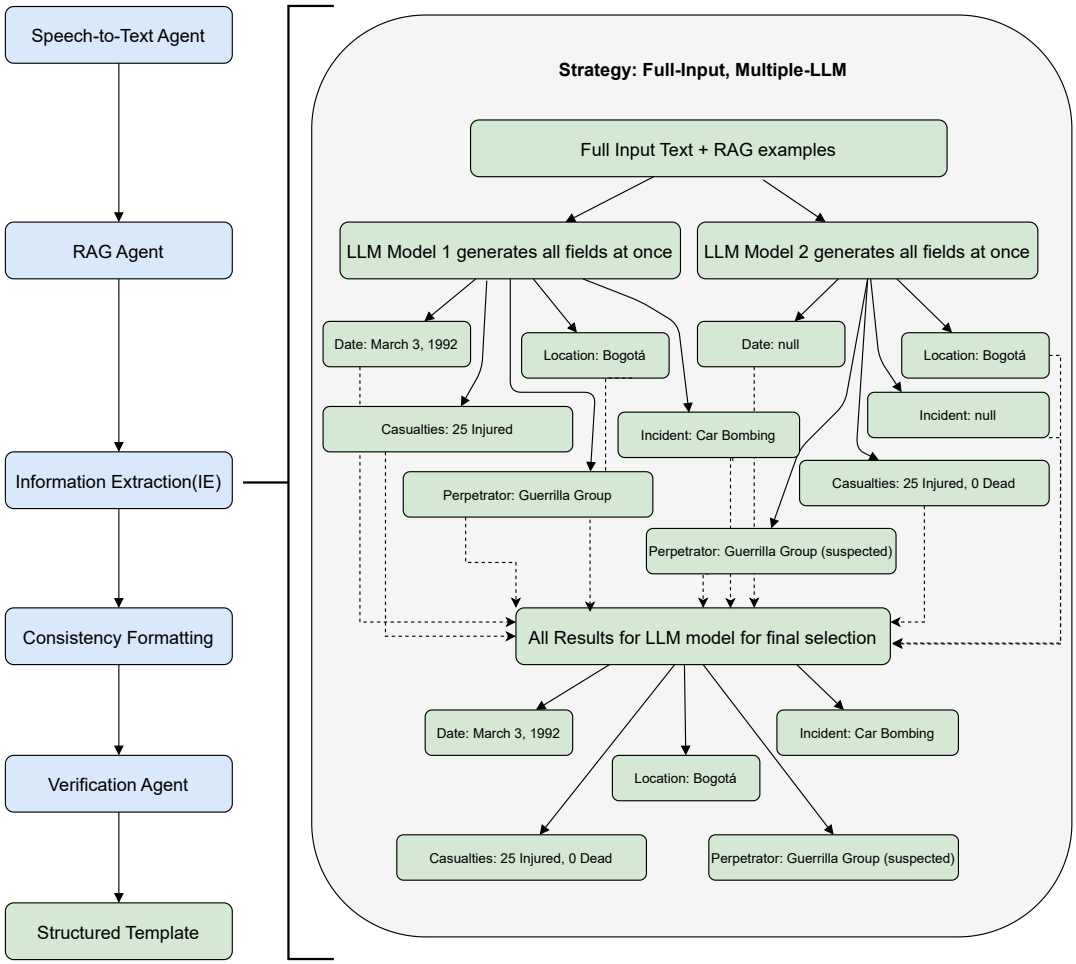

Figure 3.4.: Strategy: Full-Input, Multiple-LLM

Multiple LLMs—either different models (e.g., GPT-4, Claude, DeepSeek) or multiple runs of the same model with varied prompts—each generate a complete template from the full transcript and RAG-retrieved examples. Their outputs are compared by the Internal Verifier which has results from multiple llms, text input and the

RAG examples, which selects the most consistent or reliable values using consensus rules such as majority voting or confidence-weighted aggregation.

Advantages. Ensemble diversity reduces systematic bias: if one model hallucinates or misinterprets ambiguous phrasing, others may produce more faithful outputs. Consensus mechanisms increase robustness, particularly when models disagree on uncertain fields. This strategy also preserves cross-field context, as each model sees the full input during extraction [55, 37].

Limitations. Computational cost increases linearly with the number of models. If all models converge on the same misinterpretation—due to shared training biases or ambiguous input phrasing—consensus provides no advantage. The strategy also requires careful tuning of aggregation rules: simple majority voting may discard nuanced or hedged outputs (e.g., "suspected") in favor of overly confident but incorrect answers.

3.3.4. Strategy S4: Multi-LLM Consensus (Slot-wise)

Each slot is extracted independently by multiple LLMs, and consensus is reached per field before aggregating results. This combines the granularity of S2 (slot-wise processing) with the robustness of S3 (multi-model consensus). Their outputs are compared by the Internal Verifier which has results from multiple llms, text input and the RAG examples, which selects the most consistent or reliable values using consensus rules such as majority voting or confidence-weighted aggregation. Results are then passed through CF and VER.

Advantages. This strategy offers the highest reliability by applying ensemble methods at the most granular level. Slot-level consensus allows diverse models to specialize: one model may excel at date parsing, another at casualty extraction. Error isolation remains strong, as each field is independently verified. The approach is well-suited for safety-critical domains where accuracy justifies cost.

Limitations. Computational cost scales as (number of slots) $\times$ (number of models), making this the most expensive strategy. For a 15-field template with 3 models, this requires 45 LLM calls per document. Latency increases unless aggressive parallelization is employed. Additionally, consensus logic becomes more complex when models produce semantically equivalent but syntactically different outputs (e.g., "25 injured" vs. "25 people hurt"), requiring robust normalization before voting.

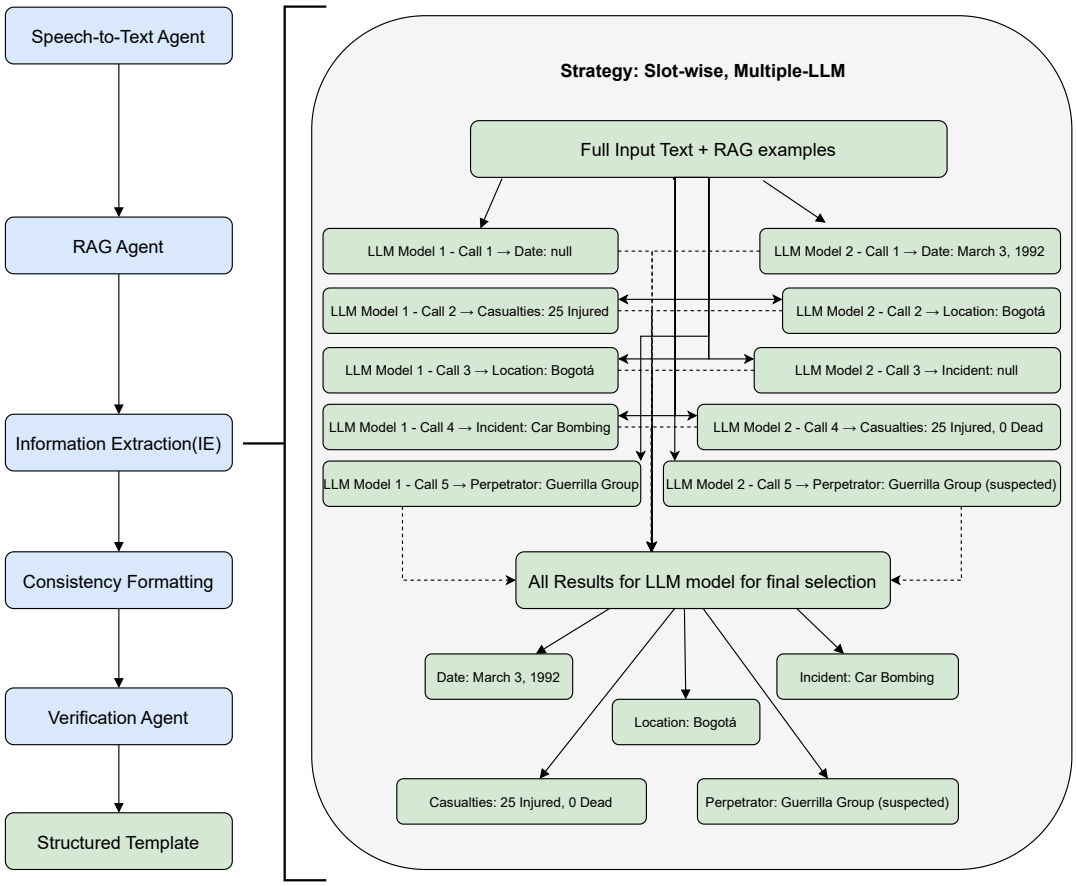

Figure 3.5.: Strategy: Slot-wise, Multiple-LLM

3.3.5. Summary and Trade-Off Analysis

Strategy	LLM Calls	Error Isolation	Robustness	Latency	Cost
S1: Single-Pass	1	Low	Low	Low	Low
S2: Iterative	n (slots)	High	Medium	Medium	Medium
S3: Consensus (Full)	m (models)	Low	High	Medium	High
S4: Consensus (Slot-Level)	$n \times m$	High	High	High	High

Table 3.2.: Comparison of architectural strategies, showing how different orchestration approaches trade off robustness, latency, and cost depending on the number of LLM calls and the level of error isolation.

Deployment requirements determine which strategy is most appropriate. **S1** is ideal for high-throughput scenarios where speed and cost outweigh accuracy needs. **S2** offers strong error isolation and fine-grained debugging, making it suitable when specific fields frequently cause failures. **S3** provides a balanced compromise between robustness and efficiency, supporting general-purpose use cases. **S4** delivers the highest reliability by combining per-slot consensus with multiple models, making it the preferred choice for safety-critical or compliance-sensitive workflows (e.g., medical records or regulatory reporting).

Chapter 5 empirically evaluates these strategies on the MUC-4 benchmark and quantifies their performance across the six requirements (R1–R6) established in Chapter 2.

3.4. System Architecture and Design

This section presents the high-level architecture of the Invox system using the C4 model, which structures architectural documentation into hierarchical levels: context (system boundaries and external actors), containers (major components and their interactions), and process (dynamic execution flow). These views provide a comprehensive understanding of how the conceptual design translates into a deployable architecture.

3.4.1. Context View: System Boundaries and External Actors

The context diagram (Figure 3.6) positions Invox within its operational environment. The system interacts with three categories of external actors:

Primary users are domain experts—such as factory operators, medical staff, or administrative personnel—who provide unstructured input (speech or text) and review populated templates. They interact with the system through a web-based interface or mobile application.

Data sources include historical template repositories, domain-specific glossaries, and organizational knowledge bases. These are accessed by the RAG agent to retrieve contextually relevant examples. In regulated environments, these repositories may reside behind secure access controls.

External services consist of third-party APIs for LLM inference (OpenAI GPT-4, Anthropic Claude, DeepSeek), speech recognition (OpenAI Whisper), and search infrastructure (OpenSearch). The system communicates with these services over REST APIs.

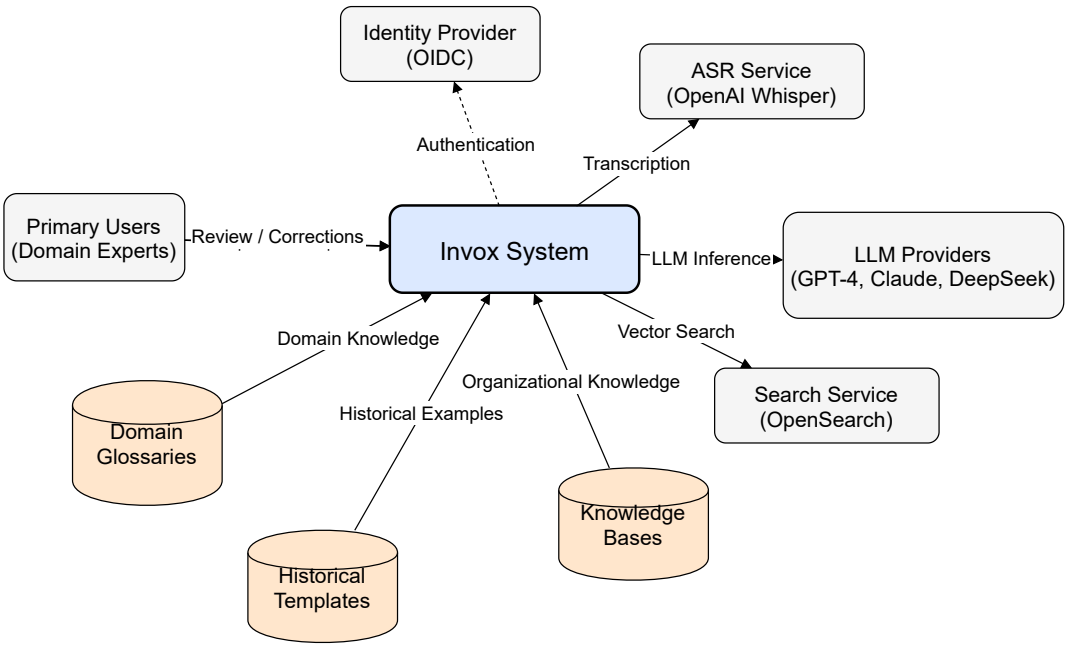

Figure 3.6.: Context diagram illustrating the system boundary of Invox and its interactions with external actors such as users, authentication services, LLM providers, speech-to-text components, and organizational knowledge sources, highlighting all inputs the system depends on.

3.4.2. Container View: Internal Component Structure

The container diagram (Figure 3.7) outlines Invox's main components and the flow of requests through the system.

Client access. Users interact with Invox via the *API Gateway*, the entry point for all structured requests over HTTP/HTTPS/tRPC. Audio may also be sent directly to the *STT Service* for transcription.

STT Service. Speech-to-text processing is handled by a Whisper-based component that wraps the external ASR provider and returns transcriptions with metadata such as confidence scores and timestamps. Output is either returned to the client or forwarded into the pipeline.

RAG Service. Once a request enters the backend, the first step is retrieval. The input is embedded and matched against the *Vector Index* to obtain semantically relevant examples that support downstream extraction.

IE Service. Using this enriched context, the IE component builds prompts and invokes the configured LLM providers. Depending on the selected strategy (S1–S4),

it may perform a single call, slot-wise calls, or multi-model consensus to produce candidate slot–value pairs.

CF Service. Extracted values are then normalized and validated. This stage standardizes formats (e.g., dates, entities) and enforces schema constraints to produce a consistent intermediate template.

VER Service. Verification is performed as a dedicated internal step. It assigns confidence scores, checks completeness and consistency, flags problematic fields, and triggers clarification prompts when needed. Verified templates, along with metadata, are stored in both the *Template Repository* and the *Vector Index*.

Data stores. Invox uses two storage layers: a *Template Repository* for finalized templates and a *Vector Index* for dense embeddings used during retrieval.

End-to-end flow. After verification, the template is persisted, indexed, and returned to the client via the API Gateway. Manual edits remain possible before final submission.

3.4.3. Process View: Workflow and Decision Logic

The process diagram (Figure 3.8) illustrates the dynamic execution flow that connects all stages of the Invox pipeline.

Input reception. The workflow starts when users submit either audio or text. Audio is routed through the STT service for transcription, while text requests enter through the API Gateway and continue directly into the backend pipeline.

Retrieval. After the initial preprocessing, the RAG service performs a similarity search over the vector index to obtain the top-k relevant examples. If no suitable matches are found, the system automatically reverts to a zero-shot prompting.

Extraction. With the retrieved examples available, the IE service invokes one or more LLM calls according to the selected strategy. For multi-model or consensus configurations, all model responses are gathered before the pipeline proceeds.

Formatting. The next stage involves normalization and schema enforcement. CF standardizes extracted values, validates them against the schema, and flags any elements that cannot be normalized for later user review.

Verification. VER checks the template for consistency, completeness, and confidence. Outcomes include: (1) a verified template returned directly to the user, (2) a clarification loop that requests missing information and re-runs IE $\rightarrow$ CF $\rightarrow$ VER, or (3) a low-confidence output with uncertain fields highlighted for review.

Finalization. Once verification is complete, the resulting template is stored in the repository, indexed for future retrieval, and returned to the client. Users may still make manual adjustments before final submission.

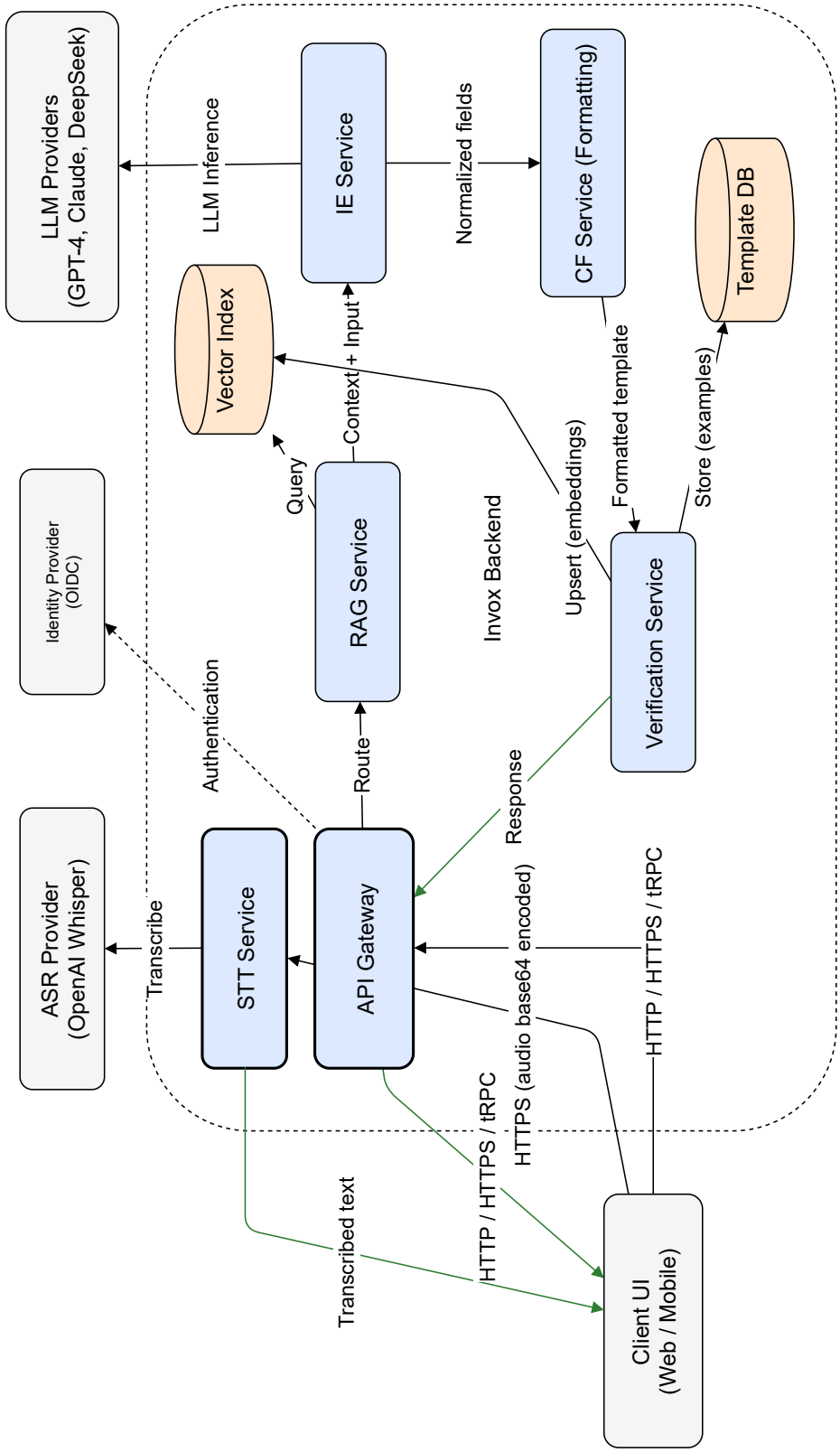

Figure 3.7.: Container diagram illustrating how requests move through Invox's internal services—STT, RAG, IE, CF, and VER—and how these components interact with external dependencies such as LLM providers, storage layers, and the client-facing API Gateway.

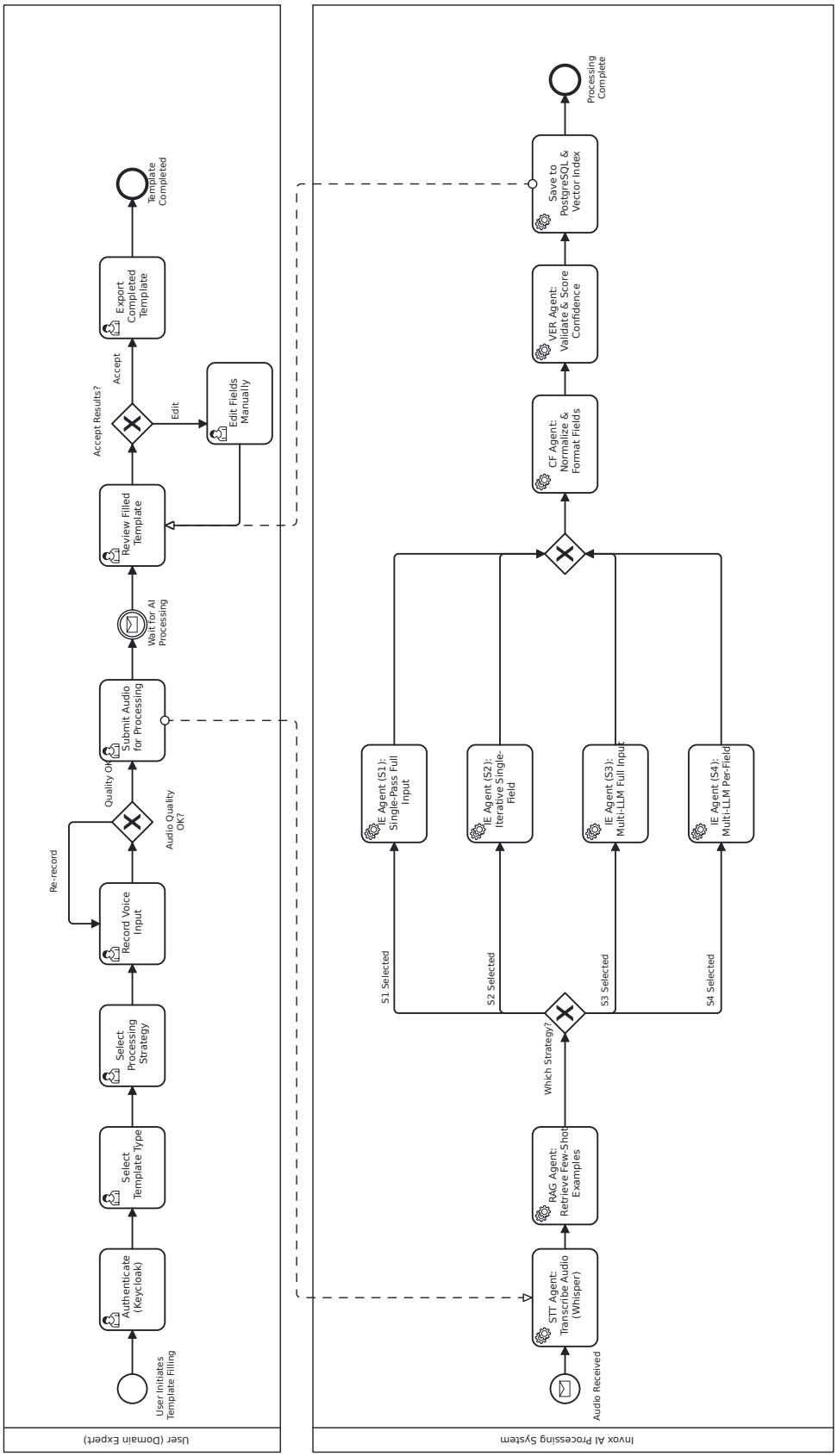

Figure 3.8.: BPMN diagram illustrating the end-to-end workflow of Invox, including input handling, retrieval, extraction, formatting, verification, and the decision logic that governs clarification loops and final template delivery.

4. Implementation

Building on the conceptual architecture introduced in the previous chapter, this section moves from theory to implementation and demonstrates how the multi-agent pipeline was realized in practice. It details the chosen technologies, system workflow, concrete agent implementations, and the execution of the four architectural strategies. The chapter concludes with a discussion of key design decisions and challenges encountered during development.

4.1. Technology Stack and Rationale

Implementation of the *Invox* system follows a modular monolith architecture that balances development simplicity with operational flexibility. A single Node.js application hosts all five agents, while specialized services—such as vector search, embedding generation, and external LLMs—operate externally. This hybrid approach preserves modular boundaries without incurring the operational overhead typical of fully distributed microservices [18, 34].

Architecture Pattern: Modular Monolith with External Services

Invox adopts a modular monolith pattern in which the STT, RAG, IE, CF, and VER agents are implemented as cohesive modules within one Node.js service. This yields a unified codebase, consistent versioning, simplified debugging, and low-latency in-process communication. Clear module interfaces maintain separation of concerns despite co-location.

External components are used where the benefits are decisive. **OpenSearch** provides vector similarity search; a dedicated embedding service generates domain-relevant vectors; and LLM providers (OpenAI, Google) perform model inference. This division allows the core pipeline to remain lightweight while delegating specialized computation to optimized services.

Containerization and Deployment

Deployment relies on **Docker** to ensure reproducibility, isolation, and consistent behavior across environments. Containers encapsulate dependencies, simplify distribution, and enable horizontal scaling by replicating services. **Docker Compose** orchestrates the multi-service environment, coordinating the main application alongside OpenSearch and auxiliary components.

Backend Implementation

Backend development is carried out in **Node.js** with **Express**, providing an adaptable foundation for the multi-agent workflow. LLM invocation is abstracted through the **Vercel AI SDK**, which enables a plugin-like architecture for switching between OpenAI, Google, Anthropic, and future providers with minimal changes. Type-safe communication between backend and frontend is achieved using **tRPC**, with **JSON** as the shared data format for inter-agent communication.

External Service Integration

OpenSearch acts as the vector index for the RAG agent, storing dense embeddings and enabling efficient retrieval of semantically related examples. Vector indexing significantly accelerates similarity search and supports context-rich extraction. Embeddings are generated using the `intfloat/multilingual-e5-large-instruct` model (1024 dimensions), selected for its strong multilingual performance and favorable trade-off between accuracy and computational cost. This model provides robust semantic representations for diverse input conditions.

Frontend Interface

The user interface is built in **React**, chosen for its wide adoption, strong ecosystem, and suitability for building interactive components that match the system's modular structure. React's predictable rendering model ensures responsive interactions when reviewing extracted fields and modifying templates.

Data Persistence and Retrieval

Persistent storage is managed by **PostgreSQL**, which offers strong consistency, reliability, and efficient querying for structured data. Finalized templates are stored relationally, while OpenSearch serves as the retrieval layer for RAG by indexing

dense vector embeddings. This combination provides both robust persistence and high-quality semantic search.

Authentication and Security

Authentication and authorization are handled by **Keycloak**, which supports OAuth2 and OpenID Connect [23]. Role-based policies ensure that only authorized users can submit, review, or modify templates, supporting secure multi-user workflows.

Architecture Rationale

A modular monolith provides a pragmatic balance between maintainability, performance, and future extensibility. It avoids the operational complexity of microservices while retaining clear module boundaries. The plugin-based LLM integration ensures adaptability as new models become available, and containerized deployment guarantees reproducibility and reliability across environments. Together, these choices provide a scalable and future-proof foundation for *Invox*.

4.2. System Workflow

This section describes the operational workflow of the Invox system, tracing the transformation of unstructured input into verified structured templates. The workflow implements the conceptual pipeline from Chapter 3 through a sequence of processing stages with user-driven quality control.

End-to-End Processing Flow

The end-to-end workflow of Invox's five-agent pipeline is illustrated in Figure 3.1. Starting from user input, the system proceeds through six sequential stages:

1. **Input Reception**: Users provide input through speech (audio recording) or direct text entry via the web interface. The system accepts both modalities and routes them appropriately through the processing pipeline.
2. **Speech Recognition**: For audio inputs, the system invokes the Whisper ASR service to generate transcripts with confidence scores and timestamps. Text inputs proceed directly to the next stage.
3. **Context Retrieval**: The RAG agent queries the vector database to retrieve the most relevant historical templates and examples, providing contextual guidance for the extraction process.

4. **Information Extraction**: Based on the selected strategy (S1–S4), the system processes the input:
   - **S1 (Single-Pass Full Input)**: A single language model extracts all template fields in one pass
   - **S2 (Iterative Single-Field Processing)**: Multiple LLM calls extract each field independently
   - **S3/S4 (Multi-LLM Consensus (Full), Multi-LLM Consensus (Iterative))**: Multiple models generate candidate answers, with a dedicated judge LLM selecting the optimal response through comparative analysis
5. **Consistency Enforcement**: The CF agent normalizes dates to ISO format, standardizes entity names, and enforces schema compliance through deterministic transformation rules.
6. **Quality Assessment and Presentation**: The system assigns confidence scores to each extracted field and presents the complete template to the user for review and potential correction.

User-Driven Quality Control

The system employs a user-in-the-loop approach where quality assurance is primarily driven by human judgment:

Confidence-Based Presentation: Fields with lower confidence scores are visually highlighted, directing user attention to potentially problematic extractions.

Multi-Model Decision Making: In strategies S3 and S4, the judge LLM evaluates candidate answers from multiple models, selecting the most appropriate response based on coherence, accuracy, and alignment with the input context.

User-Initiated Reprocessing: If users are unsatisfied with results, they can modify their input and resubmit for reprocessing, creating an iterative refinement cycle driven by human assessment rather than automated detection.

Knowledge Accumulation

Successfully processed templates are indexed in the vector database, enabling the RAG agent to leverage an expanding knowledge base for improved context retrieval in future processing cycles.

4.3. Agent Implementation

This section details the concrete implementation of each agent in the Invox system, covering their API contracts, processing logic, and integration patterns. Each agent exposes well-defined TypeScript interfaces and follows consistent error handling patterns. The implementation is designed to be domain-agnostic, supporting arbitrary template schemas across healthcare, manufacturing, administrative, and other domains.

4.3.1. Speech-to-Text (STT) Agent

The STT agent provides a single entry point `transcribeForm` that converts audio inputs to structured transcripts using OpenAI's Whisper model.

Listing 4.1 defines the input and output schema for the STT agent.

Listing 4.1: STT agent API contract

```
interface TranscribeInput {
  file: {
    base64: string;
    mimetype: "audio/wav" | "audio/mpeg" | "audio/m4a";
    originalname: string;
  };
}

interface TranscribeResponse {
  transcript: string;
  language: string;
  durationInSeconds: number;
  segments?: Array<{
    start: number;
    end: number;
    text: string;
    confidence?: number;
  }>;
}
```

The `TranscribeInput` interface encapsulates the uploaded audio file as a Base64-encoded payload with an associated MIME type and original filename. The interface `TranscribeResponse` returns the full transcription in `transcript`, the detected language as an ISO 639-1 code in `language`, and the total audio duration in seconds via `durationInSeconds`. The optional `segments` array provides time-aligned fragments

of the transcription that can be used by downstream components for error analysis and selective verification.

Implementation. The agent decodes Base64 audio data into buffers and invokes the Whisper API via the Vercel AI SDK. Long audio files (>25 MB or >10 minutes) are automatically chunked with a 10% overlap to preserve context at segment boundaries. When available, the agent returns time-aligned segments with confidence scores, enabling downstream agents to trace extracted values back to specific audio regions. Failed transcriptions (e.g., due to excessive noise or unsupported formats) result in structured errors that the orchestrator can handle gracefully, either by requesting re-recording or by falling back to manual text entry.

Configuration. Temperature is set to 0 to obtain deterministic output across repeated runs. Language detection is performed automatically unless explicitly overridden by the client. Voice activity detection (VAD) is applied to filter out silence and non-speech segments, which improves both transcription quality and processing efficiency.

4.3.2. Retrieval-Augmented Generation (RAG) Agent

The RAG agent implements semantic search over historical templates through the `getFewShotsFromTranscript` function, which retrieves contextually relevant examples to guide the information extraction process.

Listing 4.2 defines the input and output schema for the RAG agent.

Listing 4.2: RAG agent API contract

```
interface RAGInput {
  transcript: string;
  fields: FormTemplateField[];
  k?: number;
}

interface FewShot {
  text: string;
  expected: Record<string, {
    value: unknown | null;
    evidence?: { transcriptSnippet?: string };
  }>;
}
```

The `RAGInput` interface specifies the data required for semantic retrieval. The `transcript` field contains the input text used for similarity search, while `fields` holds the target form schema as an array of `FormTemplateField` objects. The

optional parameter `k` controls how many example instances are retrieved for few-shot prompting; if omitted, a default of five examples is used to keep prompt size manageable.

The `FewShot` interface represents a single retrieved example. The `text` property contains a truncated version of a historical transcript, and the `expected` map associates each field identifier with its normalized value. Each entry in `expected` may also contain `evidence` in the form of a `transcriptSnippet`, which highlights the segment of the original transcript that supports the stored value.

Implementation. The agent generates dense vector embeddings using OpenAI's `text-embedding-3-large` model. A key implementation detail is the embedding generation process: the input text is prefixed with `templateId=$TEMPLATE_ID` to create template-aware embeddings that improve retrieval relevance within specific template contexts. The agent then performs k-nearest neighbor search using cosine similarity in OpenSearch with the `knn_score` script. The search query filters results by template ID to ensure schema compatibility and uses a `script_score` approach for efficient vector similarity computation.

Result processing. Retrieved documents undergo post-processing where transcript texts are truncated to a fixed maximum length (default: 1200 characters) to maintain prompt efficiency. The `toExpectedShape` function maps raw database results to the expected field schema, ensuring that only fields present in the current template schema are included in the output.

Error handling. The implementation includes robust error handling for embedding generation failures and OpenSearch connectivity issues. If retrieval fails, the system gracefully falls back to zero-shot extraction, ensuring continuous operation despite temporary infrastructure problems.

4.3.3. Information Extraction (IE) Agent

The IE agent implements four extraction strategies through unified functions corresponding to S1–S4. All strategies share a common preprocessing pipeline and differ only in their orchestration of LLM calls.

Listing 4.3 defines the core input and output schema for the IE agent.

Listing 4.3: IE agent core API contract

```
interface IEInput {
  oldTranscript?: string;
  newTranscript: string;
  fields: FormTemplateField[];
  currentValues?: Record<string, CurrentFieldValue>;
  strategy: "S1" | "S2" | "S3" | "S4";
```

```
  fewShots?: RAGOutput["examples"];
  locale?: string;
  timezone?: string;
  templateId?: string;
}

interface IEOutput {
  filled: Record<string, FilledField>;
  model: string;
  confidence?: Record<string, number>;
}
```

The `IEInput` interface captures all information required for a single extraction step. The `newTranscript` field holds the latest user input and is treated as the primary source of new information, while `oldTranscript` optionally provides previous context for incremental updates. The `fields` array contains the target template schema as `FormTemplateField` definitions. The optional `currentValues` map stores previously filled field values, including metadata about their origin and lock status. The `strategy` parameter selects one of the four extraction strategies (S1–S4). Optional `fewShots` provide retrieved examples from the RAG agent, and `locale`, `timezone`, and `templateId` supply environment and logging metadata.

The `IEOutput` interface aggregates the final extraction result. The `filled` map associates each field identifier with a `FilledField` object, while `model` records the model identifier or combination of models used (e.g., for ensemble strategies). The optional `confidence` map stores per-field confidence scores normalized to the interval $[0, 1]$. The detailed structure of `CurrentFieldValue` and `FilledField` is provided in Appendix A.2.

Strategy implementations.

S1: Single-Pass Full-Input (`singleLlmAllField`) This strategy constructs a comprehensive prompt requesting all template fields simultaneously. The prompt includes task instructions emphasizing extraction from `newTranscript` only, few-shot examples from the RAG agent (up to three examples showing input–output mappings), field definitions with type constraints and allowed options for enum fields, current field values with their lock status, and both old and new transcripts (old for context, new for extraction).

The agent uses Zod schema validation to ensure type-safe outputs. Dates must match the `YYYY-MM-DD` format, numbers must be finite, and enum values must belong to the predefined set. The LLM is instructed to return a JSON object with one key per field identifier. Post-processing normalizes values (for example, joining arrays to comma-separated strings for multi-value fields and trimming whitespace) and applies

lock-aware merging: if a field is locked or if the new transcript does not mention a field, the current value is preserved.

S2: Iterative Single-Field (`singleLlmOneField`) This strategy processes each field independently using a field-specific prompt. For each field, the agent filters RAG examples to include only those that populated this particular field, constructs a focused prompt with field-specific guidelines and type constraints along with one or two relevant examples, calls the LLM with a schema of the form `{value, confidence}`, normalizes the returned value according to the field type, and applies lock-aware update logic.

Fields are processed in parallel using `Promise.all`, which reduces wall-clock latency on multi-core systems. Each field call is wrapped in its own error boundary: if extraction for one field fails (for example, due to an API timeout or parsing error), the remaining fields proceed unaffected and the failed field retains its current value. This isolation improves robustness compared to S1, where a single malformed response can invalidate the entire output.

S3: Multi-LLM Consensus Full-Input (`dualLlmAllField`) S3 runs two models in parallel—GPT-4 and Gemini 2.0 Flash by default—each generating a complete template using the same prompt as S1. The two candidate outputs are then passed to a verification agent that implements consensus logic with rule-based checks to validate that locked fields were not overwritten, enum values remain within their allowed set, and dates are parseable.

LLM-based reconciliation uses a judge model (GPT-4o-mini by default) that compares the two candidate outputs field-by-field and selects one of four decisions: `gpt` (use GPT's value), `gemini` (use Gemini's value), `merge` (combine both values for multi-value text fields), or `keep_current` (neither candidate is reliable; preserve the existing value). The verifier prompt includes both candidate outputs, the transcript, and field-level metadata (type, required status, current value). For multi-value fields, the merge operation deduplicates and quality-filters entries, discarding empty strings or placeholders.

S4: Multi-LLM Consensus Per-Field (`multiLlmOneField`) S4 combines the granularity of S2 with the robustness of S3. For each field, the agent runs GPT and Gemini in parallel with field-specific prompts, analogous to S2, and then passes both candidate values to a per-field verifier. The verifier applies the same decision logic as in S3 but focuses on a single field at a time, which reduces prompt size and often improves decision quality. The final value is returned together with an aggregated confidence score.

This strategy has the highest computational cost—*(number of fields)* × *(number of models)* LLM calls—but offers maximum reliability by isolating errors at the field level and leveraging ensemble diversity for each slot.

Shared preprocessing. All strategies share common logic for incremental updates (only `newTranscript` is treated as the source of new information), lock enforcement (fields marked `locked = true` are never overwritten), change detection (the `changed` flag is set only if the final value differs from the `currentValue`), and type-specific normalization to ensure consistent representations across strategies.

4.3.4. Consistency Formatting (CF) Agent

The CF agent provides `normalizeValueForField` for deterministic value standardization based on field type. Its purpose is to turn potentially noisy or ambiguous model outputs into canonical representations that can be stored and compared reliably.

The CF agent applies a set of type-specific transformations. For date fields, natural-language expressions (e.g., "March 3, 1992") are converted into ISO format (`YYYY-MM-DD`). For numeric fields, textual inputs such as "25 people" are reduced to finite numbers (e.g., `25`). Enum fields are checked against the list of allowed options in a case-insensitive manner, and invalid values are rejected. List-like or multi-value fields are converted into compact, human-readable strings (e.g., "A, B") by filtering out null, empty, or placeholder entries and joining the remainder.

For text and textarea fields, CF trims leading and trailing whitespace and applies Unicode normalization (NFC form) to ensure consistent representation of accented characters and similar multi-codepoint symbols across different input methods.

Implementation. The transformations are implemented using standard libraries (e.g., Luxon for date parsing) and simple regex-based extraction for numbers, combined with case-insensitive matching for enum validation. Ambiguous values that cannot be normalized deterministically (e.g., "`3/4/92`" with unclear month/day order) are not silently corrected: instead, they are flagged so that the verification agent can mark the affected field for clarification or manual review. The CF agent is intentionally non-stochastic: given the same input, it always produces the same output. The concrete helper functions are listed in Appendix A.1.

4.3.5. Verification Agent (VER)

The verification agent serves as the final decision layer before values are shown to the user or persisted. It handles both single-model outputs (via `runVerifier`) and multi-model consensus for S3/S4 (via `runEnsembleVerifier`). Inputs include candidate field values, the template schema, the combined transcript, and any previously locked

or prefilled values; outputs contain the consolidated field map, calibrated confidence scores, and issue flags.

Consensus logic (S3/S4). For ensemble strategies, VER compares candidates from multiple models using a judge LLM. The judge receives the competing values, relevant transcript excerpts, schema rules, and lock status, and returns a structured per-field decision (choose GPT, choose Gemini, merge, or keep the existing value). If merging is selected, normalization and deduplication ensure schema-compliant output.

Single-model verification (S1/S2). When only one candidate is available, VER performs deterministic checks such as required-field completeness, type validation, and basic cross-field consistency. Low-confidence or weakly grounded values are flagged for manual review, while stronger inconsistencies trigger clarification prompts.

Overall, VER standardizes outputs across strategies, enforces schema correctness, and records the rationale behind each decision to support transparency and user trust.

4.3.6. Agent Orchestration

The `Orchestrator` class coordinates agent execution through typed procedure calls exposed via tRPC endpoints. The orchestrator is strategy-aware: it selects the appropriate IE function (S1–S4) based on client configuration and manages the end-to-end pipeline flow.

Conceptually, the pipeline executes in six steps. First, the orchestrator performs optional transcription using the STT agent when audio is provided; otherwise, it wraps the raw text and language into a synthetic transcript object. Second, it invokes the RAG agent to retrieve few-shot examples conditioned on the transcript, template fields, and template identifier. Third, it calls the IE agent using the strategy specified by the client, passing along the transcript, schema, current values, and retrieved examples.

In the fourth step, the CF agent applies deterministic normalization to the extracted values, ensuring consistent formatting across fields and runs. Fifth, the verification agent checks completeness and consistency, potentially adjusting field-level confidences and emitting structured issues. Finally, the orchestrator inspects the verification result for issues that require re-querying (e.g., fields marked with the `"requery"` action). If necessary, it can trigger an additional IE+VER cycle using hints from the verifier before returning the final `FinalTemplate` object to the client.

The complete TypeScript implementation of the orchestration logic, including the `processTemplate` method and the `runStrategy` helper, is provided in Appendix A.4.

4.4. Strategy Implementation

This section presents the practical realization of the four architectural strategies introduced in Section 3.3. While the conceptual motivations were discussed earlier, the focus here is on their concrete execution: preparation of transcripts and few-shot examples, invocation of language models, and consolidation of outputs. Simplified TypeScript sketches of the core procedures are included in Appendix A.5.

4.4.1. S1 Implementation: Single-Pass Full-Input

The S1 strategy is implemented through the `singleLlmAllField` function, which processes the entire template in a single model call. The system first constructs a combined transcript and retrieves a small set of examples from the RAG agent. All fields are represented within a unified schema, and the prompt incorporates task instructions, field definitions, transcripts, current values, and retrieved examples.

A single `generateObject` call produces a complete candidate template. The output is subsequently normalized via the CF agent's rules and verified once in aggregate. The strategy is therefore characterized by a single prompt, a single model invocation, and batch-style post-processing. While efficient and simple to orchestrate, S1 provides limited error isolation, as malformed responses may affect all fields simultaneously.

4.4.2. S2 Implementation: Iterative Single-Field

S2, implemented by `singleLlmOneField`, introduces field-level decomposition. After preparing the combined transcript and retrieving few-shot examples, the system constructs individual prompts for each field, containing only the information relevant to that field, including filtered examples.

Each field is processed by an independent asynchronous task. Outputs pass through normalization and are merged into the final template. Errors are isolated to the corresponding field; upon failure, the system reverts to a sensible fallback (typically the previously known value). S2 thus provides improved robustness relative to S1 through parallelized extraction, field-specific prompting, and localized error handling.

4.4.3. S3 Implementation: Multi-LLM Consensus

The S3 strategy is implemented using `dualLlmAllField`, which performs model-level parallelism. A unified prompt and schema are constructed as in S1, but two model providers (e.g., GPT-4 and Gemini 2.0 Flash) are invoked in parallel. Because both

models receive identical prompts, differences in their outputs reflect model behaviour, not prompt variation.

The two candidates are forwarded to an ensemble verifier, which applies rule-based checks and a judge LLM to select, for each field, the more reliable value or retain the original value when neither candidate is adequate. A finalized template and confidence metadata are produced. S3 is therefore defined by parallel model inference, an explicit consensus phase, and structured documentation of ensemble decisions.

4.4.4. S4 Implementation: Multi-LLM Per-Field

S4, implemented through `multiLlmOneField`, combines field-wise decomposition with multi-model consensus. For each field, the system constructs a tailored prompt and invokes both model providers. Their outputs are passed to a per-field verifier, which applies the same decision logic as in S3 but at field granularity.

Fields are processed sequentially to avoid rate-limit interactions and to simplify auditing. Each iteration returns a `FilledField` entry after consensus and normalization. This yields the highest robustness and interpretability among the strategies, though at the cost of substantially increased computational overhead.

4.4.5. Strategy Orchestration Infrastructure

All strategies are executed through a dedicated `StrategyOrchestrator`, which exposes a uniform entry point (`executeStrategy`). The orchestrator maps the selected identifier (S1–S4) to a concrete implementation adhering to a shared `StrategyImplementation` interface.

Common infrastructure ensures consistent error handling, telemetry, and configuration. Strategy-specific parameters (model names, temperatures, provider settings) are injected through environment-based configuration, and all outputs are serialized into a unified `FinalTemplate` structure, ensuring that downstream evaluation and analytics remain strategy-agnostic. A minimal TypeScript sketch of the orchestrator is provided in Appendix A.5.

4.5. User Interface

The user interface implements the human-in-the-loop quality control paradigm established in Section 4.2. The interface design focuses on efficient template filling through voice input and interactive correction workflows.

4.5.1. Authentication and Template Access

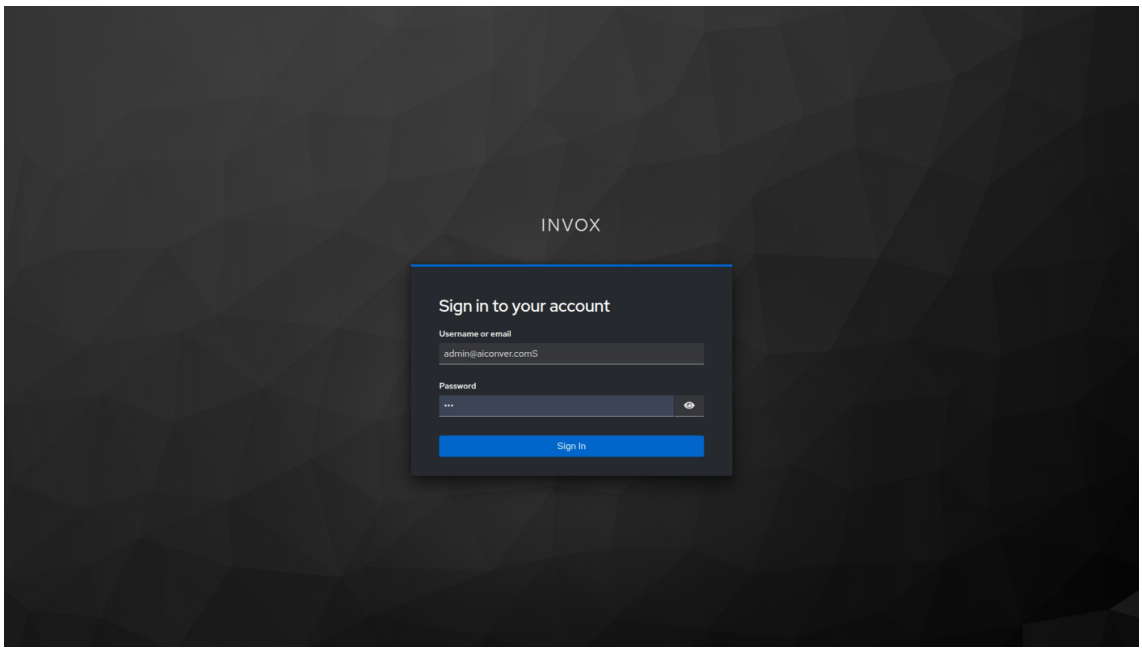

Figure 4.1.: INVOX authentication interface with Keycloak integration

The system implements role-based access through a centralized authentication interface (Figure 4.1). Users authenticate via Keycloak-managed credentials, which determine which template types they can access. Security personnel see incident reporting templates, while medical staff access patient handover forms. Users cannot create new template schemas through the interface—they can only fill templates assigned to their role.

4.5.2. Template Filling Interface

The interface consists of three main components (Figure 4.2): the audio recording panel at the bottom-left, the structured template form on the right, and an interactive chat panel for clarifications.

4.5.2.1. Audio Recording Panel

The recording panel provides a "Start" button to initiate audio capture. During recording, the button changes to "Stop & Process." When clicked, the system transcribes the audio and extracts template values using the selected strategy. A strategy selector menu allows users to choose between S1, S2, S3, or S4 before

Figure 4.2.: Template-filling interface used in Invox. The left panel offers voice recording and chat interaction for describing an incident, while the right panel displays the structured form populated by the pipeline. The layout illustrates how transcription, automated extraction, and user review occur within a unified workflow.

processing, enabling them to balance speed versus accuracy based on the template's importance.

4.5.2.2. Structured Template Form

The right side shows the structured template, with fields grouped by category (Incident Type, Perpetrator, Target, Victim, Weapon). As audio is processed, the system auto-populates the fields using extraction results. Each field uses an appropriate widget—dropdowns for enumerations, text areas for descriptions, and dedicated inputs for dates or numeric values. Confidence indicators mark AI-filled entries, and low-confidence fields are highlighted for review.

Users may edit values at any time, lock fields to protect them during later recordings, and inspect whether a value was extracted or manually entered. Before submission, the form validates all entries against the template schema.

4.5.2.3. Interactive Chat Panel

The chat panel provides real-time feedback during template filling. When the system identifies missing required fields, conflicting information, or ambiguous extractions, it posts messages requesting clarification. For example, if the perpetrator field contains uncertain information, the chat prompts the user to provide more specific details. This interactive guidance reduces iteration cycles by identifying issues immediately rather than after submission.

4.5.3. Template Export

Once users complete template review, the interface provides export options in JSON, Excel, and PDF formats through a download button group. All templates are automatically persisted to PostgreSQL upon processing, ensuring data preservation regardless of export actions. Having established the complete implementation of the Invox system—from backend agent architecture to frontend user interface—the next chapter evaluates this implementation empirically. Chapter 5 applies the system to the MUC-4 benchmark and measures its performance against the six requirements (R1–R6) established in Chapter 2.1. The evaluation compares the four architectural strategies quantitatively, examining their trade-offs in accuracy, cost, latency, and user satisfaction.

5. Evaluation

The goal of this chapter is to evaluate the proposed *Invox* system and analyze its performance across multiple extraction strategies using the **MUC-4 benchmark dataset**. The evaluation aims to measure how effectively the system transforms unstructured, noisy text into structured templates under real-world conditions. To this end, the assessment focuses on the dimensions of **accuracy**, **consistency**, **latency**, **cost**, and **modularity**, in direct correspondence with the requirements defined in Chapter 2.

Unlike traditional information extraction evaluations that rely on strict lexical overlap, the MUC-4 dataset poses a distinctive challenge: both the gold standard and system predictions may include *multiple valid fillers* per slot, expressed through paraphrases or partial phrases. A purely string-based comparison would therefore underestimate performance. To account for this, all evaluations employ an **embedding-based semantic framework** in which cosine similarity between gold and predicted values captures conceptual correctness rather than surface form matching. This provides a fairer, more robust measure of information extraction quality.

The chapter proceeds as follows. Section 5.1 introduces the MUC-4 dataset and the preprocessing pipeline applied prior to evaluation. Section 5.2 defines the semantic similarity metrics and auxiliary measures such as latency and cost. Section 5.3 describes the experimental setup and the evaluated strategies. Sections 5.4–5.9 present quantitative and qualitative results, followed by a summary that relates the findings to the original design requirements.

5.1. Dataset and Preprocessing

The **MUC-4 dataset** was introduced as part of the Message Understanding Conference (MUC) evaluation series and remains one of the most established benchmarks for information extraction in the terrorism domain. It consists of English-language newswire articles in which domain experts annotated terrorist incidents that occurred primarily in Latin America during the late 1980s. Each article may describe one or more events, and each event is represented by a structured template containing up to **24 annotated fields** that capture various attributes of the incident, including participants, timing, and contextual information.

For the purpose of this evaluation, the full schema was reduced to the subset of fields that are both semantically informative and consistently populated. The final evaluation schema includes: `incidentType`, `incidentDate`, `incidentLocation`, `incidentStage`, `perpetratorIndividual`, `perpetratorOrganization`, `target`, `victim`, and `weapon`. These correspond directly to the slots defined in the *Invox* template-filling system and reflect the most important aspects of event understanding. This reduction enables a focused evaluation of the system's ability to extract the key components of each incident rather than peripheral metadata.

Each article in MUC-4 can contain multiple annotated events. To align with the design of the *Invox* pipeline—where each input corresponds to a single structured template—the evaluation considers only one event per document. When multiple event templates are available, the preprocessing procedure automatically selects the most complete annotation, typically the first one with non-empty values across the selected fields. This ensures a one-to-one correspondence between document text and evaluated template, which simplifies both comparison and interpretation.

5.1.1. Preprocessing

The preprocessing pipeline is applied exclusively to the **gold annotations** provided by the MUC-4 dataset, as the system outputs generated by *Invox* already conform to the standardized schema. The original MUC-4 corpus is distributed in two complementary formats: (1) *value files*, which contain the full article texts along with document identifiers, and (2) *key files*, which contain the corresponding event templates with up to 24 annotated fields for each document. To construct a consistent evaluation dataset, these two sources were combined into a unified JSON representation in which each document entry contains both the raw text and the corresponding gold template.

Once the text–template pairs were combined, a sequence of normalization and filtering steps was applied to clean and harmonize the annotations:

- **Schema normalization:** all gold templates were converted into a unified JSON structure using the standardized field names: `incidentType`, `incidentDate`, `incidentLocation`, `incidentStage`, `perpetratorIndividual`, `victim`, `target`, `perpetratorOrganization`, and `weapon`. Fields outside this subset were discarded.
- **Event filtering:** each MUC-4 document can include multiple annotated events. To ensure compatibility with the *Invox* workflow, where each input corresponds to a single structured template, only one event per document was retained. The selection heuristic prioritized the event with the most complete slot coverage (i.e., the greatest number of non-empty fields).

- **Text normalization:** all article texts were lowercased, punctuation standardized, and redundant whitespace removed to facilitate reliable tokenization and embedding consistency during evaluation.
- **Placeholder cleaning:** annotation placeholders such as "–", "N/A", or empty markers were removed from the gold templates to prevent artificial mismatches during similarity computation.
- **Deduplication of fillers:** repeated mentions of the same entity within a field (for example, a perpetrator organization listed multiple times) were collapsed into unique entries to avoid double counting.

After these operations, each document was represented as a normalized JSON object containing one cleaned article text and a single corresponding event template. The processed dataset includes approximately **1,300 examples** derived from the original MUC-4 training and development files, which were used to populate the retrieval index and provide exemplars for the RAG module. A separate subset of **100 held-out examples** was reserved for evaluation and served as the test set for measuring extraction quality.

This preprocessing ensures a harmonized dataset structure that faithfully preserves the original semantic content while enabling consistent embedding-based comparison between gold annotations and the structured templates generated by the *Invox* system.

5.1.2. Schema Caveats for Slot Filling

While MUC-4 is a classic benchmark, its slot schema introduces mismatches that matter in practice. A single article often mentions several incidents, yet the template expects one event, forcing systems to choose which facts to keep and which to drop. Roles can be underspecified or mixed—generic actors like "rebels" appear alongside named groups and people—so the split between `perpetratorIndividual` and `perpetratorOrganization` is frequently ambiguous. Granularity also varies by field: locations oscillate between country–city–neighborhood and just a country; weapons range from specific ("car bomb") to coarse ("explosive") or even dashes; dates are sometimes relative in text ("yesterday") but absolute in gold. These inconsistencies create false errors for otherwise sensible fills.

A second issue is calibration and evidence. Gold templates sometimes omit entities that are clearly in the text, and at other times include fillers that are only implied (e.g., an organization named via allegation). This rewards systems that guess and penalizes ones that abstain without confirmation. Combined with translation/OCR artifacts (casing, accents, bracketed notes), exact-match scoring can overstate mistakes. Our setup mitigates this with normalization, soft/semantic matching, and an explicit

fill-vs-abstain decision, but these schema quirks remain important when interpreting scores and designing post-processing.

5.2. Evaluation Metrics

We evaluate *Invox* along complementary axes: (i) semantic accuracy under paraphrase and set-size mismatch, (ii) behavior under typed schema constraints, and (iii) operational efficiency. Because MUC-4 slots often admit paraphrases or near-synonyms, we use embedding-based measurements for text and list fields, while keeping exact comparison for typed slots (enums, dates). To disentangle empty-field effects, we report results in two regimes: *baseline* (standard scoring, where gold-empty $\wedge$ pred-empty contributes positively) and *gold-nonempty-only* (scores computed *only* when the gold slot is nonempty).

5.2.1. Embedding-Based Accuracy (R2)

For text and list-valued slots, we compare unordered sets of fillers using sentence embeddings (`all-MiniLM-L6-v2`) and cosine similarity. Let $G = \{g_i\}$ and $P = \{p_j\}$ be the gold and predicted sets. We define gold-anchored recall and prediction-anchored precision surrogates:

$$\text{SoftCoverage} = \frac{1}{|G|} \sum_i \max_j \cos\Big(\text{enc}(g_i),\, \text{enc}(p_j)\Big), \tag{5.1}$$

$$\text{SoftSpecificity} = \frac{1}{|P|} \sum_j \max_i \cos\Big(\text{enc}(p_j),\, \text{enc}(g_i)\Big). \tag{5.2}$$

Their symmetric mean gives a threshold-free Chamfer similarity,

$$\text{Chamfer} \;=\; \tfrac{1}{2}\Big(\text{SoftCoverage} + \text{SoftSpecificity}\Big),$$

and we also report a *Soft-F1* (**SF1**) as the harmonic mean of SoftCoverage and SoftSpecificity:

$$\text{SF1} \;=\; \frac{2\,\text{SoftCoverage} \cdot \text{SoftSpecificity}}{\text{SoftCoverage} + \text{SoftSpecificity}}.$$

For **enums** and **dates**, we use *exact* scoring (match = 1, otherwise 0), aligning with the typed nature of these slots. Free-form **locations** are treated as text (singletons) and scored via embeddings.

Two scoring regimes. We compute all scores twice: (i) **baseline** (standard), and (ii) **gold-nonempty-only** (ignores any (doc,slot) where the gold is empty). This separation isolates true content quality from empty-field advantages.

5.2.2. Decision Quality Under Typed Constraints (R1)

Beyond similarity, we evaluate the model's *fill decision* with a presence matrix over four outcomes per slot: gold-empty/pred-empty, gold-empty/pred-nonempty, gold-nonempty/pred-empty, gold-nonempty/pred-nonempty. From this we derive compact decision metrics:

- **FDA** (Fill-Decision Accuracy): fraction of cases where the empty vs. nonempty choice matches gold.
- **HR** (Hallucination Rate): Pr(pred nonempty | gold empty).
- **MR** (Missing Rate): Pr(pred empty | gold nonempty).
- **RFA** (Required-Fill Accuracy): average slot score *conditioned* on gold-nonempty *and* pred-nonempty, capturing content quality when the model commits to a fill.

To summarize empty-field sensitivity we report the **Empty Advantage Index (EAI)**:

$$\mathrm{EAI} \;=\; \mathrm{OBS} - \mathrm{NES},$$

where **OBS** is the overall baseline score and **NES** is the overall score in the gold-nonempty-only regime. Higher EAI indicates stronger reliance on empty↔empty matches.

5.2.3. Consistency (R1-core): Two-Score Summary

Overall formatting. A record passes a slot's lints if type/enums/regex (dates), and list hygiene hold. Overall format quality is the fraction of records with *all* slots passing:

$$\mathrm{FPR}_{\mathrm{overall}} \;=\; \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}\{\forall f \in \mathcal{F} :\; \mathrm{lint_pass}(x_f^{(n)})\}.$$

Overall style. Per (doc,slot) we compute an input-aware style score $s_f^{(n)}$:

$$s_f^{(n)} = \begin{cases} 1, & \text{gold empty} \;\wedge\; \text{pred empty} \\ 0, & \text{gold empty} \;\wedge\; \text{pred nonempty} \\ 0, & \text{gold nonempty} \;\wedge\; \text{pred empty} \\ \cos\Big(\phi(\mathrm{mask}(\mathrm{pred})),\, \phi(\mathrm{mask}(\mathrm{exemplar}_f))\Big), & \text{gold nonempty} \;\wedge\; \text{pred nonempty} \end{cases}$$

where mask maps letters$\to T$, digits$\to D$, keeps punctuation; ϕ is a normalized char-3gram TF vector; locations use the max over canonical forms and list slots treat each item as a single T. We report the macro style score:

$$\mathrm{SC}_{\mathrm{macro}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \left(\frac{1}{N} \sum_{n=1}^{N} s_f^{(n)} \right).$$

5.2.4. Latency and Cost at Inference (R6)

Operational efficiency is measured per document from `timing.duration_ms`, reporting p50 (median), p90, p95, p99, mean, min, and max. When available, we also estimate cost from API token usage (prompts, completions, and embeddings), reported as USD per document. Because retrieval and embedding are lightweight relative to decoding, schema- and example-guided approaches remain compatible with near real-time use.

5.2.5. User Correction and Human-in-the-Loop Reliability (R4)

Although MUC-4 does not capture interactive editing, *Invox* surfaces all auto-filled fields for review with typed widgets (date pickers, enum selectors, multi-value inputs). This design safely combines "empty-when-uncertain" behavior with rapid human confirmation, preserving downstream data quality.

5.2.6. Modularity and Extensibility (R5)

The architecture decouples retrieval, prompting, decoding, and verification into swappable modules. We evaluate alternative extraction strategies under the same metrics (OBS/NES/EAI, SF1/Chamfer, FDA/HR/MR/RFA, latency), enabling targeted optimization. Because slots are declarative (descriptions, types, allowed values), new domains or fields integrate without changing the evaluation stack.

Summary. Our protocol reports (i) **content similarity** (SoftCoverage, SoftSpecificity, Chamfer, SF1), (ii) **decision quality** (FDA, HR, MR, RFA) with an empty-field sensitivity index (EAI), under both **baseline** and **gold-nonempty-only** regimes, and (iii) **operational efficiency** (latency, cost). Together these satisfy requirements on accuracy (R2), consistency (R1), domain alignment (R3), modularity (R5), operational efficiency (R6), and human oversight (R4) in a single, coherent evaluation framework.

5.3. Experimental Setup

This section describes the configuration of the experiments conducted to evaluate the *Invox* system under different orchestration strategies. All evaluations were executed using the preprocessed MUC-4 dataset described in Section 5.1 and the embedding-based metrics defined in Section 5.2. The setup ensures comparability across strategies by maintaining identical model parameters, embedding configurations, and evaluation scripts.

5.3.1. Compared Strategies

The *Invox* system supports multiple extraction strategies that differ in how they allocate reasoning effort across large-language-model (LLM) calls. Each strategy is evaluated independently but within the same pipeline, demonstrating the modular nature of the architecture.

- **S1: Single-Pass Full Input** — the entire document is processed in a single inference step using one LLM instance. This approach minimizes latency and cost but provides limited internal verification.
- **S2: Iterative Single-Field** — each field is extracted sequentially in separate LLM calls. This allows focused reasoning for difficult slots such as `perpetratorIndividual` or `target`, often improving precision at the cost of longer inference time.
- **S3: Multi-LLM Consensus (Full)** — multiple LLMs (e.g., GPT-5 and Gemini 2.5) process the entire input independently. The Verification Agent aggregates their results via consensus scoring to improve overall reliability.
- **S4: Multi-LLM Per-Field** — similar to S3 but the consensus process operates at the field level, combining specialized model outputs per slot. This configuration provides the most granular control over extraction quality.
- **S5: LangExtract Baseline** — a non-LLM rule-based extraction pipeline used as an external baseline. It relies on handcrafted patterns and regular expressions and serves to contextualize the semantic improvements achieved by *Invox*.

All strategies share the same input format and produce outputs following the schema (`incidentType`, `incidentDate`, `incidentLocation`, `incidentStage`, `target`, `victim`, `perpetratorIndividual`, `perpetratorOrganization`, `weapon`). This unified design enables a direct, component-wise comparison of accuracy, latency, and cost.

Note on the Hybrid Strategy. A separate Hybrid Refinement strategy was planned, but not implemented. During development it became clear that all four main strategies (S1–S4) already include user review and correction, which creates the same hybrid behaviour in practice. Adding a fifth strategy would have duplicated functionality without offering a new method.

5.3.2. Embedding Models

Semantic similarity between predicted and reference fillers is computed using the `sentence-transformers` library. The primary encoder is `all-MiniLM-L6-v2`, chosen for its balance between speed and representational quality. To verify metric robustness, a secondary encoder, `multi-qa-MiniLM-L6-cos-v1`, was used for cross-checking. Both models produce 384-dimensional embeddings normalized to unit length, ensuring that cosine similarity values fall within the range [0, 1]. Empirical tests showed that the two encoders produce nearly identical ranking orders across strategies, confirming the stability of the evaluation results.

5.3.3. Environment and Settings

All experiments were run on a CPU-only Ubuntu 24.04.3 LTS machine (kernel 6.14) with an Intel i7-6600U (2 cores, 4 threads), 16 GB RAM, and integrated Intel HD 520 graphics (no discrete NVIDIA GPU / no CUDA). Local preprocessing and evaluation were implemented in Python 3.10; the orchestration service was built with Node.js 20 (TypeScript).

Model inference used hosted APIs (OpenAI, Gemini) with deterministic settings (`temperature=0.2`, `top-p=1.0`) and fixed random seeds for reproducibility. Latency was measured at the API boundary per document (request–response wall time, including network). Token usage and provider-reported pricing were logged to estimate cost. All strategies used identical batching and concurrency to avoid scheduling bias.

5.4. Results – LangExtract Baseline (S5)

LangExtract is an open-source, LLM-powered Python library for schema-constrained extraction. In this work it is used as an external baseline rather than a core component of the proposed system: it provides a deterministic orchestration wrapper around model calls (strict JSON schema enforcement and span grounding), but is not integrated into the multi-agent architecture.

On the MUC-4 subset (N=100 documents), LangExtract achieves an overall balanced score (OBS) of 0.684 and a format-driven accuracy (FDA) of 0.756, with a relatively low hallucination rate (HR) of 0.088 and a high rate of fully correct fields (RFA = 0.796) (see Table B.1 in Appendix B). This pattern indicates a conservative filler: when the system commits to a value it is usually accurate, but it frequently abstains. The non-empty specific metrics reflect this behaviour: the non-empty F_1 score remains modest (0.406), and the non-empty exact accuracy (EAI) of 0.182 shows that much of the performance comes from correctly leaving fields empty when the gold label is empty.

Field-wise analysis (Table B.2) reveals a skew towards easier categorical slots such as `incidentType`, `weapon`, and `incidentStage`, which achieve comparatively high scores, whereas more complex or ambiguous fields such as `target`, `incidentLocation`, and especially `perpetratorOrganization` perform noticeably worse under gold-nonempty evaluation. The `incidentDate` field is particularly under-filled: in cases where the gold label is nonempty, the model often abstains, contributing to the higher miss rate (MR = 0.369).

The embedding-based metrics in Table B.3 and Table B.4 confirm that LangExtract produces stylistically consistent outputs with reasonable semantic coverage: soft coverage and specificity are balanced, and the symmetric Chamfer scores are stable across text and list fields. Consistency metrics (schema format vs. style, see Appendix A.2) show perfect schema adherence ($\text{FPR}_{\text{overall}} = 1.000$) and a style consistency score of $\text{SC}_{\text{macro}} = 0.709$, which is comparable to the best multi-agent strategies.

In terms of runtime, LangExtract exhibits moderate latency, with a median of approximately 21 s per document and a long tail extending to around 110 s (Table B.5). This makes it practical as a batch or asynchronous baseline, but less attractive as an interactive component. Overall, S5 serves as a useful external point of comparison: it demonstrates that a mature off-the-shelf extractor can reach solid accuracy under conservative filling, but it does not outperform the best multi-agent strategies in the non-empty and robustness-oriented metrics that are central to this thesis.

5.5. Results – Strategy S1: Single-Pass Full Input

The **S1: Single-Pass Full Input** configuration extracts all nine MUC-4 slots in a single LLM call. We evaluate three S1 variants: **S1.0** without few-shot examples on the original MUC-4 dataset, **S1.1** with few-shot examples on the original dataset, and **S1.2** with few-shot examples on a modified MUC-4 dataset that mimics speech-style transcripts.

Figure 5.1 demonstrates that few-shot prompting provides clear gains over zero-shot extraction. S1.1 improves slot-level accuracy (OBS, SF1) and reduces hallucination

(HR) compared to S1.0, with the strongest effects on entity-related fields. The speech-style variant, S1.2, performs slightly below S1.1 but still maintains most of the few-shot benefits, indicating robustness under noisier inputs.

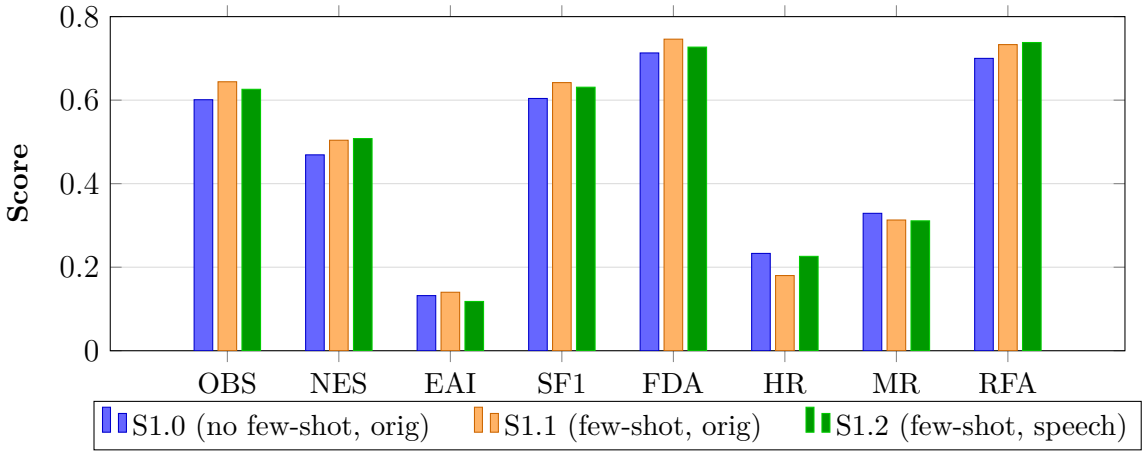

Figure 5.1.: Performance of S1 strategy variants on the MUC-4 subset (N=100). The chart compares zero-shot prompting (S1.0), few-shot prompting on clean text (S1.1), and few-shot prompting on speech-style inputs (S1.2). Few-shot prompting improves key metrics such as OBS, SF1, and FDA while lowering HR and MR, with S1.2 maintaining most benefits despite noisier inputs.

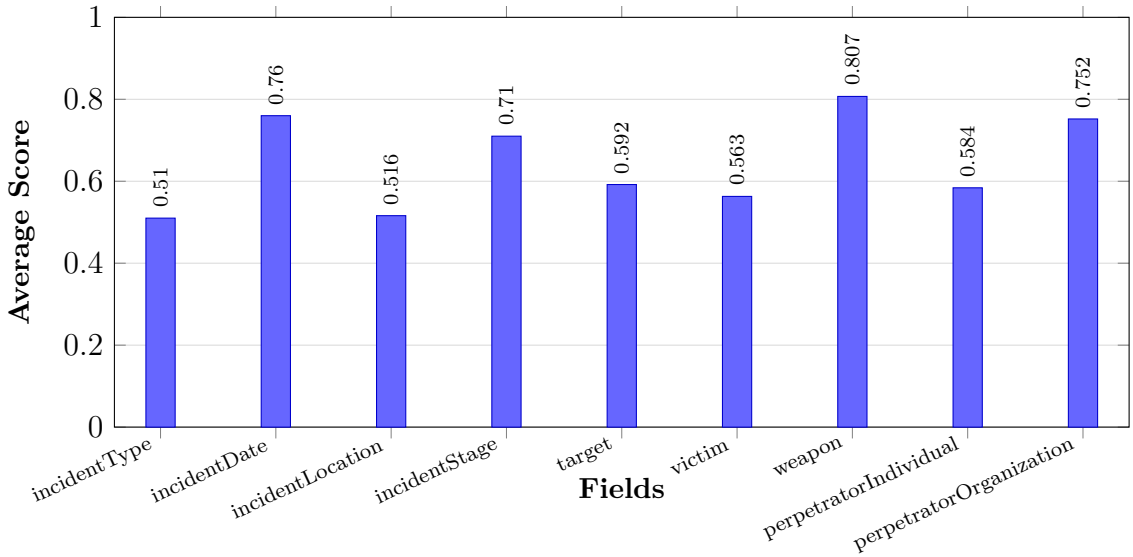

Figure 5.2.: Per-field extraction performance for S1.1 on the MUC-4 subset (N=100). Structured fields such as *incidentDate* and *weapon* show high accuracy, whereas open-ended categories like *incidentType* and *incidentLocation* remain challenging. The pattern illustrates where few-shot prompting is effective and where additional support mechanisms may be required.

Per-field behaviour (S1.1). `perpetratorOrganization` and `weapon` are the strongest fields under S1.1, whereas `perpetratorIndividual` and `incidentLocation` remain more challenging.

Latency

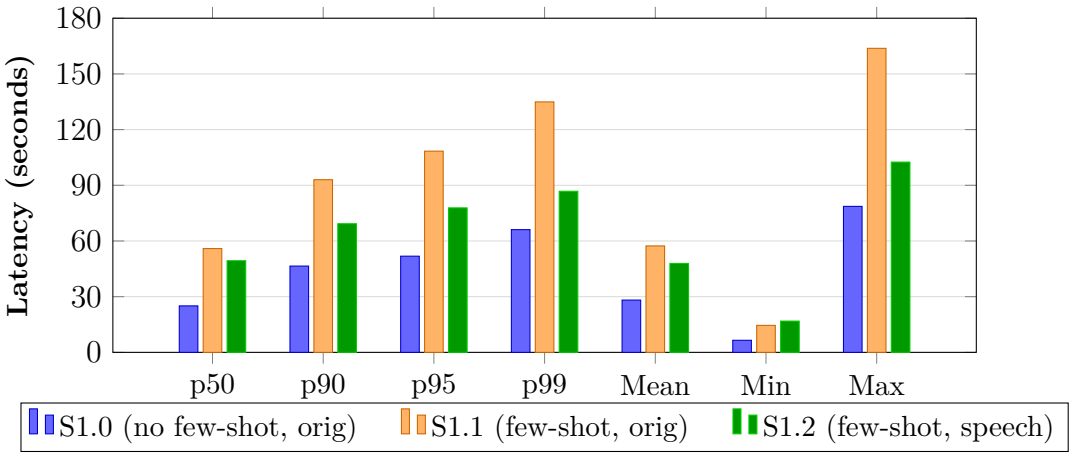

Figure 5.3.: Latency statistics for S1 variants (seconds).

Figure 5.2 confirms that performance is not uniform across slots. Categorical fields such as `incidentStage`, `perpetratorOrganization`, and especially `weapon` achieve relatively high scores, reflecting their clearer lexical patterns and more constrained label spaces. In contrast, `perpetratorIndividual` and `incidentLocation` show lower averages, consistent with their higher ambiguity and sparsity in the underlying texts. Dates are handled comparatively well (average 0.760 for `incidentDate`), but the remaining gap to perfect accuracy indicates residual issues with normalization and partial matches.

Figure 5.3 illustrates the latency–quality trade-off within the S1 family. S1.0 is clearly the fastest variant, with a median of around 25 s per document and a relatively short tail, but it is also the weakest in terms of accuracy. S1.1 roughly doubles the median latency to about 56 s and exhibits a long tail up to approximately 164 s, reflecting the additional computation needed to incorporate few-shot context. S1.2 offers an intermediate profile: its median latency ($\sim$ 49 s) and mean latency are lower than S1.1, and the upper tail is substantially shorter (p99 = 86.8 s vs. 134.9 s), making it more attractive in scenarios where robustness to speech-style inputs is required but extreme latency must be avoided.

Cost Analysis (S1: Single-Pass Full Input)

Assumptions. Extraction uses *GPT-5* with I=3,000 input tokens and O=300 output tokens; verification uses *GPT-5-mini* with V_{in}=1,000 and V_{out}=100. If audio is present, an additional Whisper transcription cost is incurred for D minutes of audio.

Prices. GPT-5: input \$1.25/M, output \$10.00/M. GPT-5-mini: input \$0.25/M, output \$2.00/M. Whisper: \$0.006/min.

Formula.

$$\text{Cost} = \frac{I}{10^6} p_{\text{in}} + \frac{O}{10^6} p_{\text{out}} + \frac{V_{\text{in}}}{10^6} p_{\text{in}}^{(\text{mini})} + \frac{V_{\text{out}}}{10^6} p_{\text{out}}^{(\text{mini})} + 0.006 \cdot D$$

Per-record (no audio).

$$\underbrace{\tfrac{3000}{10^6} \cdot 1.25}_{\$0.00375} + \underbrace{\tfrac{300}{10^6} \cdot 10.00}_{\$0.00300} + \underbrace{\tfrac{1000}{10^6} \cdot 0.25}_{\$0.00025} + \underbrace{\tfrac{100}{10^6} \cdot 2.00}_{\$0.00020} = \$\mathbf{0.00720}\ (\approx 0.72¢/\text{doc})$$

With audio (Whisper). Adding Whisper introduces a simple linear term $0.006 \cdot D$. For D=1 min, the total cost becomes \$0.00720 + 0.006 = \$**0.01320** (approximately 1.32 ¢ per document). At the token scales considered here, S1's LLM component remains inexpensive (around \$0.0072 per document), and audio processing dominates cost for longer recordings.

Consistency (Formatting & Style)

Figure 5.4 shows that few-shot prompting improves both schema-level and style-level consistency. S1.1 achieves the highest schema format preservation ($\text{FPR}_{\text{overall}}$=0.960) and the best style consistency score (SC_{macro}=0.724), confirming that the additional examples help the model adhere more reliably to the expected JSON schema and textual conventions. S1.2 remains close to S1.1 despite the more challenging speech-style input, with only a small drop in both scores, which indicates that the single-pass architecture can maintain formatting and stylistic regularity even under noisier conditions.

Taken together, the three S1 variants illustrate how single-pass extraction behaves under different prompting and input conditions. The addition of few-shot examples consistently strengthens the model: S1.1 delivers the highest overall accuracy on clean text, with notable gains in FDA, SF1, and schema–style consistency, albeit at the cost of higher latency. S1.2 provides a balanced alternative for speech-like transcripts, maintaining competitive accuracy while reducing tail latency and showing the strongest behaviour under nonempty–nonempty evaluation (NES, RFA). In contrast, S1.0 represents a minimal baseline without contextual guidance—fastest but

clearly weakest across core metrics. Overall, S1.1 is the most effective configuration for high-quality offline extraction, whereas S1.2 is preferable in interactive or speech-driven deployments where robustness and latency stability matter.

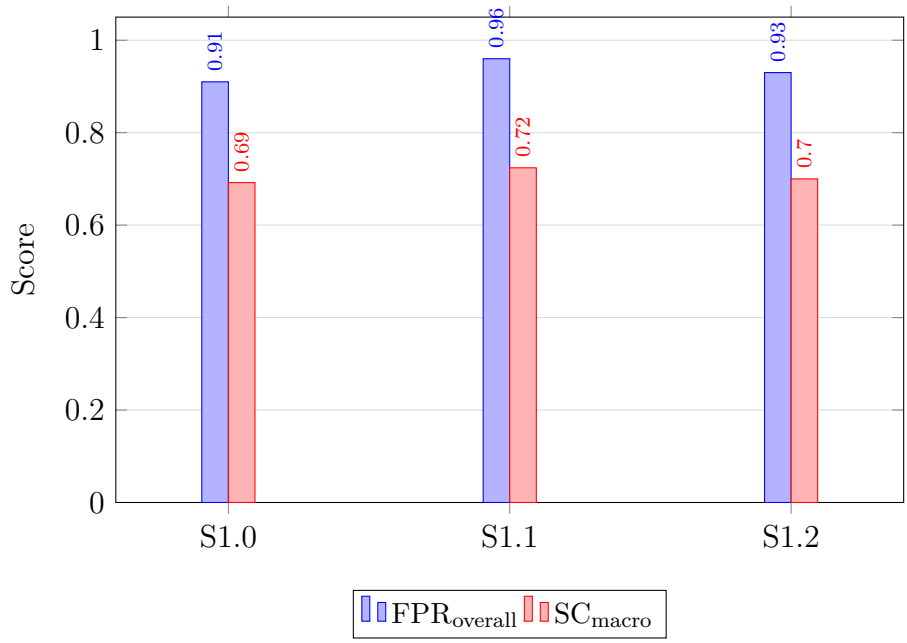

Figure 5.4.: Consistency (S1 variants): schema formatting vs. input-aware style.

5.6. Results – Strategy S2: Iterative Single-Field

The **S2: Iterative Single-Field** strategy fills the template sequentially, invoking the LLM once per slot. We evaluate three S2 variants: **S2.0** without few-shot examples on the original MUC-4 dataset, **S2.1** with few-shot examples on the original dataset, and **S2.2** with few-shot examples on a modified dataset that mimics speech-style transcripts.

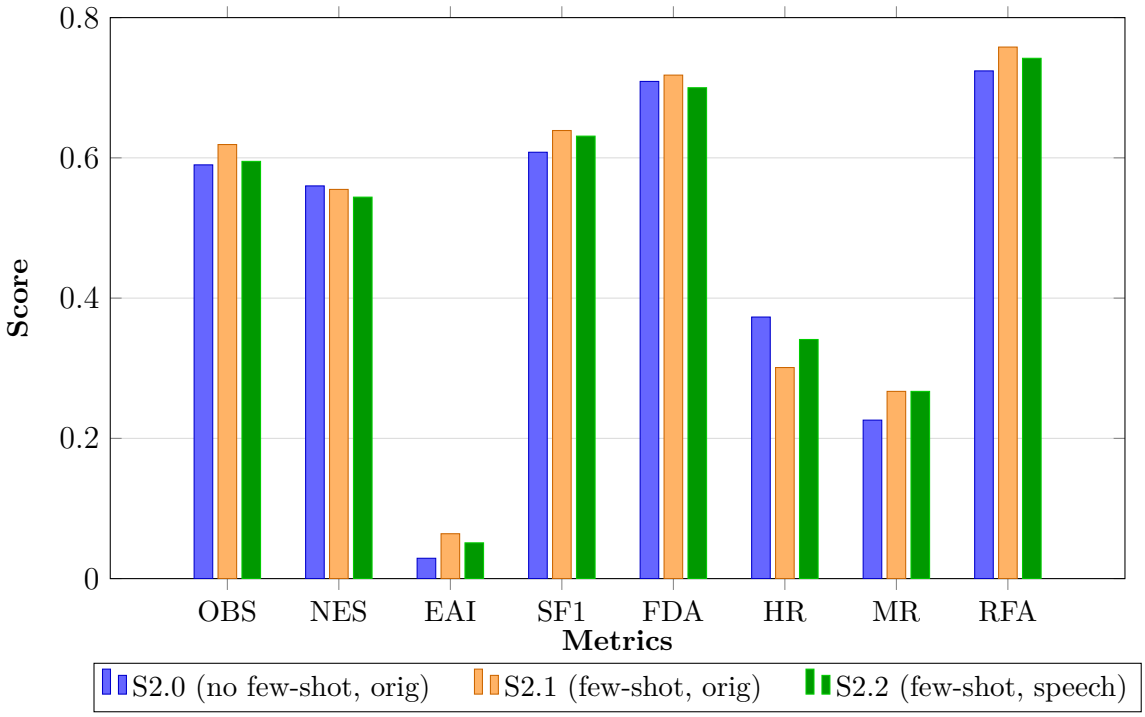

Figure 5.5.: Headline metrics for S2 variants on MUC-4 (N=100). Few-shot prompting (S2.1) improves most accuracy-oriented metrics (OBS, SF1, RFA) and lowers hallucination rates compared to the zero-shot baseline (S2.0). The speech-style variant (S2.2) performs slightly below S2.1 but remains more stable than S2.0, showing how iterative slot-wise extraction benefits from example-based grounding while still being sensitive to ASR-like noise.

Figure 5.5 shows that few-shot prompting (S2.1) strengthens the iterative strategy by improving slot-level accuracy and reducing hallucination compared to the zero-shot baseline. The model becomes slightly more conservative—reflected in a higher missing rate—which suggests that examples help it avoid unsupported guesses. The speech-style variant (S2.2) maintains most of these gains, though with somewhat weaker formatting consistency and higher hallucination, indicating that ASR-like noise mainly affects precision rather than overall extraction capability.

Per-field behaviour (S2.1). `perpetratorOrganization` and `weapon` are the strongest slots under S2.1, `perpetratorIndividual` remains difficult, and `incidentLocation` benefits from per-slot prompting.

Figure 5.6 highlights how iterative, field-specific prompting shifts performance across the nine slots. As in other strategies, `perpetratorOrganization` and `weapon` achieve the highest averages (0.738 and 0.751), reflecting that organization names and weapons are often expressed with relatively distinctive lexical cues that the

model can exploit in a field-focused prompt. The `incidentLocation` slot benefits noticeably from the single-field set-up, reaching 0.597, which is higher than in the single-pass baseline. This supports the hypothesis that dedicated instructions per slot help the model to attend more precisely to location mentions, rather than treating them as one of many competing targets in a joint prompt. By contrast, `incidentDate` remains moderate at 0.550, and `perpetratorIndividual` continues to be one of the hardest fields due to sparse and ambiguous references to individuals in the underlying texts. Iterative prompting improves some aspects of locality and grounding, but does not fully overcome the inherent data limitations for this slot.

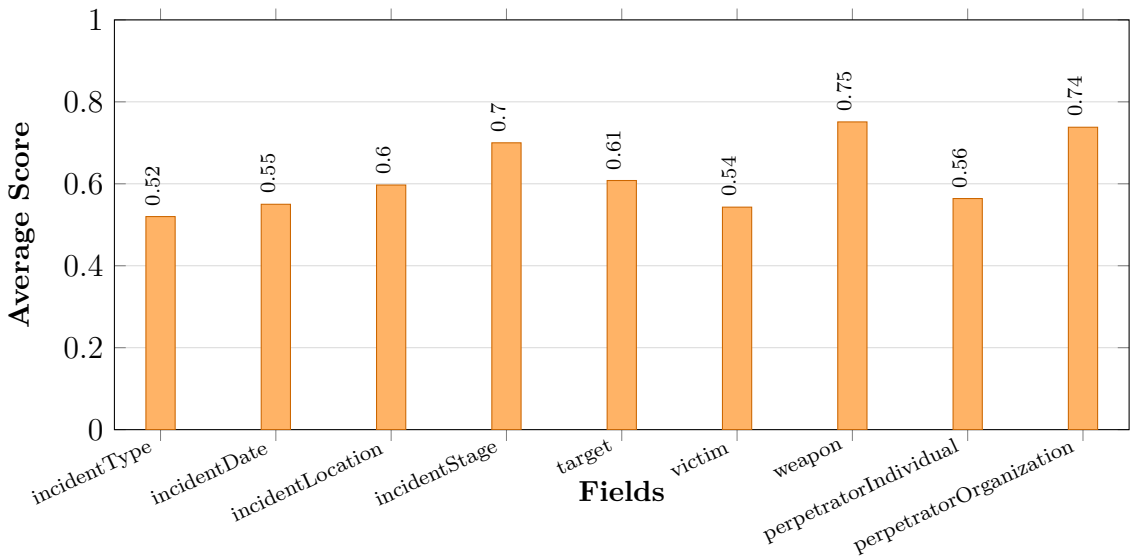

Figure 5.6.: Per-field extraction performance for S2.1 on the MUC-4 subset (N=100). The iterative strategy shows uneven improvements across fields: structured categories such as *weapon* and *incidentStage* benefit most, whereas open-class fields like *incidentType* and *victim* remain comparatively challenging. These results illustrate how slot-wise prompting stabilizes certain fields while exposing residual difficulty in categories requiring interpretation or broader context.

Latency

Figure 5.7 shows that all S2 variants are comparatively fast, even though they issue one call per field. S2.0 is the speed leader, with a median latency of about 20 s per document and a tight upper tail (p99 below 50 s). Adding few-shot examples in S2.1 increases median latency to roughly 25 s and extends the tail (p99 around 72 s), but still keeps the strategy within a range that is substantially faster than the most accurate S1 configuration. S2.2, evaluated on speech-style inputs, sits between S2.0 and S2.1 in terms of latency: its median and mean latencies are

slightly lower than S2.1, and its p99 value (about 53 s) is considerably shorter than S2.1's, indicating more stable runtime behaviour. Overall, S2 provides an attractive accuracy–throughput trade-off: the strategy is noticeably faster than S1.1 and S3.1, while still achieving strong headline metrics.

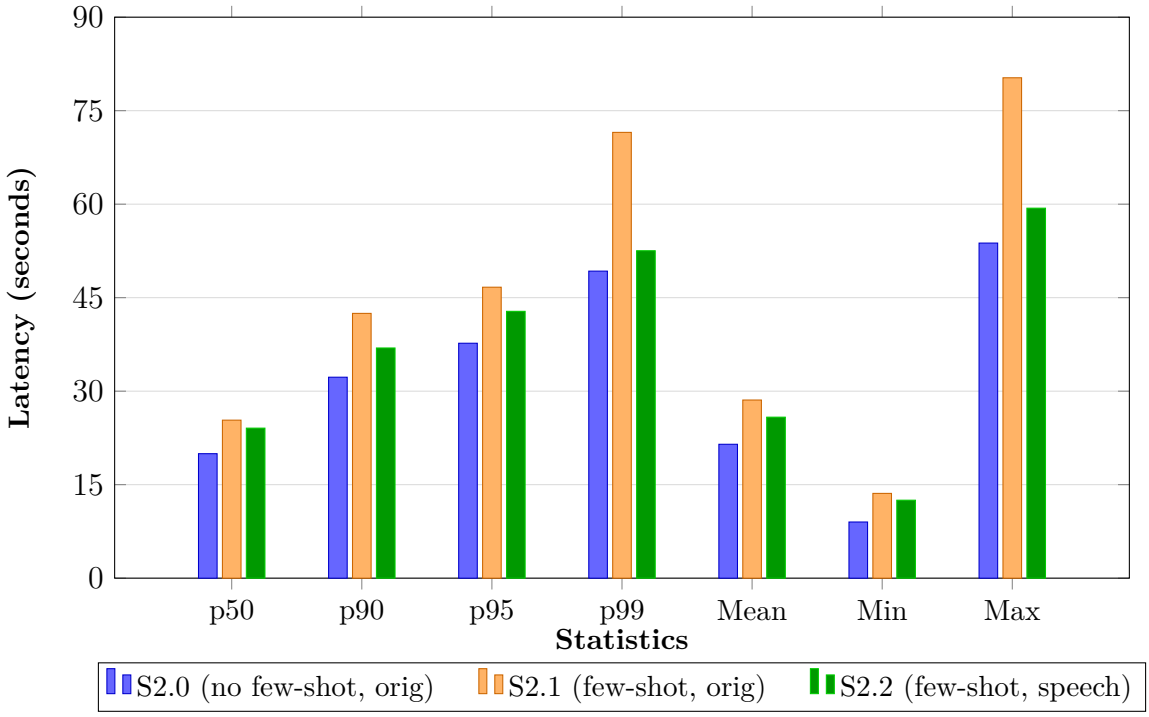

Figure 5.7.: Latency statistics for S2 variants (seconds). The plot compares execution times for S2.0 (no few-shot), S2.1 (few-shot), and S2.2 (few-shot with speech-style input). S2.1 shows the highest latency because few-shot prompts increase token processing, while S2.0 remains the fastest. The speech-style variant (S2.2) reduces extreme latencies relative to S2.1, suggesting that prompt length impacts cost more than input complexity. Overall, the results illustrate the trade-off between improved accuracy from few-shot prompting and the computational overhead of iterative, slot-wise extraction.

Cost Analysis (S2: Single-Field Iterative)

Assumptions. Each of the F=9 MUC-4 slots is extracted with a separate *GPT-5* call. Per field, the average token usage is i_f=1,500 input tokens and o_f=50 output tokens. Verification with *GPT-5-mini* is performed once per record (not per field) with V_{in}=1,000 and V_{out}=100. If audio is used, Whisper transcription for D minutes is added once per record.

Prices. GPT-5: input \$1.25/M, output \$10.00/M. GPT-5-mini: input \$0.25/M, output \$2.00/M. Whisper: \$0.006/min.

Formula.

$$\mathrm{Cost}_{\mathrm{S2}} = \underbrace{\sum_{f=1}^{F}\left(\frac{i_f}{10^6}p_{\mathrm{in}} + \frac{o_f}{10^6}p_{\mathrm{out}}\right)}_{\text{per-field extraction on GPT-5}} + \underbrace{\frac{V_{\mathrm{in}}}{10^6}p_{\mathrm{in}}^{(\mathrm{mini})} + \frac{V_{\mathrm{out}}}{10^6}p_{\mathrm{out}}^{(\mathrm{mini})}}_{\text{single verification on GPT-5-mini}} + 0.006 \cdot D$$

Per-record (no audio).

$$9 \times \left(\underbrace{\tfrac{1500}{10^6}\cdot 1.25}_{\$0.001875} + \underbrace{\tfrac{50}{10^6}\cdot 10.00}_{\$0.000500}\right) + \underbrace{\tfrac{1000}{10^6}\cdot 0.25}_{\$0.000250} + \underbrace{\tfrac{100}{10^6}\cdot 2.00}_{\$0.000200} = \$\mathbf{0.02183}\ (\approx 2.18¢/\mathrm{doc})$$

With audio (Whisper). Adding Whisper introduces a linear term of $0.006 \cdot D$. For D=1 min, the total cost becomes \$0.02183 + 0.006 = \$**0.02783** (approximately 2.78 ¢ per document). Under the assumed token budgets, S2 therefore costs about 3.0× as much as S1 (\$0.02183 vs. \$0.00720 per document, no audio), driven by the F separate extraction calls; running verification only once per record keeps the arbiter overhead comparatively small.

Consistency (Formatting & Style)

Figure 5.8 indicates that the iterative strategy maintains very high schema conformity and stable style. All variants achieve $\mathrm{FPR}_{\mathrm{overall}} \geq 0.940$, with S2.0 reaching 0.970, which reflects that the per-field design and explicit JSON schemas make it easy for the model to respect structural constraints. The style-aware score $\mathrm{SC}_{\mathrm{macro}}$ modestly improves from 0.672 in S2.0 to 0.683 in S2.1, suggesting that few-shot prompts help the model settle on more consistent textual realizations across documents. S2.2, evaluated on speech-style input, shows a small drop in both metrics but remains close to S2.1, which indicates that formatting and style are largely preserved even when the input becomes noisier. Taken together with the headline metrics, this suggests that S2 trades a slight reduction in schema strictness (relative to S2.0) for better content accuracy and calibration in S2.1, and that these gains mostly carry over to spoken-like transcripts in S2.2.

Overall, the S2 family occupies a middle ground between single-pass and full consensus strategies. Iterative, field-specific prompting improves challenging slots such as `incidentLocation`, while still struggling with intrinsically difficult fields like `perpetratorIndividual`. Few-shot prompting (S2.1) clearly improves accuracy and reduces hallucinations compared to S2.0, at the cost of a moderate latency and cost increase. The speech-style variant S2.2 preserves much of this behaviour on noisier inputs, with strong RFA and only modest degradation in calibration metrics.

Compared to S1.1, S2.1 offers similar or slightly lower headline scores on clean text but achieves substantially better throughput; compared to S3.1 and S4.1, it provides a simpler and more economical alternative that still benefits from per-field control, making it a compelling choice for deployments where latency and cost are constrained but field-level interpretability is desirable.

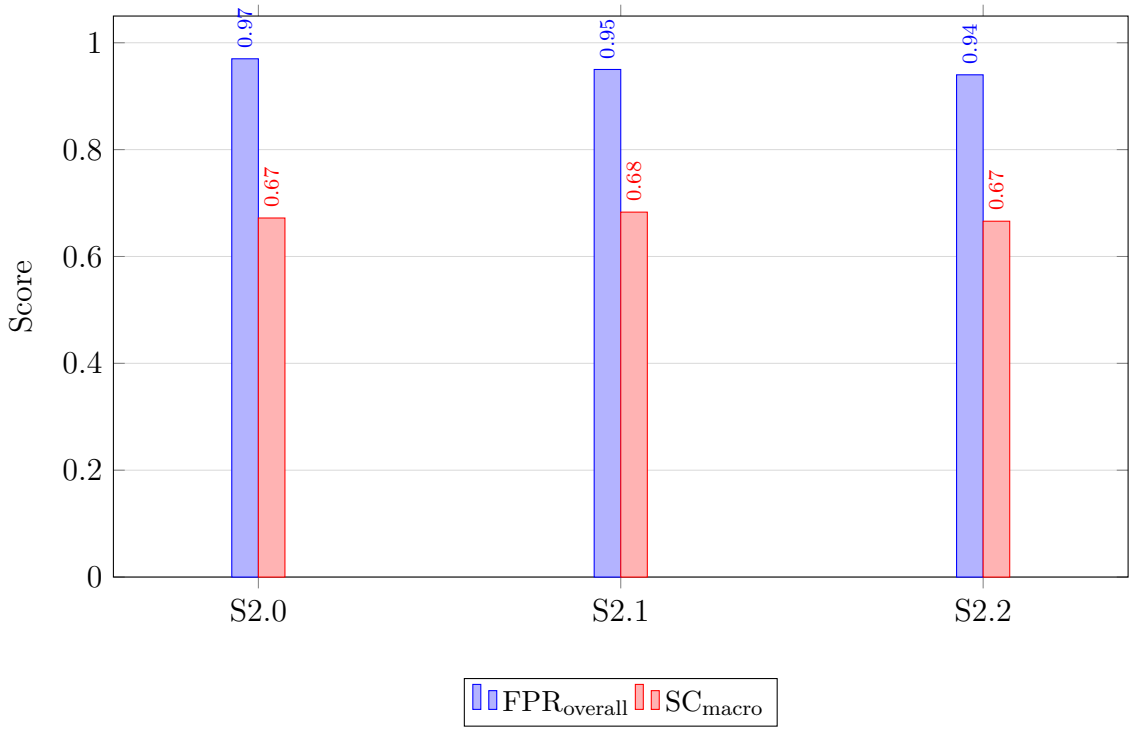

Figure 5.8.: Consistency (S2 variants): schema formatting vs. input-aware style.

5.7. Results – Strategy S3: Multi-LLM Consensus (Full)

The **S3: Multi-LLM Consensus (Full)** strategy runs multiple LLMs on the full document and aggregates their outputs with a verification/consensus step. We evaluate three variants: **S3.0** without few-shot examples on the original MUC-4 dataset, **S3.1** with few-shot examples on the original dataset, and **S3.2** with few-shot examples on a speech-style variant of MUC-4.

Figure 5.9 shows that adding both consensus and few-shot prompting (S3.1) strengthens the multi-model strategy by improving overall accuracy and reducing hallucinations compared to S3.0. The gains are concentrated in metrics that reflect correct and confident fills, indicating that cross-model agreement helps filter out unreliable predictions. The speech-style variant (S3.2) performs close to S3.1, suggesting that

consensus mitigates much of the noise introduced by ASR-like phrasing. Compared to the single-pass baseline, S3.1 keeps overall accuracy similar but produces higher-quality nonempty predictions, reflecting the value of aggregating multiple model outputs.

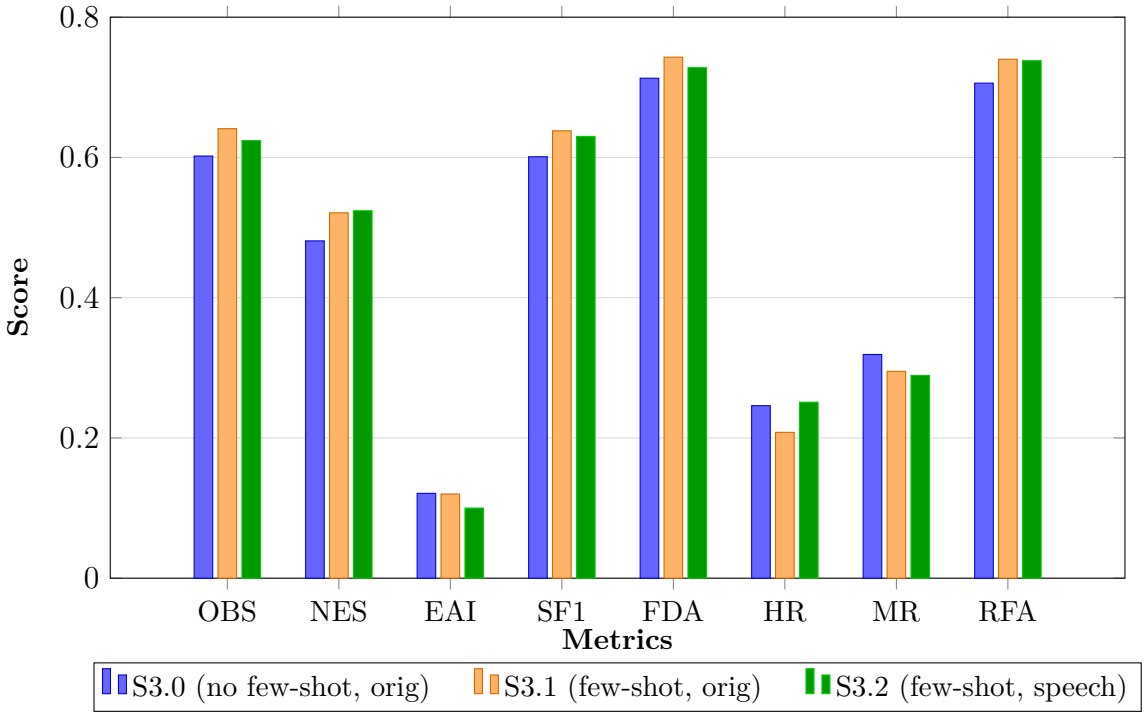

Figure 5.9.: Headline metrics for S3 variants on MUC-4 (N=100). The plot compares the multi-model consensus strategy across S3.0 (no few-shot), S3.1 (few-shot), and S3.2 (few-shot with speech-style input). Few-shot prompting (S3.1) offers consistent gains in overall accuracy (OBS), schema adherence (FDA), and fully correct fields (RFA). The speech-style variant (S3.2) remains close to S3.1, showing that consensus aggregation reduces sensitivity to noise introduced by ASR-like phrasing.

Per-field behaviour (S3.1). Consensus is particularly strong for `perpetratorOrganization` and `weapon`; `perpetratorIndividual` remains hardest; `incidentLocation` improves compared to single-pass strategies.

Figure 5.10 illustrates how full-document consensus redistributes performance across individual slots. As in S1 and S2, `perpetratorOrganization` and `weapon` remain the easiest fields, with average scores of 0.756 and 0.793 respectively, reflecting that these categories are well supported by clear lexical cues and benefit from both models agreeing on salient mentions. `incidentStage` and `incidentDate` also perform strongly (0.730 for both), indicating that structured, temporally anchored information is well captured when the entire document is considered jointly. `incidentLocation`

reaches 0.554, which is higher than in the single-pass S1.1 setting and suggests that cross-model checking over the full context helps eliminate some location-related errors. In contrast, `perpetratorIndividual` remains comparatively weak (0.561), confirming that even an ensemble has difficulty with sparse and ambiguous references to individual actors. Overall, the field-level pattern supports the intuition that S3's gains are concentrated in slots where multiple mentions and global cues are present, while inherently under-specified fields remain challenging.

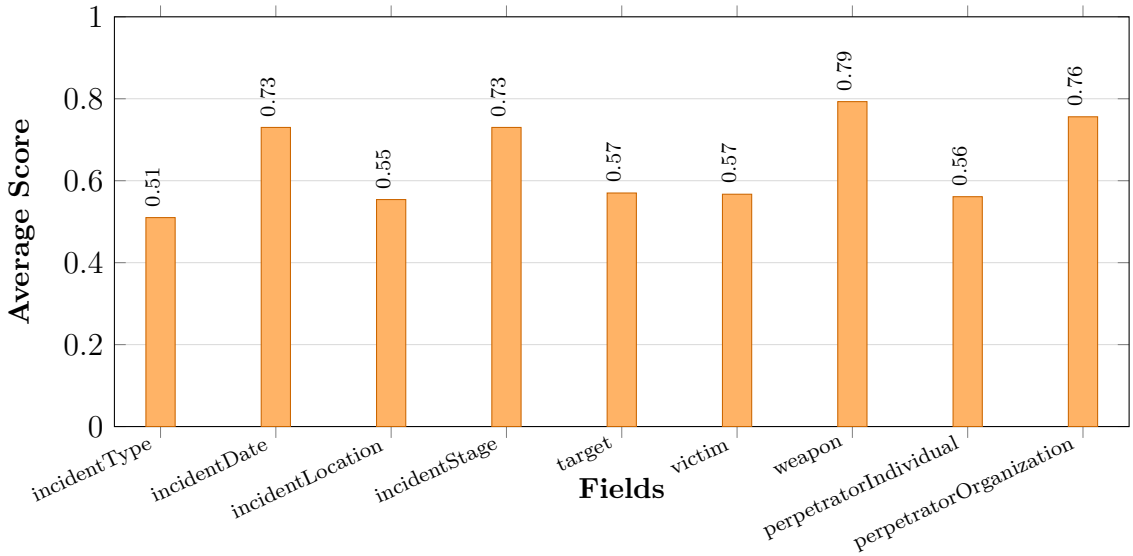

Figure 5.10.: Per-field extraction performance for S3.1 on the MUC-4 subset (N=100). The consensus strategy improves stability across fields, with strong performance on structured attributes such as *incidentDate*, *incidentStage*, and *weapon*. More open-ended fields like *incidentType* and *incidentLocation* remain comparatively difficult, reflecting the inherent ambiguity of these categories. Overall, S3.1 delivers a balanced improvement across most fields compared to non-consensus baselines.

Latency

Figure 5.11 highlights the computational cost of running multiple full-input models. S3.0 has a median latency of about 31 s per document and a moderate upper tail (p99 around 87 s). Incorporating few-shot examples in S3.1 increases the median latency to roughly 59 s and the mean to about 61 s, with a long tail that reaches 165 s, reflecting the overhead of larger prompts and dual model calls. S3.2, which operates on speech-style transcripts, is slightly faster than S3.1: its median and mean latencies drop to about 53 s and 55 s respectively, and the tail is somewhat shorter (p99 around 105 s). Compared to the single-pass few-shot strategy S1.1, S3.1 is slower in the median but has a somewhat shorter extreme tail (p99 118.3 s vs.

134.9 s), suggesting that the cost of consensus is predictable but not catastrophic. In practice, S3 therefore occupies a middle ground in the accuracy–latency space: more expensive than S1 and S2, but still feasible for batched or asynchronous processing.

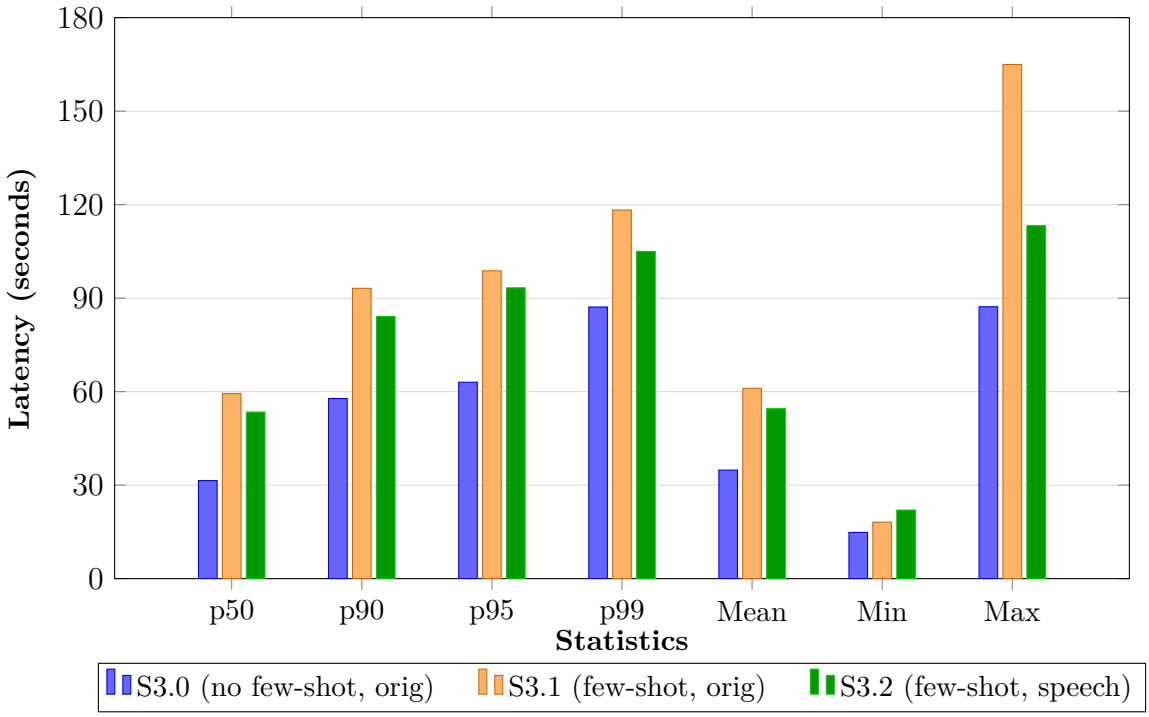

Figure 5.11.: Latency statistics for S3 variants (seconds). The multi-model consensus strategy exhibits substantially higher latency than single-model approaches, with S3.1 (few-shot) producing the largest delays due to multiple model calls combined with longer prompts. The speech-style configuration (S3.2) reduces extreme latencies compared to S3.1 but remains significantly slower than the no–few-shot baseline (S3.0). These results highlight the computational cost of consensus-based extraction, where improved robustness and accuracy come at the expense of markedly increased processing time.

Cost Analysis (S3: Multi-LLM Consensus, Full Input)

Assumptions. Two parallel extractions are run on the full input: one with *GPT-5* and one with *Gemini 2.5 Pro*, each using I=3,000 input tokens and O=300 output tokens. A single arbiter/verification step runs on *GPT-5-mini* with V_{in}=1,000 and V_{out}=100. If audio is used, Whisper transcription for D minutes is added once per record.

Prices. GPT-5: input \$1.25/M, output \$10.00/M; GPT-5-mini: input \$0.25/M, output \$2.00/M; Gemini 2.5 Pro (prompts ≤200k): input \$1.25/M, output \$10.00/M;

Whisper: \$0.006/min.

Formula.

$$\mathrm{Cost}_{\mathrm{S3}} = \left(\frac{I}{10^6}p_{\mathrm{in}}^{(5)} + \frac{O}{10^6}p_{\mathrm{out}}^{(5)}\right) + \left(\frac{I}{10^6}p_{\mathrm{in}}^{(\mathrm{Gemini})} + \frac{O}{10^6}p_{\mathrm{out}}^{(\mathrm{Gemini})}\right) + \left(\frac{V_{\mathrm{in}}}{10^6}p_{\mathrm{in}}^{(\mathrm{mini})} + \frac{V_{\mathrm{out}}}{10^6}p_{\mathrm{out}}^{(\mathrm{mini})}\right) + 0.006 \cdot D$$

Per-record (no audio).

$$\begin{aligned}
\text{GPT-5 extract:}\ & \tfrac{3000}{10^6}\cdot 1.25 + \tfrac{300}{10^6}\cdot 10.00 = \$\mathbf{0.00675} \\
\text{Gemini 2.5 Pro extract:}\ & \tfrac{3000}{10^6}\cdot 1.25 + \tfrac{300}{10^6}\cdot 10.00 = \$\mathbf{0.00675} \\
\text{mini arbiter:}\ & \tfrac{1000}{10^6}\cdot 0.25 + \tfrac{100}{10^6}\cdot 2.00 = \$\mathbf{0.00045} \\
\textbf{Total:}\ & \$\mathbf{0.01395}\ (\approx 1.40¢/\text{doc})
\end{aligned}$$

With audio (Whisper). Adding Whisper introduces a linear term $0.006 \cdot D$. For D=1 min, the total cost becomes \$0.01395 + 0.006 = \$**0.01995** (approximately 2.00 ¢ per document). Under these settings, S3 costs roughly 1.9× as much as S1 (\$0.01395 vs. \$0.00720 per document, no audio), reflecting the dual full-input passes while keeping the arbiter overhead small compared to the extraction steps.

Consistency (Formatting & Style)

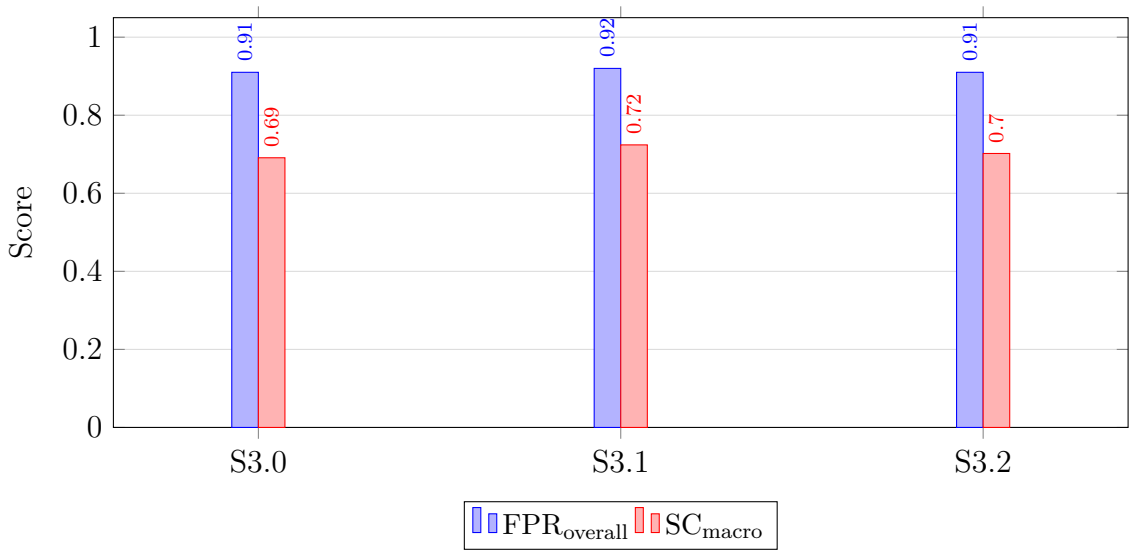

Figure 5.12.: Consistency (S3 variants): schema formatting vs. input-aware style.

Figure 5.12 indicates that multi-LLM consensus maintains high structural and stylistic consistency. All S3 variants achieve $\mathrm{FPR}_{\mathrm{overall}}$=0.910 or higher, with S3.1 slightly ahead at 0.920, which shows that most outputs adhere to the expected JSON

schema even after aggregation. The style-aware score SC_{macro} improves from 0.691 in S3.0 to 0.724 in S3.1, suggesting that the combination of few-shot prompting and consensus helps the system converge on more uniform phrasing and formatting across documents. S3.2, evaluated on speech-style input, remains close to S3.1 in both metrics, which indicates that the ensemble design is robust to noisier transcripts: consensus can smooth out some of the variability introduced by speech-like text while preserving schema fidelity.

Taken together, the S3 family demonstrates how full-document multi-LLM consensus trades additional cost and latency for improved calibration and robustness. Relative to the iterative S2.1 configuration, S3.1 reduces hallucinations (HR 0.301 $\rightarrow$ 0.208) and raises FDA (from 0.718 to 0.743) while also achieving higher OBS, suggesting that cross-model agreement provides more trustworthy fills at the expense of a small increase in misses. Compared to the best single-pass configuration S1.1, S3.1 reaches very similar headline scores but offers better performance on gold-nonempty slots (higher NES) and slightly lower MR, indicating stronger content quality when fields are present. The speech-style variant S3.2 maintains this pattern, trading a small decrease in overall accuracy for higher NES and stable RFA, which makes it attractive for spoken-text deployments where correctness on filled slots is more important than exploiting empty–empty alignment. Overall, S3 is best suited for settings where accuracy, calibration, and robustness justify a roughly twofold increase in LLM cost over S1 and a moderate latency premium.

5.8. Results – Strategy S4: Multi-LLM Per-Field

The **S4: Multi-LLM Per-Field** strategy applies consensus at the slot level: each field is extracted by multiple models and consolidated via a field-aware verifier. We evaluate three variants: **S4.0** without few-shot prompting on the original MUC-4 dataset, **S4.1** with few-shot prompting on the original dataset, and **S4.2** with few-shot prompting on a speech-style variant.

Figure 5.13 shows that per-field consensus behaves differently from the other strategies: it produces strong nonempty-field performance (high NES) but tends to overfill slots, as reflected in the negative EAI values and elevated hallucination rates. This indicates that the model ensemble is biased toward producing a value rather than abstaining. Few-shot prompting moderates this behaviour slightly by reducing hallucinations and improving overall accuracy, though the conservative–vs.–aggressive fill trade-off persists. The speech-style variant remains close to the clean-text version, showing that slot-wise consensus dampens some noise effects but does not fully counteract the tendency to oversupply values.

Per-field consensus boosts `incidentLocation`; `perpetratorIndividual` remains hardest; `weapon` is lower than in S1 and S3.

Figure 5.14 illustrates the asymmetric effects of per-field voting on different slots. `incidentLocation` benefits most, reaching an average score of 0.641 in S4.1 and surpassing the corresponding values in the single-pass and full-document consensus strategies, which suggests that slot-specific prompts plus ensemble agreement help to stabilise location predictions. `incidentStage` also performs strongly (0.720), consistent with its comparatively structured label space. In contrast, `incidentType` and especially `incidentDate` underperform relative to other strategies (down to 0.370 for dates), which indicates that aggressive filling at the field level can lead to more spurious or partially incorrect outputs when the signal is weak. `weapon` achieves 0.692, clearly lower than in S1 and S3, suggesting that per-field voting may over-accept marginal mentions and noise in this slot. `perpetratorIndividual` remains one of the hardest fields despite the richer consensus mechanism, reflecting the persistent difficulty of sparse, ambiguous references to individual actors.

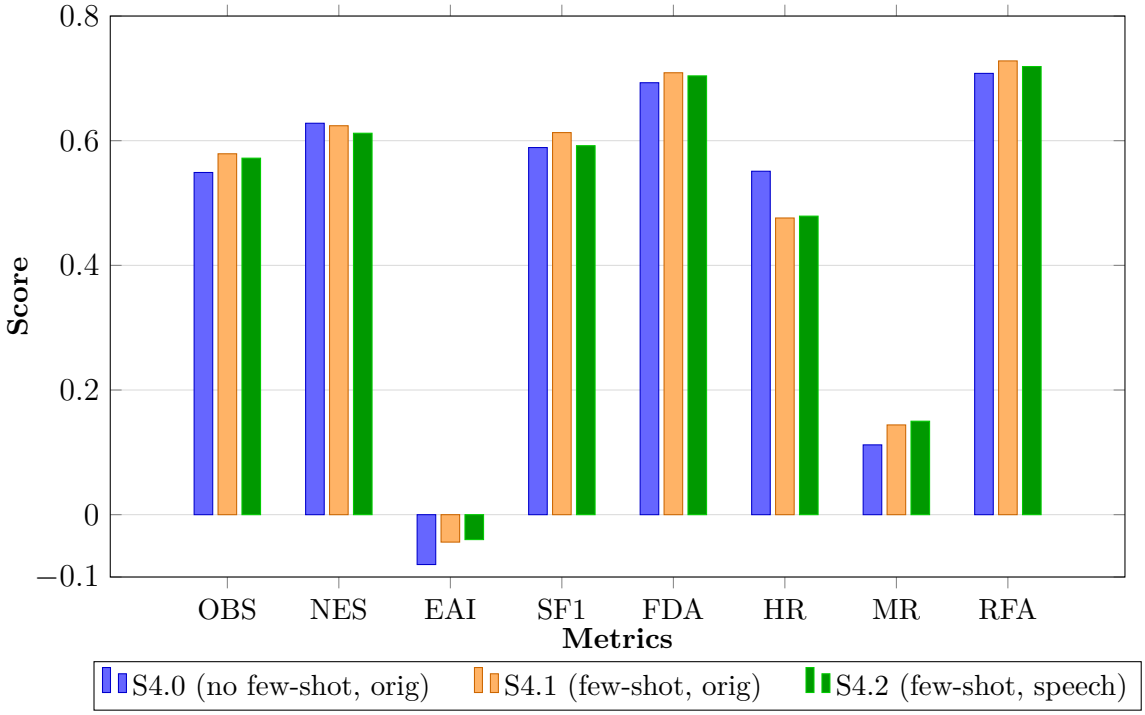

Figure 5.13.: Headline metrics for S4 variants on MUC-4 (N=100). Few-shot prompting (S4.1) improves most accuracy-oriented metrics (OBS, SF1, FDA, RFA) but still produces negative EAI values, indicating disagreement across models at the slot level. The speech-style variant (S4.2) performs similarly to S4.1, suggesting that per-field consensus mitigates some ASR-related noise. Overall, S4 offers strong schema adherence but also highlights how per-field aggregation can amplify model inconsistencies.

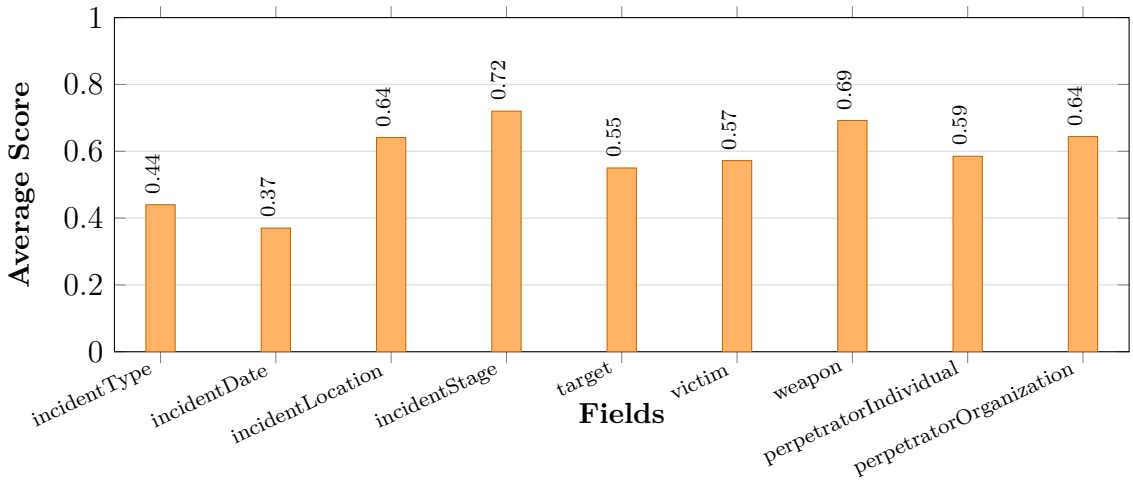

Figure 5.14.: Per-field extraction performance for S4.1 on the MUC-4 subset (N=100). The per-field multi-LLM strategy performs well on structured fields such as *incidentStage*, *weapon*, and *incidentLocation*, but struggles on more context-dependent fields like *incidentType* and *incidentDate*. This pattern shows how per-field consensus can stabilize well-defined slots while amplifying uncertainty in categories that require broader reasoning.

Latency

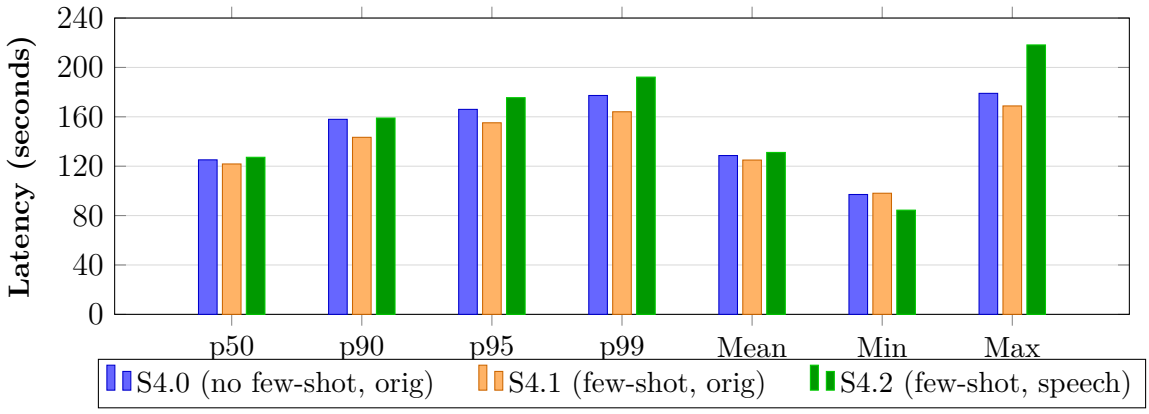

Figure 5.15.: Latency statistics for S4 variants (seconds). The per-field multi-LLM strategy shows the highest computational cost, with median latencies near 2 minutes and p99 values above 3 minutes. Few-shot prompting (S4.1) slightly lowers extreme latencies relative to S4.0, while the speech-style variant (S4.2) adds more variance. Overall, S4 highlights the trade-off between the robustness of per-field consensus and the considerable runtime required for multiple LLM calls.

Figure 5.15 makes clear that S4 is the slowest strategy family. All three variants have median latencies in the 122–127 s range per document and heavy upper tails, with p99 values between roughly 164 s and 192 s. This behaviour is expected because S4 runs multiple models for each individual slot rather than once per document. Few-shot prompting slightly reduces median and mean latency when moving from S4.0 to S4.1, but the differences are small compared to the overall cost of per-field ensemble processing. S4.2 is marginally slower than S4.1 on average and exhibits the heaviest tail (Max above 218 s), reflecting additional variability in convergence on speech-style inputs. In comparison to S3.1, which also uses multi-LLM consensus but at the document level, S4 sacrifices throughput for more aggressive slot-level filling; this makes it less attractive for high-volume or time-sensitive deployments unless very high recall on nonempty fields is the primary objective.

Cost Analysis (S4: Multi-LLM Per-Field Consensus)

Assumptions. For each of the F=9 fields, two parallel extractions are run—*GPT-5* and *Gemini 2.5 Pro*—with per-field averages i_f=1,500 input tokens and o_f=50 output tokens. A single arbiter/verification step per record runs on *GPT-5-mini* with V_{in}=1,000 and V_{out}=100. If audio is used, Whisper transcription for D minutes is added once per record.

Prices. GPT-5: input \$1.25/M, output \$10.00/M. Gemini 2.5 Pro: input \$1.25/M, output \$10.00/M. GPT-5-mini: input \$0.25/M, output \$2.00/M. Whisper: \$0.006/min.

Formula.

$$\text{Cost}_{\text{S4}} = \sum_{f=1}^{F} \Bigg[\underbrace{\tfrac{i_f}{10^6} p_{\text{in}}^{(5)} + \tfrac{o_f}{10^6} p_{\text{out}}^{(5)}}_{\text{GPT-5 per field}} + \underbrace{\tfrac{i_f}{10^6} p_{\text{in}}^{(\text{Gemini})} + \tfrac{o_f}{10^6} p_{\text{out}}^{(\text{Gemini})}}_{\text{Gemini per field}} \Bigg] + \underbrace{\tfrac{V_{\text{in}}}{10^6} p_{\text{in}}^{(\text{mini})} + \tfrac{V_{\text{out}}}{10^6} p_{\text{out}}^{(\text{mini})}}_{\text{single arbiter}} + 0.006 \cdot D$$

Per-record (no audio).

$$\begin{aligned}
\text{Per field (GPT-5):}\ & \tfrac{1500}{10^6} \cdot 1.25 + \tfrac{50}{10^6} \cdot 10.00 = \$0.002375 \\
\text{Per field (Gemini):}\ & \tfrac{1500}{10^6} \cdot 1.25 + \tfrac{50}{10^6} \cdot 10.00 = \$0.002375 \\
\text{Both models per field:}\ & \$0.00475 \\
\text{Across } F\text{=9 fields:}\ & 9 \times 0.00475 = \$0.04275 \\
\text{Arbiter (mini, once):}\ & \tfrac{1000}{10^6} \cdot 0.25 + \tfrac{100}{10^6} \cdot 2.00 = \$0.00045 \\
\textbf{Total:}\ & \textbf{\$0.04320}\ (\approx 4.32\text{¢/doc})
\end{aligned}$$

With audio (Whisper). Adding Whisper introduces a linear term $0.006 \cdot D$. For D=1 min, the total cost becomes \$0.04320 + 0.006 = **\$0.04920** (approximately 4.92 ¢ per document). At these settings, S4 costs about 6× as much as S1 (\$0.04320 vs. \$0.00720 per document, no audio) and roughly 2× as much as S2 (\$0.04320 vs. \$0.02183), with the main driver being the per-field dual-model passes; the single-record arbiter remains a relatively minor component of the overall cost.

Consistency (Formatting & Style)

Figure 5.16 shows that S4 maintains reasonably high structural and stylistic consistency despite its aggressive fill behaviour. All variants achieve $FPR_{overall}$ above 0.87, and few-shot prompting again helps: S4.1 and S4.2 reach 0.94, closing most of the gap to the best-performing strategies in this regard. Style-aware consistency SC_{macro} increases from 0.6599 in S4.0 to 0.6881 in S4.1, with a small drop to 0.6815 for S4.2 on speech-style input. This suggests that the combination of per-slot prompting and consensus does not destabilise output formatting, even though the model ensemble is more willing to produce content in marginal cases.

Taken together, the S4 results characterise multi-LLM per-field consensus as an aggressive, recall-oriented strategy. It achieves high NES and strong performance on genuinely nonempty slots, particularly for context-sensitive fields such as `incidentLocation`, but this comes at the cost of elevated hallucination rates, negative EAI, and substantially higher latency and monetary cost. Compared to full-document consensus (S3.1), S4.1 offers higher NES (0.624 vs. 0.521) but lower OBS (0.579 vs. 0.641) because it overfills when gold is empty. In scenarios where maximising content recall on nonempty slots is critical and throughput is less of a concern, S4.1 is a compelling option—especially if combined with downstream post-filters. For more balanced deployments that must trade off accuracy, calibration, latency, and cost, S3.1 provides a more favourable operating point than S4's per-field ensemble.“‘

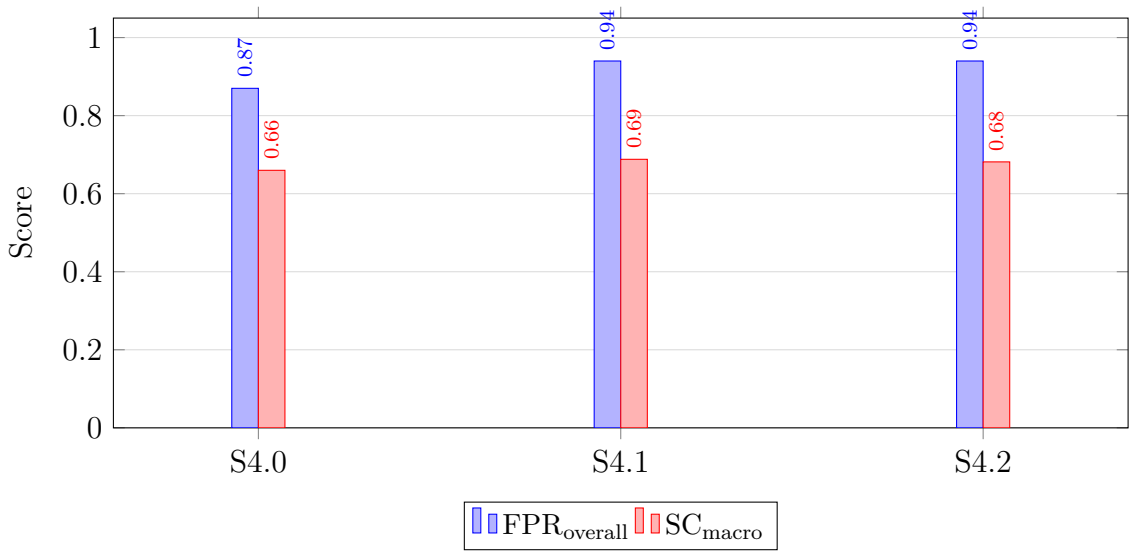

Figure 5.16.: Consistency (S4 variants): schema formatting vs. input-aware style.

5.9. Comparative Analysis of Strategies

This section compares the four Invox strategies (S1, S2, S3, S4) on the same dataset to identify their relative strengths and trade-offs. All results use few-shot prompting on the original MUC-4 test set (N=100).

Accuracy Comparison

Figure 5.17 shows the main performance metrics. S1.1 reaches the highest OBS (0.644) and SF1 (0.642), with low hallucination (HR = 0.180). S4.1 has the highest NES (0.624), meaning it performs best when gold values exist, but its HR is much higher (0.476), indicating it often fills empty fields incorrectly. S2.1 and S3.1 fall between these extremes. The Empty Advantage Index (EAI) shows how much each strategy benefits from correctly leaving fields empty: S1.1 has EAI = 0.140, while S4.1 has negative EAI (-0.044), confirming it relies on filling rather than abstaining.

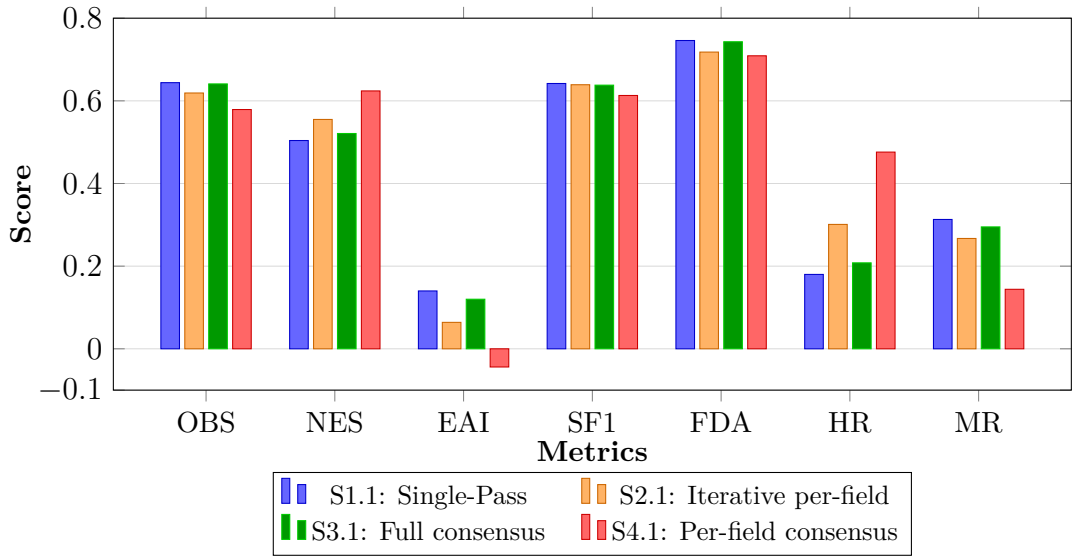

Figure 5.17.: Performance comparison across four strategies (few-shot, orig MUC-4).

Cost Comparison

Figure 5.18 shows how cost increases with audio duration. All strategies add $0.006 per minute for Whisper transcription. The base LLM cost varies: S1.1 costs 0.72¢, S3.1 costs 1.40¢, S2.1 costs 2.18¢, and S4.1 costs 4.32¢per document (without audio). At 1 minute of audio, S4.1 is 3.7× more expensive than S1.1.

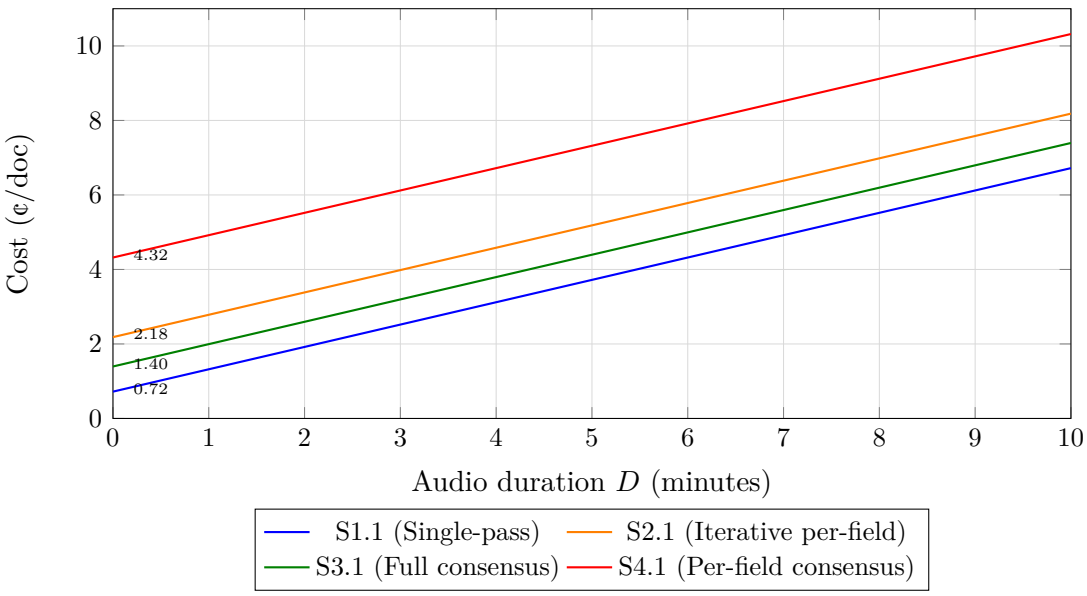

Figure 5.18.: Cost vs. audio duration: $\text{cost}(D) = \text{base} + 0.60D$ (¢/doc).

Latency Comparison

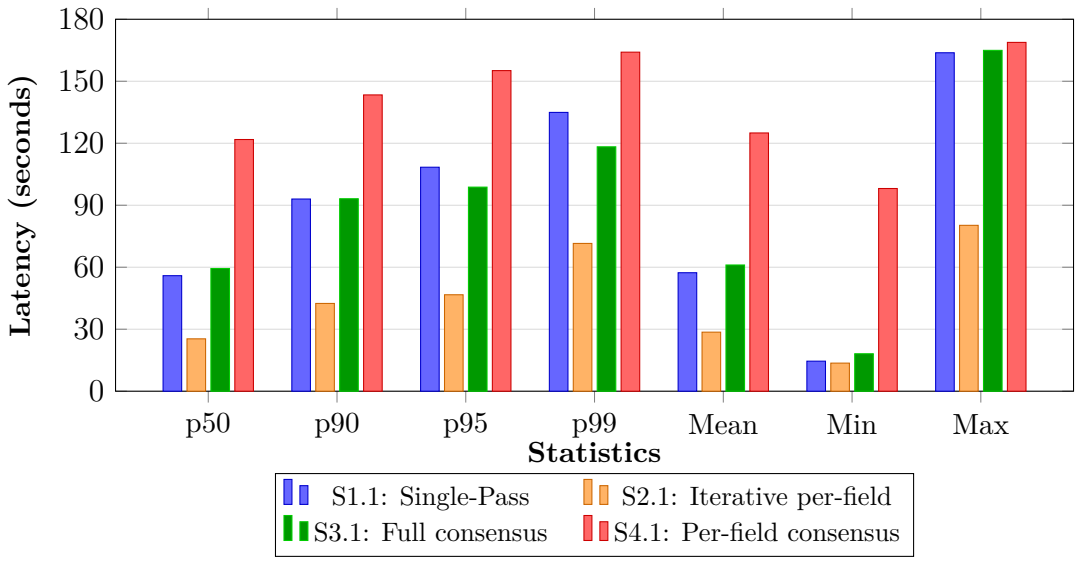

Figure 5.19.: Latency distribution across strategies (seconds).

Consistency Comparison

Figure 5.20 compares how well each strategy follows the schema format ($\text{FPR}_{\text{overall}}$) and maintains consistent writing style (SC_{macro}). The baseline S5 (LangExtract) has

perfect format compliance (1.000) because it uses strict validation rules. Among Invox strategies, S1.1 has the highest format score (0.960). For style consistency, S1.1 and S3.1 both reach 0.724, slightly higher than S2.1 (0.683) and S4.1 (0.688).

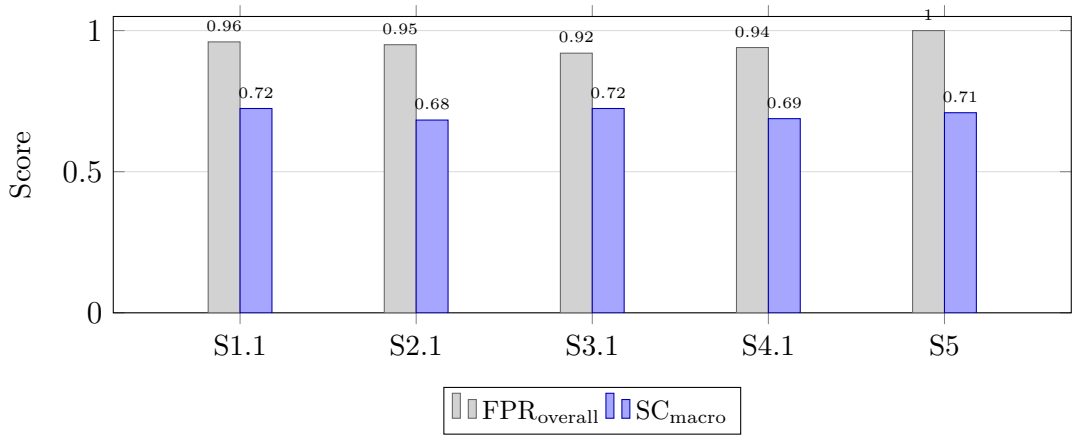

Figure 5.20.: Format compliance and style consistency across strategies.

Observed Trade-offs

The four strategies show clear patterns:

S1.1 (Single-LLM Full-Input) achieves the best overall scores (OBS = 0.644), balancing accuracy, speed (median 55.91s), cost (0.72¢), and lowest hallucination (0.180).

S2.1 (Single-LLM One-Field) is fastest (median 25.35s) through parallel processing. Accuracy is slightly lower (OBS = 0.619) with higher hallucination (0.301), but improves location fields.

S3.1 (Multi-LLM Full-Input) runs two models, reducing hallucination (HR = 0.208) and improving NES (0.521 vs. 0.504). Speed and cost fall between S1.1 and S2.1.

S4.1 (Multi-LLM One-Field) runs two models per field, achieving highest NES (0.624) and lowest missing rate (0.144), but highest hallucination (0.476). It is slowest (median 121.81s) and most expensive (4.32¢).

The main trade-off is filling correctly when values exist (NES, MR) versus abstaining when absent (HR, EAI). S1.1 and S3.1 balance both. S2.1 prioritizes speed. S4.1 prioritizes recall over precision.

6. Conclusion

This chapter summarizes the system developed in this thesis, its empirical findings, limitations, and implications for practice. The aim was to design and evaluate a modular, multi-agent system that converts unstructured speech and text into structured templates. Compared with single end-to-end models, the proposed architecture improves accuracy, transparency, adaptability, and debuggability.

6.1. Summary of Contributions

Invox: a modular multi-agent pipeline. This thesis presents **Invox**, a five-agent system covering Speech-to-Text (STT), Retrieval-Augmented Generation (RAG), Information Extraction (IE), Consistency Formatting (CF), and Verification (VER). Clear separation of roles reduces brittleness, localizes failures, and allows targeted upgrades (e.g., swapping IE or strengthening VER) without redesigning the pipeline. Templates are defined declaratively, enabling domain transfer.

Four orchestration strategies. Four strategies balance cost, speed, and robustness: S1 (single pass), S2 (per-field extraction), S3 (multi-model consensus on full input), and S4 (per-field consensus). These match different deployment needs: S1 for cost efficiency, S2 for low latency, S3 for stable consensus outputs, and S4 for maximum recall when fields are expected.

Evaluation focused on meaning and decisions. Exact match was complemented with embedding-based scores (Soft-F1) and behavioral metrics: fill-decision accuracy, hallucination rate, missing rate, and required-fill accuracy. These reflect both semantic correctness and system decision quality.

Cost and latency profiling. Each strategy was profiled on MUC-4 (N = 100), reporting median latency and per-document cost to guide deployment trade-offs.

Requirements-driven design. Six requirements (R1–R6) shaped the system: consistency, extraction quality, transparency, user correction, learning/adaptation, and usability.

6.2. Key Findings

No single strategy dominates. On MUC-4, S1.1 offered the best overall balance (OBS = 0.644; HR = 0.180; FDA = 0.746). S2.1 was fastest (25.35 s) with strong accuracy (OBS = 0.619). S3.1 provided consensus-stable outputs (OBS = 0.641; FDA = 0.743) with lower hallucination. S4.1 produced the highest quality on nonempty fields (NES = 0.624; MR = 0.144) but overfilled empty slots (HR = 0.476), requiring post-filtering. Strategy selection therefore depends on latency, cost, and risk constraints.

RAG consistently improves performance. Across S1–S4, adding retrieved few-shot examples improved accuracy by 3–4 points and reduced hallucination by 5–7 points, especially for entity-heavy fields (e.g., `perpetratorOrganization`, `weapon`).

Speech-style inputs remain robust. Speech-like transcripts caused only minor degradation. S1.2 preserved 97% of S1.1's OBS (0.626 vs. 0.644). Consensus strategies remained stable; per-field approaches did not gain a consistent advantage. This suggests resilience to lexical and structural variability even without explicit ASR noise.

Embedding metrics capture meaning better than exact match. Soft-F1 credited near-synonyms ("guerrillas" vs. "rebel forces") missed by exact match. S1.1 and S3.1 aligned best with semantic gold values. The Empty Advantage Index (EAI = OBS - NES) separated content quality from abstention: S1.1 showed balanced abstention; S4.1's negative EAI reflected excellent quality on nonempty fields but aggressive filling.

Cost and latency scale with complexity. Cost per document ranked S1 < S3 < S2 < S4 ($0.0072–$0.0432). Latency followed the same order. Whisper costs scaled predictably by duration. Field-wise, `perpetratorOrganization` and `weapon` scored highest (0.7–0.8), while `perpetratorIndividual` lagged (0.56–0.58). Per-field strategies excelled on location fields.

6.3. Limitations

Scope and generalizability. Results rely on MUC-4, limiting generalization to domains such as healthcare or manufacturing. Speech-style inputs do not model real ASR noise or confidence distributions.

Statistical depth. N = 100 with no confidence intervals, multiple runs, or significance tests; findings should be interpreted directionally.

Methodological gaps. RAG benefits were isolated, but CF and VER contributions were not. Embedding metrics can conflate related concepts; no human evaluation was conducted.

Operational and ethical readiness. The system was not load-tested, nor evaluated for security, compliance, or adversarial robustness. Fairness aspects (accent, dialect, gender) remain unexamined.

6.4. Implications for Practice

When to use a modular pipeline. Modularity is advantageous when templates contain many heterogeneous fields, when traceability is required, or when vocabularies evolve. Agent-level separation simplifies maintenance and upgrades.

When a single model is enough. Small templates (<5 fields), low correction needs, and strict latency/cost constraints may favor a single-model setup.

Choosing a strategy. For <30 s latency: choose S2. For <\$0.01 per document: choose S1. For safety-critical robustness: choose S3. For maximum recall on required fields: choose S4 with empty-slot filtering.

Pre-production checklist. Validate schemas; seed RAG with ≥ 100 examples; monitor low-confidence and high-disagreement cases; train users briefly on evidence/-correction flows; implement manual fallback; ensure compliance; monitor latency and cost.

Cost illustration. A 15-minute form at \$25/hour costs \$1,250/month for 200 reports. Invox S1 adds only $\approx$ \$2.60 plus $\approx$ \$50 infrastructure, suggesting $\approx \$1,197$ monthly savings (excluding setup and oversight).

6.5. Conclusion

Modular multi-agent systems offer transparency, adaptability, and maintainability that single end-to-end models cannot match. Invox evaluations show that S1 optimizes cost, S2 optimizes speed, S3 improves robustness via consensus, and S4 maximizes recall when fields are present. Embedding-based evaluation better captures semantic correctness, especially for paraphrased content. Such systems are intended to support, not replace, expert judgment—reducing repetitive entry, surfacing evidence, and enabling users to focus on higher-level decisions.

7. Future Work

Building on the contributions and limits above, several next steps stand out.

Real-World Speech Validation and Domain Expansion. Evaluate the full pipeline on real audio from target settings (factory reports, clinical handovers, mixed-language speech). Collect at least 500 annotated clips across domains and audio conditions. Bucket by WER and measure effects on extraction (F1, NES, HR/MR). Calibrate Whisper confidence vs. downstream quality. Include a controlled error study (substitutions, deletions, insertions). Use evidence-span annotation with agreement checks and ensure GDPR-compliant handling.

User Studies and Human-in-the-Loop Evaluation. Run a 4–6 week study with 10–15 trained users and counterbalanced tasks. Log time to complete, correction rates/types, and VER confidence; collect SUS and NASA-TLX pre/post. Analyze with paired tests or mixed-effects; report effect sizes and CIs. Feed correction logs back: update retrieval, refine prompts, tighten schema rules.

Adaptive Strategy Selection and Continuous Learning. Since no single strategy is best (see Sec. 5.9), train a router that picks a strategy using text features (length, diversity, sentence count), schema difficulty, role/urgency, and predicted cost/latency. Start with gradient-boosted trees on past (*input*, *strategy*, *outcome*) tuples. Evaluate offline, then A/B test vs. the best fixed strategy at equal cost. For continuous improvement: review low-confidence or high-disagreement cases; after corrections, refresh retrieval, refine prompts, tighten validators; periodically fine-tune domain adapters. Target a 20–30% correction drop over 12 weeks.

Technical Enhancements.

- **Ablations:** compare with/without retrieval and alternative validation; report ΔF1, Δlatency, Δcost, plus RFA, HR, MR.
- **Explainability:** return (*value*, *evidence_span*, *confidence*); highlight spans and add brief "Why?" notes; include a residual-info channel.
- **Cost optimization:** distill the strongest setup into a smaller model for routine use; compare accuracy at matched cost.
- **Multimodal:** add image/video/telemetry alongside audio/text; time-align signals and evaluate on inspection photos and imaging reports.

A. Detailed Implementation Results

A.1. Consistency Formatting (CF) Agent

Listing A.1 shows the main helper functions used by the CF agent for deterministic normalization. These correspond to the transformation rules described in Section 4.3.4.

Listing A.1: CF agent transformation function signatures

```
function normalizeDate(value: string): string | null;

function normalizeNumber(value: string): number | null;

function validateEnum(v: string, o: string[]): string | null;

function joinMultiValue(values: any[]): string | null;

function normalizeText(value: string): string | null;
```

A.2. IE Agent Supporting Types

Listing A.2 defines the supporting types used by the IE agent to represent current and filled field values.

Listing A.2: Supporting types for the IE agent

```
interface CurrentFieldValue {
  value: any;
  locked?: boolean;
  source?: "ai" | "manual";
}

interface FilledField {
  value: any;
  changed: boolean;
  previousValue?: any;
```

```
  source: "ai" | "manual";
  confidence?: number;
}
```

A.3. Verification Agent API

Listing A.3 provides the TypeScript interfaces for the verification agent, as described conceptually in Section 4.3.5.

Listing A.3: Verification agent API contract

```
interface VerificationInput {
  cand: Record<string, FilledField> | Record<string, FilledField>[];
  transcript: string;
  fields: FormTemplateField[];
  currentValues?: Record<string, CurrentFieldValue>;
}

interface VerificationOutput {
  filled: Record<string, FilledField>;
  confidence: Record<string, number>;
  issues?: Array<{
    field: string;
    type: "missing" | "conflict" | "low_conf" | "invalid";
    detail: string;
    action?: "requery" | "clarify" | "manual_review";
  }>;
  decisions?: Array<{
    field: string;
    decision: "gpt" | "gemini" | "merge" | "keep_current";
    reason: string;
  }>;
}
```

A.4. Orchestrator Implementation

Listing A.4 presents the core orchestration logic as implemented in the `Orchestrator` class. This corresponds to the conceptual pipeline described in Section 4.3.6.

Listing A.4: Core orchestration logic of the Invox pipeline

```
class Orchestrator {
```

```
async processTemplate(in: ProcessingRequest): Promise<FinalTemplate>
  // Step 1: Transcription (optional)
  const transcript = in.audio
    ? await stt.transcribe(in.audio)
    : { transcript: in.text, language: in.lang ?? "en" };

  // Step 2: Retrieval (RAG)
  const examples = await rag.retrieve(
    transcript.transcript,
    in.fields,
    in.templateId
  );

  // Step 3: Extraction (strategy-dependent)
  const extraction = await this.runStrategy({
    strategy: in.strategy,
    transcript: transcript.transcript,
    fields: in.fields,
    currentValues: in.currentValues,
    fewShots: examples.examples,
    ...in.metadata,
  });

  // Step 4: Formatting (deterministic normalization)
  const norm = await cf.normalize(extraction.filled, in.fields);

  // Step 5: Verification (completeness + consistency)
  const verified = await ver.verify({
    candidates: norm,
    transcript: transcript.transcript,
    fields: in.fields,
    currentValues: in.currentValues,
  });

  // Step 6: Clarification loop (if needed)
  if (verified.issues?.some(i => i.action === "requery")) {
    // Re-run IE with hints from VER, then re-verify
  }

  return {
    filled: verified.filled,
    confidence: verified.confidence,
    issues: verified.issues,
    model: extraction.model,
```

```
      transcript: transcript.transcript,
    };
  }

  private async runStrategy(p: StrategyParams): Promise<IEOutput> {
    switch (p.strategy) {
      case "S1": return await singleLlmAllField(p);
      case "S2": return await singleLlmOneField(p);
      case "S3": return await dualLlmAllField(p);
      case "S4": return await multiLlmOneField(p);
    }
  }
}
```

A.5. Strategy Function Implementations

This section provides simplified TypeScript sketches of the four strategy functions and the strategy-level orchestrator described in Section 4.4. The listings are not intended as production-ready code, but as an executable illustration of the control flow.

S1: `singleLlmAllField`

Listing A.5: Core S1 implementation sketch (`singleLlmAllField`)

```
export async function singleLlmAllField(
  input: GetFilledTemplateInput
): Promise<GetFilledTemplateResult> {
  const {
    oldText,
    newText,
    fields,
    currentValues,
    modelName,
    transcriptData,
  } = input;

  // 1. Combine transcripts
  const combTrans = oldText ? `${oldText}\n${newText}` : newText;

  // 2. Retrieve few-shot examples
```

```
  const fewShots = await getFewShotsFromTranscript(
    combinedTranscript,
    fields,
    3
  );

  // 3. Single LLM call for all fields
  const schema = z.object(/* comprehensive field schema */);
  const result = await generateObject({
    model: openai(modelName),
    schema,
    prompt: buildComprehensivePrompt(
      fields,
      oldText,
      newText,
      fewShots
    ),
  });

  // 4. Process results
  const filled = processAllFields(
    result.object,
    fields,
    currentValues
  );
  const verified = await runVerifier(
    combTrans,
    fields,
    filled
  );

  return {
    filled: verified,
    model: modelName,
    transcript: transcriptData,
  };
}
```

S2: `singleLlmOneField`

Listing A.6: Core S2 implementation sketch (`singleLlmOneField`)

```
export async function singleLlmOneField(
```

```
  input: GetFilledTemplateInput
): Promise<GetFilledTemplateResult> {
  const {
    oldText,
    newText,
    fields,
    currentValues,
    modelName,
    transcriptData,
  } = input;

  const combTrans = oldText ? `${oldText}\n${newText}` : newText;

  // Retrieve few-shot examples once
  const fewShots = await getFewShotsFromTranscript(
    combTrans,
    fields,
    3
  );

  // Parallel field processing with individual error boundaries
  const tasks = fields.map(field =>
    runField({
      field,
      oldText,
      newText,
      fewShots,
      current: currentValues?.[field.id],
      modelName,
    }).catch(error => {
      // Field-level error recovery
      return [field.id, getFallbackField(field, currentValues)];
    })
  );

  const entries = await Promise.all(tasks);

  return {
    filled: Object.fromEntries(entries),
    model: modelName,
    transcript: transcriptData,
  };
}
```

S3: dualLlmAllField

Listing A.7: Core S3 implementation sketch (dualLlmAllField)

```
export async function dualLlmAllField(
  input: GetFilledTemplateInput
): Promise<GetFilledTemplateResult> {
  const {
    oldText,
    newText,
    fields,
    currentValues,
    gptModel,
    geminiModel,
    transcriptData,
  } = input;

  const combTrans = oldText ? `${oldText}\n${newText}` : newText;
  const fewShots = await getFewShotsFromTranscript(
    combTrans,
    fields,
    3
  );

  const schema = z.object(/* comprehensive field schema */);
  const prompt = buildComprehensivePrompt(
    fields,
    oldText,
    newText,
    fewShots
  );

  // Parallel model execution
  const [gptResult, geminiResult] = await Promise.all([
    generateObject({ model: openai(gptModel), schema, prompt }),
    generateObject({ model: google(geminiModel), schema, prompt }),
  ]);

  // Build candidates
  const gptCandidate = buildCandidate(
    gptResult.object,
    fields,
    currentValues
  );
```

```
  const geminiCandidate = buildCandidate(
    geminiResult.object,
    fields,
    currentValues
  );

  // Consensus verification
  const verified = await runEnsembleVerifier({
    combTrans,
    fields,
    currentValues,
    gpt: gptCandidate,
    gemini: geminiCandidate,
  });

  return {
    filled: verified.filled,
    model: `ensemble:${gptModel}+${geminiModel}`,
    transcript: transcriptData,
  };
}
```

S4: `multiLlmOneField`

Listing A.8: Core S4 implementation sketch (`multiLlmOneField`)

```
export async function multiLlmOneField(
  input: GetFilledTemplateInput
): Promise<GetFilledTemplateResult> {
  const {
    oldText,
    newText,
    fields,
    currentValues,
    gptModel,
    geminiModel,
    verifierModel,
    transcriptData,
  } = input;

  const combTrans = oldText ? `${oldText}\n${newText}` : newText;
  const fewShots = await getFewShotsFromTranscript(
    combTrans,
```

```
    fields ,
    3
  );

  const results: [string, FilledField][] = [];

  // Sequential field processing (rate limit avoidance)
  for (const field of fields) {
    const entry = await runOneFieldWithEnsemble({
      field ,
      oldText ,
      newText ,
      fewShots ,
      current: currentValues?.[field.id],
      gptModel ,
      geminiModel ,
      verifierModel ,
    });
    results.push(entry);
  }

  return {
    filled: Object.fromEntries(results),
    model: `ensemble-per-field:${gptModel}+${geminiModel}`,
    transcript: transcriptData ,
  };
}
```

StrategyOrchestrator Sketch

Listing A.9: Strategy-level orchestrator sketch

```
class StrategyOrchestrator {
  constructor(
    private readonly singlePass: SinglePassStrategy ,
    private readonly iterative: IterativeStrategy ,
    private readonly multiModelFull: MultiModelFullStrategy ,
    private readonly multiModelSlot: MultiModelSlotStrategy
  ) {}

  async executeStrategy(
    input: ProcessingRequest
  ): Promise<FinalTemplate> {
```

```
    const impl = this.getStrategyImplementation(input.strategy);
    return await impl.execute(input);
  }

  private getStrategyImplementation(
    strategy: Strategy
  ): StrategyImplementation {
    switch (strategy) {
      case "S1":
        return this.singlePass;
      case "S2":
        return this.iterative;
      case "S3":
        return this.multiModelFull;
      case "S4":
        return this.multiModelSlot;
      default:
        throw new Error(`Unknown strategy: ${strategy}`);
    }
  }
}
```

B. Detailed Experimental Results

B.1. LangExtract Baseline (S5)

This section reports detailed metrics for the external LangExtract baseline (Strategy S5) on the MUC-4 subset (N=100 documents). LangExtract is evaluated in the same regime as the proposed strategies, but it is not part of the multi-agent architecture described in this thesis.

Headline and Per-Field Scores

Table B.1.: Headline metrics for LangExtract on MUC-4 (N=100 documents).

OBS	NES	EAI	SF1 (base)	SF1 (nonempty)	FDA	HR	MR	RFA
0.684	0.502	0.182	0.653	0.406	0.756	0.088	0.369	0.796

Table B.2.: Per-field average scores for LangExtract (baseline regime; N=100).

Field	Average Score	#Docs
incidentType	0.880	100
incidentDate	0.570	100
incidentLocation	0.512	100
incidentStage	0.830	100
perpetratorIndividual	0.719	100
perpetratorOrganization	0.681	100
target	0.523	100
victim	0.596	100
weapon	0.844	100
Overall Average (OBS)	**0.684**	**900 comps**

Embedding Metrics

Table B.3.: Embedding metrics (overall) for LangExtract.

Regime	Soft-Coverage	Soft-Specificity	Chamfer (Sym.)
Baseline	0.646	0.661	0.653
Gold-nonempty-only	0.391	0.421	0.406

Table B.4.: Per-field embedding metrics for LangExtract (baseline; text/list fields).

Field	Soft-Coverage	Soft-Specificity	Chamfer
incidentLocation	0.512	0.512	0.512
perpetratorIndividual	0.719	0.730	0.724
perpetratorOrganization	0.681	0.679	0.680
target	0.523	0.531	0.527
victim	0.596	0.660	0.628
weapon	0.844	0.854	0.849

Latency and Consistency

Table B.5.: Latency statistics for LangExtract (seconds).

p50	p90	p95	p99	Mean	Min	Max
21.15	57.61	65.39	99.19	27.03	1.95	109.84

Table B.6.: Consistency (two-score) — schema format vs. style uniformity for LangExtract.

Strategy	$\text{FPR}_{\text{overall}} \uparrow$	$\text{SC}_{\text{macro}} \uparrow$
LangExtract (S5)	**1.000**	**0.709**

Note: $\text{FPR}_{\text{overall}}$ is schema-only (fields that are empty but allowed by the schema still count as format-preserving). SC_{macro} is an input-aware style score based on masked character 3-gram cosine similarity.

This appendix provides comprehensive numerical results for all strategy variants evaluated in this thesis. All experiments were conducted on the MUC-4 dataset with $N = 100$ samples.

B.2. Strategy 1: Single-Pass Full Input

Table B.7.: Headline metrics for S1 variants on MUC-4 (N=100).

Variant	OBS	NES	EAI	SF1	FDA	HR	MR	RFA
S1.0 (no few-shot, orig)	0.601	0.469	0.132	0.604	0.713	0.233	0.329	0.700
S1.1 (few-shot, orig)	**0.644**	0.504	**0.140**	**0.642**	**0.746**	**0.180**	0.313	0.733
S1.2 (few-shot, speech)	0.626	**0.508**	0.118	0.631	0.727	0.226	**0.311**	**0.738**

Table B.8.: Latency statistics for S1 variants (seconds).

Variant	p50	p90	p95	p99	Mean	Min	Max
S1.0 (no few-shot, orig)	25.04	46.49	51.82	66.12	28.17	6.51	78.65
S1.1 (few-shot, orig)	**55.91**	**93.01**	**108.40**	**134.93**	**57.34**	14.54	**163.79**
S1.2 (few-shot, speech)	49.36	69.30	77.84	86.79	47.89	**16.85**	102.50

B.3. Strategy 2: Iterative Single-Field Processing

Table B.9.: Headline metrics for S2 variants on MUC-4 (N=100).

Variant	OBS	NES	EAI	SF1	FDA	HR	MR	RFA
S2.0 (no few-shot, orig)	0.590	0.560	0.029	0.608	0.709	0.373	**0.226**	0.724
S2.1 (few-shot, orig)	**0.619**	**0.555**	**0.064**	**0.639**	**0.718**	**0.301**	0.267	**0.758**
S2.2 (few-shot, speech)	0.595	0.544	0.051	0.631	0.700	0.341	0.267	0.742

Table B.10.: Latency statistics for S2 variants (seconds).

Variant	p50	p90	p95	p99	Mean	Min	Max
S2.0 (no few-shot, orig)	19.97	32.24	37.69	49.26	21.48	9.01	53.76
S2.1 (few-shot, orig)	25.35	42.48	46.68	71.52	28.58	13.60	80.28
S2.2 (few-shot, speech)	24.07	36.93	42.81	52.53	25.83	12.51	59.36

B.4. Strategy 3: Multi-LLM Consensus (Full Input)

Table B.11.: Headline metrics for S3 variants on MUC-4 (N=100).

Variant	OBS	NES	EAI	SF1	FDA	HR	MR	RFA
S3.0 (no few-shot, orig)	0.602	0.481	0.121	0.601	0.713	0.246	0.319	0.706
S3.1 (few-shot, orig)	**0.641**	0.521	0.120	0.638	**0.743**	**0.208**	0.295	**0.740**
S3.2 (few-shot, speech)	0.624	**0.524**	**0.100**	0.630	0.728	0.251	**0.289**	0.738

Table B.12.: Latency statistics for S3 variants (seconds).

Variant	p50	p90	p95	p99	Mean	Min	Max
S3.0 (no few-shot, orig)	31.47	57.79	63.02	87.16	34.83	14.85	87.24
S3.1 (few-shot, orig)	**59.39**	**93.15**	**98.77**	**118.28**	**61.09**	18.13	**164.96**
S3.2 (few-shot, speech)	53.45	84.15	93.31	104.99	54.55	**22.01**	113.26

B.5. Strategy 4: Multi-LLM Consensus (Iterative)

Table B.13.: Headline metrics for S4 variants on MUC-4 (N=100).

Variant	OBS	NES	EAI	SF1	FDA	HR	MR	RFA
S4.0 (no few-shot, orig)	0.549	**0.628**	**-0.080**	0.589	0.693	**0.551**	**0.112**	0.708
S4.1 (few-shot, orig)	**0.579**	0.624	-0.044	**0.613**	**0.709**	0.476	0.144	**0.728**
S4.2 (few-shot, speech)	0.572	0.612	-0.040	0.592	0.704	0.479	0.150	0.719

Table B.14.: Latency statistics for S4 variants (seconds).

Variant	p50	p90	p95	p99	Mean	Min	Max
S4.0 (no few-shot, orig)	125.14	157.96	166.02	177.27	128.64	97.08	179.00
S4.1 (few-shot, orig)	121.81	143.39	155.16	164.08	124.98	98.10	168.81
S4.2 (few-shot, speech)	**127.23**	**159.02**	**175.50**	**192.16**	**131.15**	**84.42**	**218.22**

B.6. Strategy Comparison Summary

Table B.15.: Macro comparison (clean text; few-shot original).

Strategy	OBS	NES	EAI	SF1	FDA	HR	MR
S1.1: Single-Pass	**0.644**	0.504	0.140	**0.642**	**0.746**	**0.180**	0.313
S2.1: Iterative per-field	0.619	**0.555**	**0.064**	0.639	0.718	0.301	0.267
S3.1: Full consensus	0.641	0.521	0.120	0.638	0.743	0.208	0.295
S4.1: Per-field consensus	0.579	**0.624**	**-0.044**	0.613	0.709	0.476	**0.144**

Table B.16.: Latency profile (seconds; clean text).

Strategy	p50	p90	p95	p99	Mean	Min	Max
S1.1: Single-Pass	55.91	93.01	108.40	134.93	57.34	14.54	163.79
S2.1: Iterative per-field	**25.35**	42.48	46.68	71.52	28.58	13.60	80.28
S3.1: Full consensus	59.39	93.15	98.77	118.28	61.09	18.13	164.96
S4.1: Per-field consensus	121.81	143.39	155.16	164.08	124.98	98.10	168.81

Costs Comparison (Tabular)

Table B.17.: Per-document cost (¢) vs. audio duration D (minutes). Totals follow $\text{cost}(D) = \text{base} + 0.60D$.

Strategy	D=0	D=1	D=2	D=5	D=10
S1.1 (Single-pass)	0.720	1.320	1.920	3.720	6.720
S2.1 (Iterative per-field)	2.183	2.783	3.383	5.183	8.183
S3.1 (Full consensus)	1.395	1.995	2.595	4.395	7.395
S4.1 (Per-field consensus)	4.320	4.920	5.520	7.320	10.320

Table B.18.: Cost decomposition at D=1 minute (¢ per document).

Strategy	LLM (base)	Whisper (1 min)	Total
S1.1 (Single-pass)	0.720	0.600	1.320
S2.1 (Iterative per-field)	2.183	0.600	2.783
S3.1 (Full consensus)	1.395	0.600	1.995
S4.1 (Per-field consensus)	4.320	0.600	4.920

Consistency Metrics (Formatting & Style)

Table B.19.: Consistency metrics for S1 variants on MUC-4 (N=100).

Variant	$\text{FPR}_{\text{overall}}$	$\text{FPR}_{\text{macro}}$	SC_{macro}
S1.0 (no few-shot, orig)	0.910	0.9889	0.6919
S1.1 (few-shot, orig)	**0.960**	**0.9956**	**0.7239**
S1.2 (few-shot, speech)	0.930	0.9911	0.7002

Table B.20.: Consistency metrics for S2 variants on MUC-4 (N=100).

Variant	$\text{FPR}_{\text{overall}}$	$\text{FPR}_{\text{macro}}$	SC_{macro}
S2.0 (no few-shot, orig)	**0.970**	**0.9967**	0.6718
S2.1 (few-shot, orig)	0.950	0.9944	**0.6828**
S2.2 (few-shot, speech)	0.940	0.9933	0.6655

Table B.21.: Consistency metrics for S3 variants on MUC-4 (N=100).

Variant	$\text{FPR}_{\text{overall}}$	$\text{FPR}_{\text{macro}}$	SC_{macro}
S3.0 (no few-shot, orig)	0.910	0.9889	0.6907
S3.1 (few-shot, orig)	**0.920**	**0.9911**	**0.7243**
S3.2 (few-shot, speech)	0.910	0.9889	0.7022

Table B.22.: Consistency metrics for S4 variants on MUC-4 (N=100).

Variant	$\text{FPR}_{\text{overall}}$	$\text{FPR}_{\text{macro}}$	SC_{macro}
S4.0 (no few-shot, orig)	0.870	0.9856	0.6599
S4.1 (few-shot, orig)	**0.940**	**0.9933**	**0.6881**
S4.2 (few-shot, speech)	**0.940**	**0.9933**	0.6815

Bibliography

[1] Amershi, S., Weld, D.S., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S.T., Bennett, P.N., Inkpen, K., Teevan, J., Kikin-Gil, D., Horvitz, E.: Guidelines for human-ai interaction. In: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. pp. 1–13. ACM (2019)

[2] Anderson, P., Fernando, B., Johnson, M., Gould, S.: Guided open vocabulary image captioning with constrained beam search. In: Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 936–945. Association for Computational Linguistics (2017)

[3] Baldwin, T., Chai, K.J.: Noisy text processing. In: Proceedings of the 23rd International Conference on Computational Linguistics: Tutorial Abstracts. pp. 1–3. ACL (2013)

[4] Bastianelli, E., Vanzo, A., Swietojanski, P., Rieser, V., Lemon, O.: Slurp: A spoken language understanding resource package. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 7252–7262. Association for Computational Linguistics (2020)

[5] Bohus, D., Rudnicky, A.I.: Sorry, i didn't catch that! an investigation of non-understanding errors and recovery strategies. In: Proceedings of SIGdial Workshop on Discourse and Dialogue (2005)

[6] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems. vol. 33, pp. 1877–1901 (2020)

[7] Chen, F.K., Liu, C.H., You, S.D.: Using large language model to fill in web forms to support automated web application testing. Information 16(2), 102 (2024)

[8] Chen, F.K., Liu, C.H., You, S.D.: Using large language model to fill in web forms to support automated web application testing. Information 16(2), 102 (2024)

[9] Chen, G., Dong, S., Shu, Y., Zhang, G., Sesay, J., Karlsson, B.F., Fu, J., Shi, Y.: Autoagents: A framework for automatic agent generation. arXiv preprint arXiv:2309.17288 (2023), submitted to IJCAI 2024

[10] Chinchor, N.: Muc-4 evaluation metrics. In: Proceedings of the 4th Message Understanding Conference (MUC-4). pp. 22–29 (1992), https://www.semanticscholar.org/paper/3dcda1bec36586b46b1dc67a477beca2c5a105be

[11] Chinchor, N.: Muc-4 evaluation metrics. In: Proceedings of the 4th Message Understanding Conference (MUC-4). pp. 22–29. Association for Computational Linguistics (1992)

[12] Clark, J.: Designing Accessible Voice Interfaces. O'Reilly Media (2020)

[13] Doddington, G., Mitchell, A., Przybocki, M., Ramshaw, L., Strassel, S., Weischedel, R.: The automatic content extraction (ace) program – tasks, data, and evaluation. In: Proceedings of the 4th International Conference on Language Resources and Evaluation (LREC). pp. 837–840. European Language Resources Association (2004)

[14] Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608 (2017)

[15] Du, X., Cardie, C.: Event extraction by answering (almost) natural questions (2020)

[16] Du, X., Rush, A.M.: Template filling with generative transformers. ACL Anthology (2021), https://www.semanticscholar.org/paper/Template-Filling-with-Generative-Transformers-Du-Rush/e9b3fed266dff06d44be861cce8bd9c247b7d42a

[17] Fathullah, Y., Wu, C., Lakomkin, E., Jia, J., Shangguan, Y., et al.: Prompting large language models with speech recognition abilities (2023), arXiv preprint arXiv:2307.11795

[18] Fowler, M.: Monolithfirst. https://martinfowler.com/bliki/MonolithFirst.html (2015)

[19] Friedman, C., Hripcsak, G., DuMouchel, W., Johnson, S.B., Clayton, P.D.: Natural language processing in an operational clinical information system. Natural Language Engineering 10(3-4), 327–349 (2004)

[20] Gatt, A., Krahmer, E.: Survey of the state of the art in natural language generation: Core tasks, applications and evaluation. Journal of Artificial Intelligence Research 61, 65–170 (2018)

[21] Google Research: Langextract: Schema-based information extraction with large language models. https://github.com/google-research/langextract (2024), accessed: 2025-08-31

[22] Grishman, R.: Information extraction: Techniques and challenges. Information extraction: A multidisciplinary approach to an emerging information technology pp. 10–27 (1997)

[23] Hardt, D.: The oauth 2.0 authorization framework (rfc 6749). https://www.rfc-editor.org/rfc/rfc6749 (2012)

[24] Hoy, M.B.: Alexa, siri, cortana, and more: An introduction to voice assistants. Medical Reference Services Quarterly 37(1), 81–88 (2018)

[25] Huang, B., Chen, C., Shu, K.: Authorship attribution in the era of llms: Problems, methodologies, and challenges. SIGKDD Explorations 26(2), 21–43 (2024)

[26] Jonnalagadda, S.R., Gonzalez, G.: Medical concept normalization for clinical narratives using a domain-specific knowledge base. In: Proceedings of the 2010 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). pp. 167–172. IEEE (2010)

[27] Jurafsky, D., Martin, J.H.: Speech and Language Processing (3rd Edition draft) (2023), draft available at https://web.stanford.edu/~jurafsky/slp3/

[28] Jurafsky, D., Martin, J.H.: Speech and language processing (3rd edition draft). https://web.stanford.edu/~jurafsky/slp3/ (2023), draft textbook

[29] Le, N.K., Bui, Q.M., Nguyen, M.N., Nguyen, H., Vo, T., Luu, S.T., Nomura, S., Nguyen, M.L.: Automated web application testing: End-to-end test case generation with large language models and screen transition graphs. arXiv preprint arXiv:2506.02529 (2025), https://arxiv.org/abs/2506.02529

[30] Liang, Y., Qin, Y., Liu, Y., Ye, Y., Liu, Z., Sun, M., et al.: Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis. arXiv preprint arXiv:2303.16434 (2023)

[31] Lin, Z., Liu, B., Moon, S., Crook, P., Zhou, Z., Wang, Z., Yu, Z., Madotto, A., Cho, E., Subba, R.: Leveraging slot descriptions for zero-shot cross-domain dialogue state tracking. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. pp. 5640–5648 (2021)

[32] Liu, B., Hu, H., Yu, D., Zhang, Y., Williams, J.D.: Exploring conversational semantic parsing for task-oriented dialog systems. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 3129–3147 (2022)

[33] Mialon, G., Deleu, T., Dohan, D., et al.: Augmented language models: A survey. arXiv preprint arXiv:2302.07842 (2023)

[34] Newman, S.: Building Microservices. O'Reilly Media (2015)

[35] Norman, D.: The Design of Everyday Things. Basic Books (2013)

[36] Pan, W., Chen, Q., Xu, X., Che, W., Qin, L.: A preliminary evaluation of chatgpt for zero-shot dialogue understanding. arXiv preprint arXiv:2304.04256 (2023)

[37] Park, H.W., Lee, S., Kim, J.: Generative agents for document processing. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (2023)

[38] Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., Gao, J.: Check your facts and try again: Improving large language models with external knowledge and automated feedback. arXiv preprint arXiv:2302.12813 (2023)

[39] Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision. arXiv preprint arXiv:2212.04356 (2023), `https://cdn.openai.com/papers/whisper.pdf`

[40] Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. pp. 1135–1144 (2016)

[41] Samek, W., Montavon, G., Vedaldi, A., Hansen, L.K., Müller, K.R.: Explainable AI: Interpreting, explaining and visualizing deep learning. Springer Nature (2019)

[42] Sarawagi, S.: Information extraction. Foundations and Trends in Databases 1(3), 261–377 (2008)

[43] Schick, T., Dwivedi-Yu, J., Maini, P., Chadha, A., et al.: Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761 (2023)

[44] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. arXiv preprint arXiv:2303.17580 (2023)

[45] Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., Diakopoulos, N.: Designing the User Interface: Strategies for Effective Human-Computer Interaction. Pearson, 6 edn. (2016)

[46] Sun, G., Feng, S., Jiang, D., Zhang, C., Gašić, M., Woodland, P.C.: Speech-based slot filling using large language models (2023), arXiv preprint arXiv:2311.07418

[47] Thomas, C., Verma, S., Sen, S., et al.: Interacting with explanations: Understanding and visualizing model predictions with human feedback. In: Proceedings of IUI (2019)

[48] Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the conll-2003 shared task: Language-independent named entity recognition. In: Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003. pp. 142–147. Association for Computational Linguistics (2003)

[49] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems. vol. 30 (2017)

[50] Wang, B., Yu, D., Wu, Y., et al.: Spoken language understanding: A survey. IEEE/ACM Transactions on Audio, Speech, and Language Processing 29, 1135–1152 (2021)

[51] Wang, B., Yu, D., Wu, Y.: Spoken language understanding: A survey. IEEE/ACM Transactions on Audio, Speech, and Language Processing 29, 1135–1152 (2021)

[52] Wang, Y., Siau, K.: Trust and transparency in ai: a systematic review. Journal of Strategic Information Systems 31(1), 101717 (2022)

[53] Wang, Z., Zhao, Z., Lyu, Y., Chen, Z., de Rijke, M., Ren, Z.: A cooperative multi-agent framework for zero-shot named entity recognition. In: Proceedings of the ACM Web Conference 2025 (WWW '25). p. 13. ACM (2025)

[54] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., et al.: Emergent abilities of large language models. Transactions on Machine Learning Research (2022)

[55] Wu, Z., Li, X., Zhang, J., et al.: Autoagents: An open-source framework for building multi-agent systems with large language models. arXiv preprint arXiv:2308.08155 (2023)

[56] Ye, X., Zhou, W., Du, J., Zhang, X., Chen, M.: Guided and knowledgeable multi-agent debate for fact verification. In: Proceedings of the AAAI Conference on Artificial Intelligence (2025)

[57] Zhang, X., Peng, B., Li, K., Zhou, J., Meng, H.: Sgp-tod: Building task bots effortlessly via schema-guided llm prompting. arXiv preprint arXiv:2305.09067 (2023)